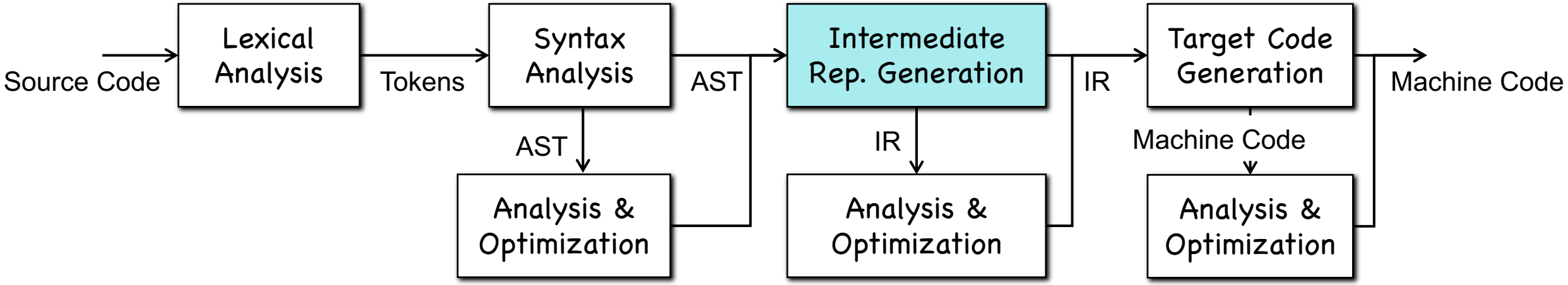
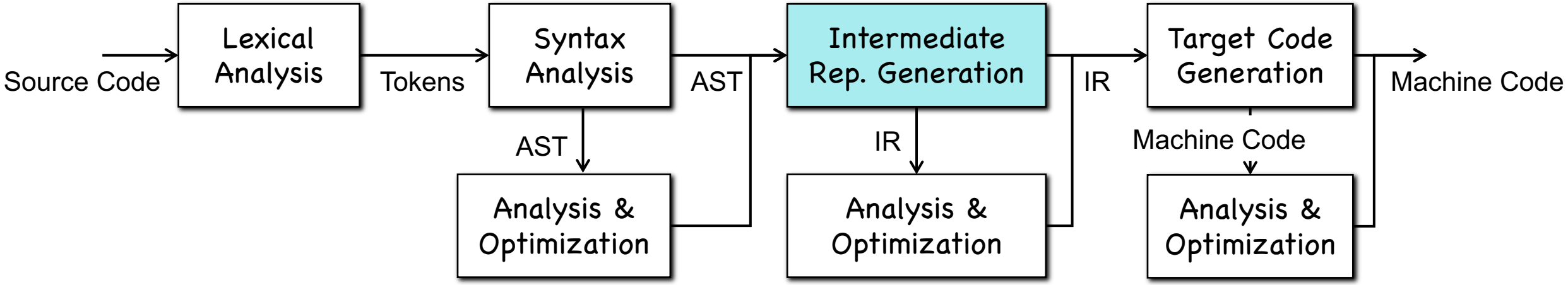
Made by Qingkai Shi
qingkaishi@nju.edu.cn
南京大学编译技术
及软件安全研究组

Chapter 6-1
Intermediate Representation (IR)

Intermediate Representation

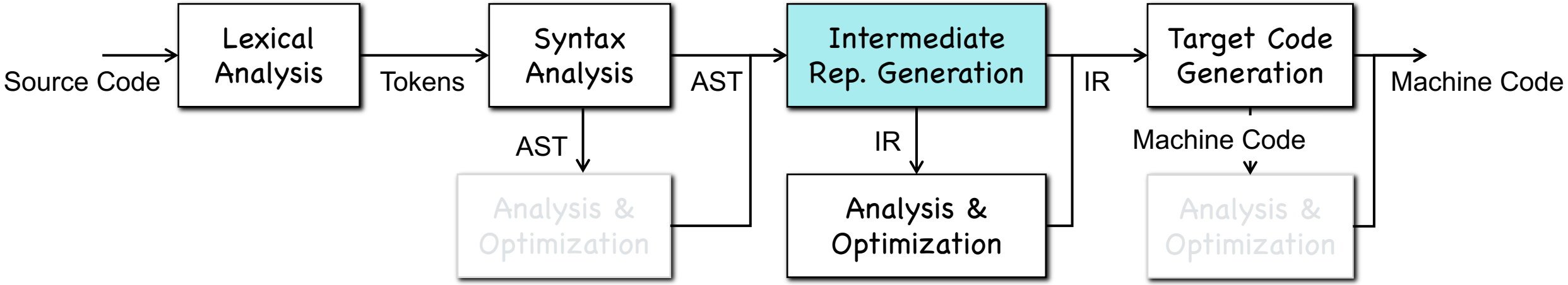


Intermediate Representation



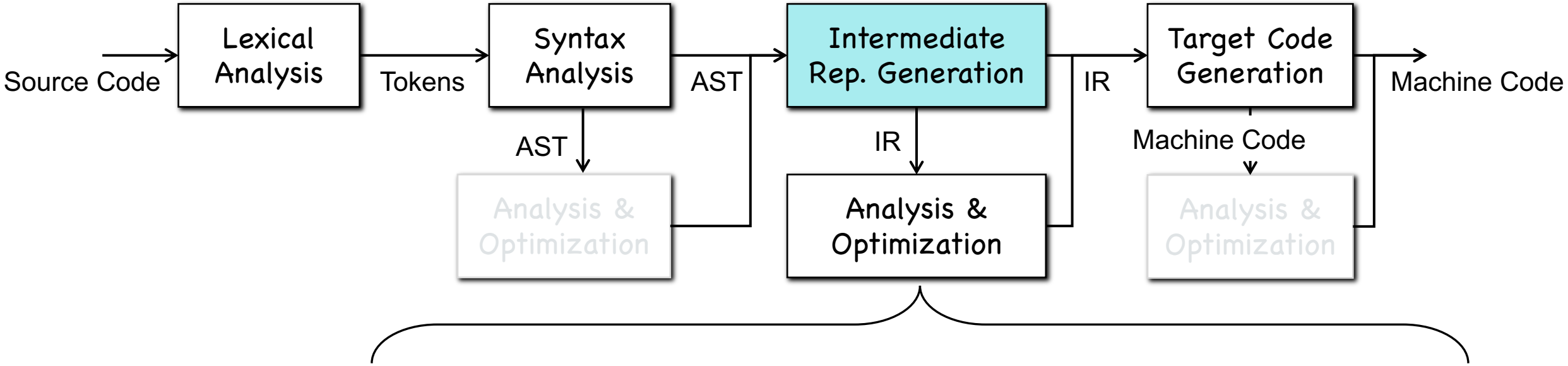
- Using *syntax-directed translation* to generate IR (**Ch 6**)

Intermediate Representation



- Using *syntax-directed translation* to generate IR (**Ch 6**)
- Intermediate rep. is friendly to analysis and optimization (**Ch 9**)
  - Better structure, including control flow graph, etc.
  - Rich source code information, including types, etc.

Intermediate Representation



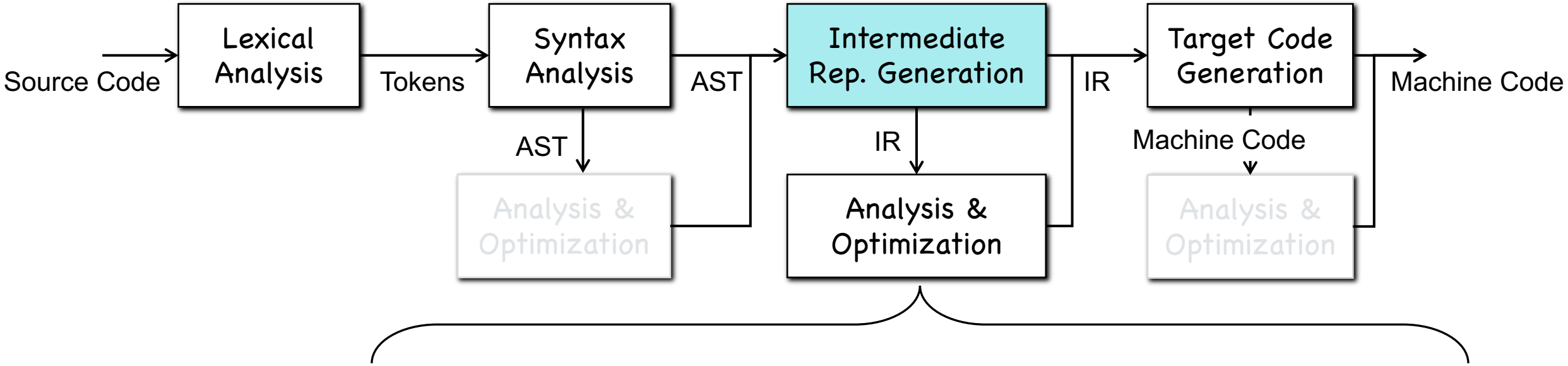
Machine Independent Analysis

1. Type inference
2. Bug detection
3. Code instrumentation
   a) logging behaviors
   b) defending attacks

Machine Independent Optimization

1. Dead code elimination
2. Constant propagation
3. Live variable analysis
4. Vectorization
5. …

Intermediate Representation



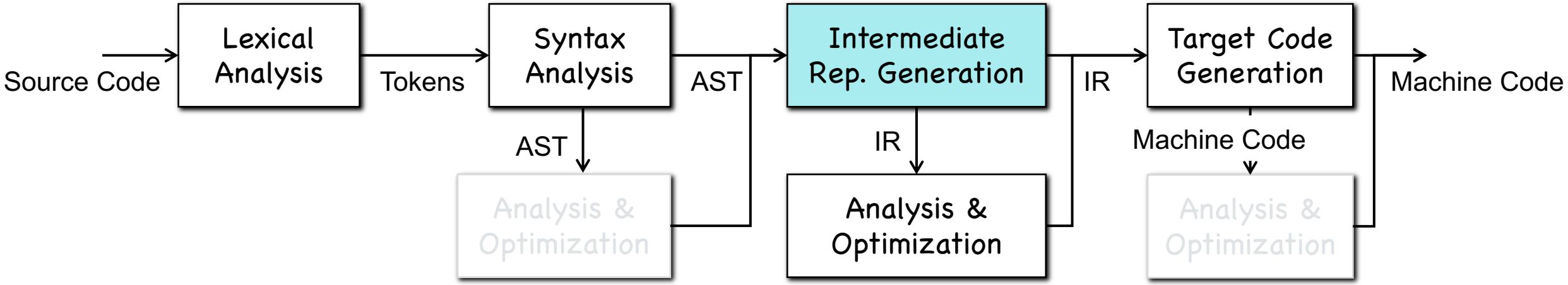
Machine Independent Analysis

1. Type inference
2. Bug detection
3. Code instrumentation
   a) logging behaviors
   b) defending attacks

Machine Independent Optimization

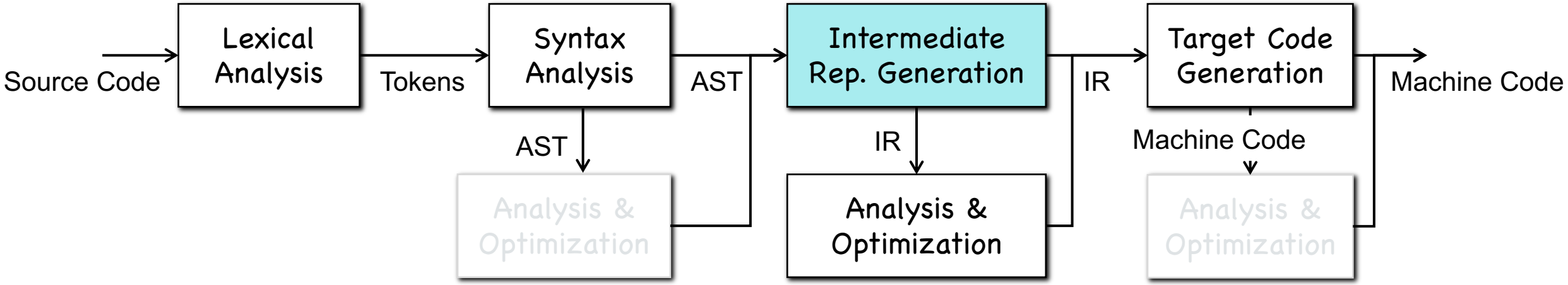
1. Dead code elimination
2. Constant propagation
3. Live variable analysis
4. Vectorization
5. …

Intermediate Representation




```
char get(char *buffer, int size, int index) {
    return buffer[index];
}
```

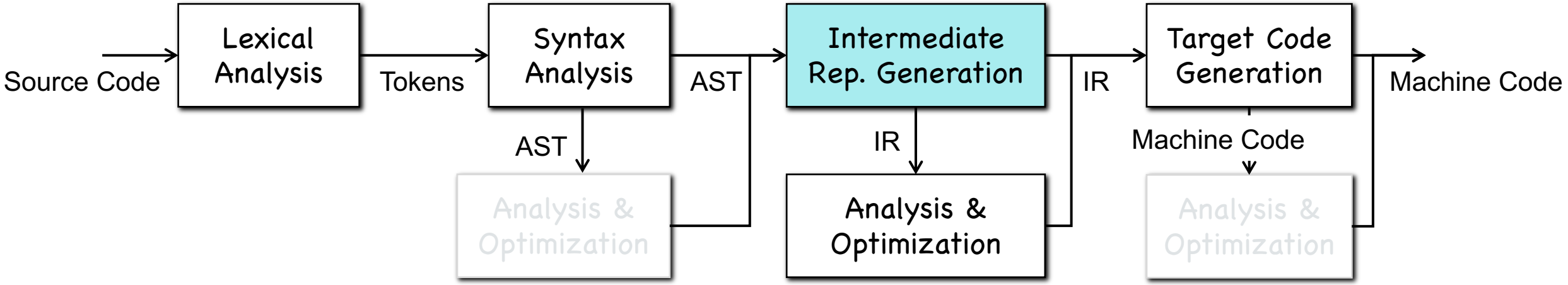
Intermediate Representation




```
char get(char *buffer, int size, int index) {
    return buffer[index];
}
```

```
char get(char *buffer, int size, int index) {
    // if (index >= size) ……
    return buffer[index];
}
```

Intermediate Representation




```
char get(char *buffer, int size, int index) {
    return buffer[index];
}
```

```
char get(char *buffer, int size, int index) {
    // if (index >= size) ……
    return buffer[index];
}
```

CVE-2022-26125 Detail

> 1000 Bugs, > 100 CVEs in mature systems, e.g., Apache, MySQL, etc.

Description

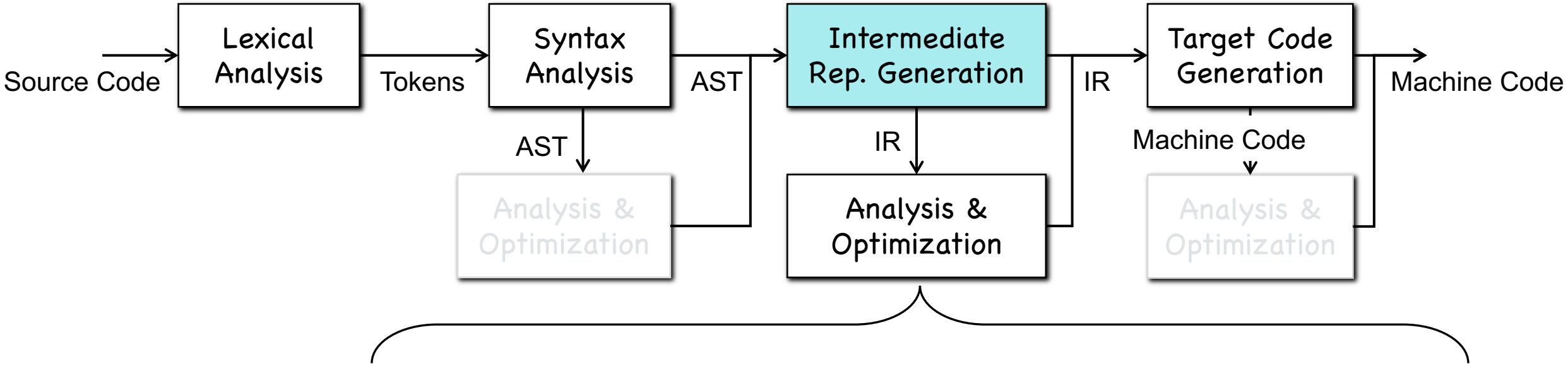
Buffer overflow vulnerabilities exist in FRRouting through 8.1.0 due to wrong checks on the input packet length in isisd/isis_tlvs.c.

Severity CVSS Version 3.x | CVSS Version 2.0

CVSS 3.x Severity and Metrics:

NIST: NVD **Base Score:** 7.8 HIGH **Vector:** CVSS:3.1/AV:L/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H

Intermediate Representation



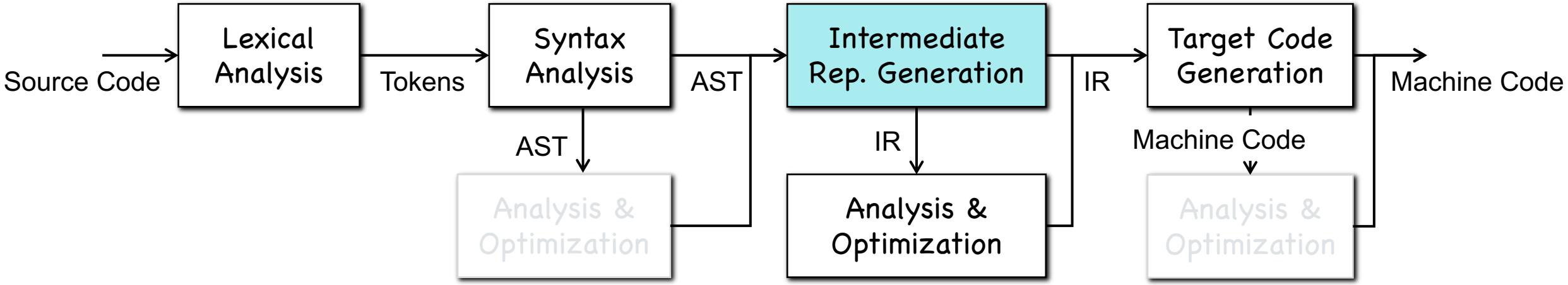
Machine Independent Analysis

1. Type inference
2. Bug detection
3. Code instrumentation
   a) logging behaviors
   b) defending attacks

Machine Independent Optimization

1. Dead code elimination
2. Constant propagation
3. Live variable analysis
4. Vectorization
5. …

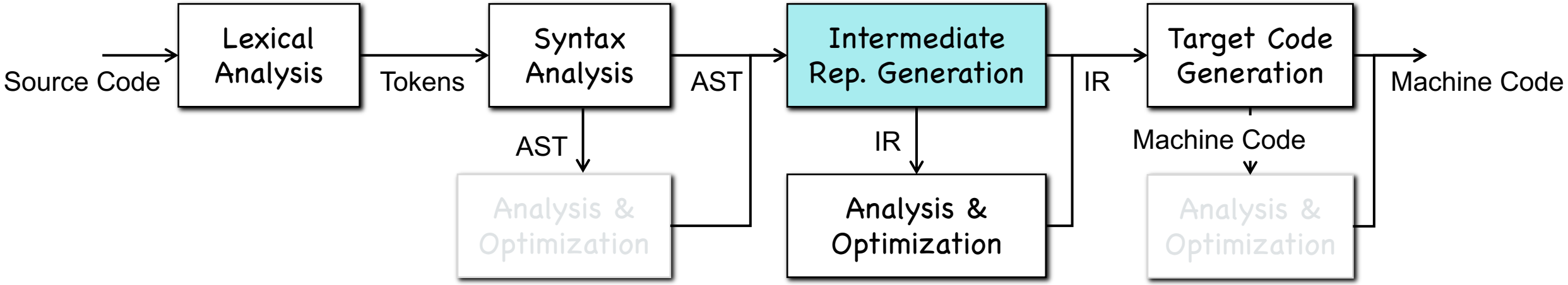
Intermediate Representation




```
class List { … }
class ArrayList: public List { … }
class LinkedList: public List { … }

void foo(List *list) { list->bar(); }
```

Intermediate Representation



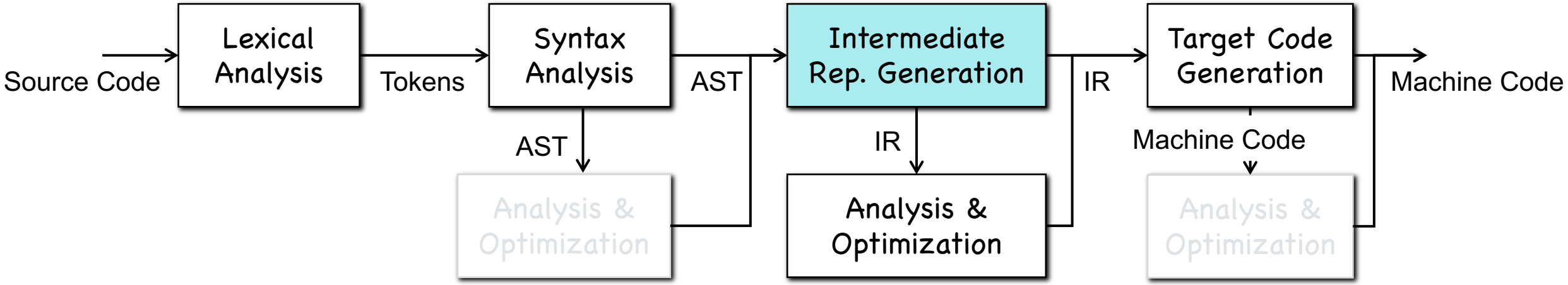

```
class List { … }
class ArrayList: public List { … }
class LinkedList: public List { … }

void foo(List *list) { list->bar(); }
```

```
class List { … }
class ArrayList: public List { … }
class LinkedList: public List { … }

void foo(ArrayList *list) { (*list).bar(); }
```

Intermediate Representation




```
class List { … }
class ArrayList: public List { … }
class LinkedList: public List { … }

void foo(List *list) { list->bar(); }
```

```
class List { … }
class ArrayList: public List { … }
class LinkedList: public List { … }

void foo(ArrayList *list) { (*list).bar(); }
```

Code Size Reduction!
Time Cost Reduction!

Intermediate Representation

- **Part I: Three-Address Code**
  - Definition & Implementations of Three-Address Code
- **Part II: Generation of Three-Address Code**
  - Syntax-Directed Translation
  - Generating IR for Variable Declaration/Expressions/Statements
  - Generating IR for Control Flows

PART I: Three-Address Code

Three-Address Code

- Provide a standard representation, not as flexible as the source

Three-Address Code

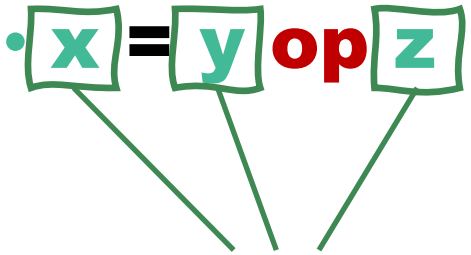
- Provide a standard representation, not as flexible as the source

- At most one operator on the right side of each instruction
- At most three **addresses/variables** in an instruction
  - **x = y op z**

Three-Address Code

- Provide a standard representation, not as flexible as the source

- At most one operator on the right side of each instruction
- At most three **addresses/variables** in an instruction



Variables or constant;

Variables are from the source code or created by the compiler

Three-Address Code

Instruction Type	Instruction Forms	Examples
Computation and Assignment		
Arithmetic Operation	x = y ⊕ z; x = ⊖ z;	x = y + z; x = -z;
Logic Operation	x = y ⊕ z; x = ⊖ z;	x = y && z; x = !z;
Relational Operation	x = y ⊕ z;	x = y > z;
Copy/Assignment	x = y;	-

Three-Address Code

Instruction Type	Instruction Forms	Examples
Computation and Assignment		
Arithmetic Operation	x = y ⊕ z; x = ⊖ z;	x = y + z; x = -z;
Logic Operation	x = y ⊕ z; x = ⊖ z;	x = y && z; x = !z;
Relational Operation	x = y ⊕ z;	x = y > z;
Copy/Assignment	x = y;	-
Control Flow		
Unconditional Jump	goto L;	-
Conditional Jump	if x goto L else goto L’;	-
	if x⊕y goto L else goto L’;	if x > y goto L else goto L’;

Three-Address Code

Instruction Type	Instruction Forms	Examples
Computation and Assignment		
Arithmetic Operation	x = y ⊕ z; x = ⊖ z;	x = y + z; x = -z;
Logic Operation	x = y ⊕ z; x = ⊖ z;	x = y && z; x = !z;
Relational Operation	x = y ⊕ z;	x = y > z;
Copy/Assignment	x = y;	-
Control Flow		
Unconditional Jump	goto L;	-
Conditional Jump	if x goto L else goto L';	=> if x goto L;
	if x⊕y goto L else goto L';	if x > y goto L else goto L';

Three-Address Code

Instruction Type	Instruction Forms	Examples
Computation and Assignment		
Arithmetic Operation	x = y ⊕ z; x = ⊖ z;	x = y + z; x = -z;
Logic Operation	x = y ⊕ z; x = ⊖ z;	x = y && z; x = !z;
Relational Operation	x = y ⊕ z;	x = y > z;
Copy/Assignment	x = y;	-
Control Flow		
Unconditional Jump	goto L;	-
Conditional Jump	if x goto L else goto L';	=> if x goto L;
	if x⊕y goto L else goto L';	if x > y goto L else goto L';
		=> if x > y goto L;

Three-Address Code

Instruction Type	Instruction Forms	Examples
Computation and Assignment		
Arithmetic Operation	x = y ⊕ z; x = ⊖ z;	x = y + z; x = -z;
Logic Operation	x = y ⊕ z; x = ⊖ z;	x = y && z; x = !z;
Relational Operation	x = y ⊕ z;	x = y > z;
Copy/Assignment	x = y;	-
Control Flow		
Unconditional Jump	goto L;	-
Conditional Jump	if x goto L else goto L';	=> if x goto L;
	if x⊕y goto L else goto L';	if x > y goto L else goto L';
		=> if x > y goto L; ≡ z = x > y; if z goto L;

Three-Address Code

Instruction Type	Instruction Forms	Examples
Memory Operations		
Indexed Copy	x = y[z]; y[z] = x;	
Address-Taken	x = &y;	-
Load Operation	x = *y;	-
Store Operation	*x = y;	-

Three-Address Code

Instruction Type	Instruction Forms	Examples
	Memory Operations	
Indexed Copy	x = y[z]; y[z] = x;	x = y[z]; ≣ t = y + z; x = *t;
Address-Taken	x = &y;	-
Load Operation	x = *y;	-
Store Operation	*x = y;	-

Three-Address Code

Instruction Type	Instruction Forms	Examples
Memory Operations		
Indexed Copy	x = y[z]; y[z] = x;	x = y[z]; ≡ t = y + z; x = *t; / y[z] = x; ≡ t = y + z; *t = x;
Address-Taken	x = &y;	-
Load Operation	x = *y;	-
Store Operation	*x = y;	-

Three-Address Code

Instruction Type	Instruction Forms	Examples
Memory Operations		
Indexed Copy	x = y[z]; y[z] = x; x = y[z]; ≡ t = y + z; x = *t;	y[z] = x; ≡ t = y + z; *t = x;
Address-Taken	x = &y;	-
Load Operation	x = *y;	-
Store Operation	*x = y;	-
Function Call and Return		
Function Call	param p_1; param p_2; ... param p_n; x = call func n; call func n;	param 1; param 2; call foo 2;
Function Return	return; return x;	-

Three-Address Code

Instruction Type	Instruction Forms	Examples
Memory Operations		
Indexed Copy	x = y[z]; y[z] = x; x = y[z]; ≡ t = y + z; x = *t;	y[z] = x; ≡ t = y + z; *t = x;
Address-Taken	x = &y;	-
Load Operation	x = *y;	-
Store Operation	*x = y;	-
Function Call and Return		
Function Call	param p_1; param p_2; ... param p_n; x = call func n; call func n;	param 1; param 2; call foo 2; => foo(1, 2)
Function Return	return; return x;	-

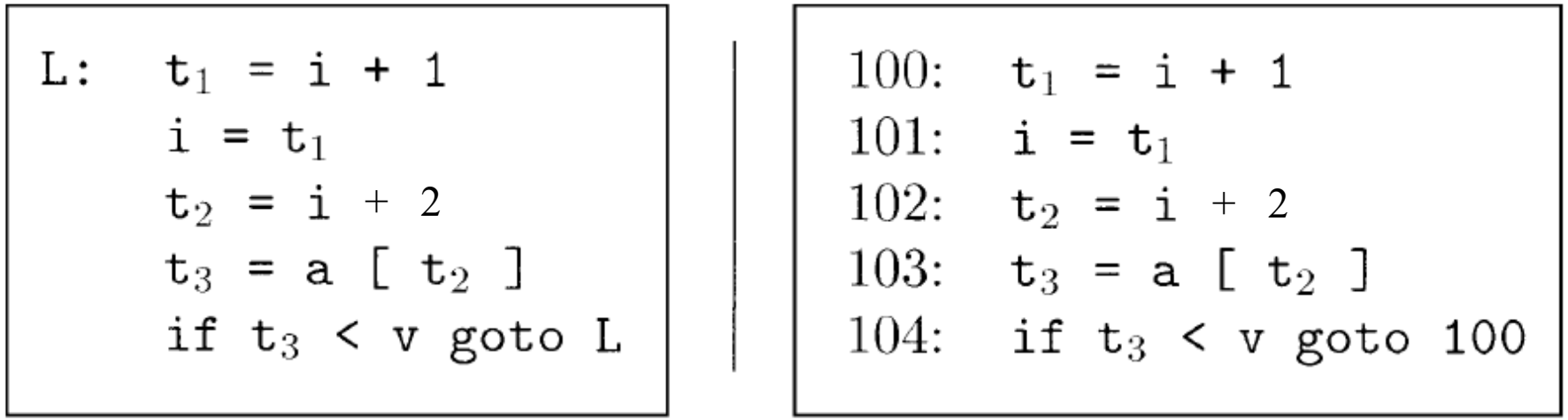
Three-Address Code

- do i = i + 1; while (a[i + 2] < v);

Try!

Three-Address Code

- do i = i + 1; while (a[i + 2] < v);



Symbolic Labels

Numeric Labels

- Implementation methods: *quadruples*, *triples*, etc.

Quadruples

- a = b * (-c) + b * (-c)

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
 a = t5
```

Three-Address Code

	op	arg_1	arg_2	*result*
0	minus	c		t_1
1	*	b	t_1	t_2
2	minus	c		t_3
3	*	b	t_3	t_4
4	+	t_2	t_4	t_5
5	=	t_5		a
	...			

Quadruples

Quadruples

- a = b * (-c) + b * (-c)

		op	arg_1	arg_2	$result$
t_1 = minus c	0	minus	c		t_1
t_2 = b * t_1	1	*	b	t_1	t_2
t_3 = minus c	2	minus	c		t_3
t_4 = b * t_3	3	*	b	t_3	t_4
t_5 = t_2 + t_4	4	+	t_2	t_4	t_5
a = t_5	5	=	t_5		a
		...			

Three-Address Code

Quadruples

Quadruples

- a = b * (-c) + b * (-c)

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
 a = t5
```

Three-Address Code

	op	arg_1	arg_2	*result*
0	minus	c		t_1
1	*	b	t_1	t_2
2	minus	c		t_3
3	*	b	t_3	t_4
4	+	t_2	t_4	t_5
5	=	t_5		a
	...			

Quadruples

Quadruples

- a = b * (-c) + b * (-c)

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5
```

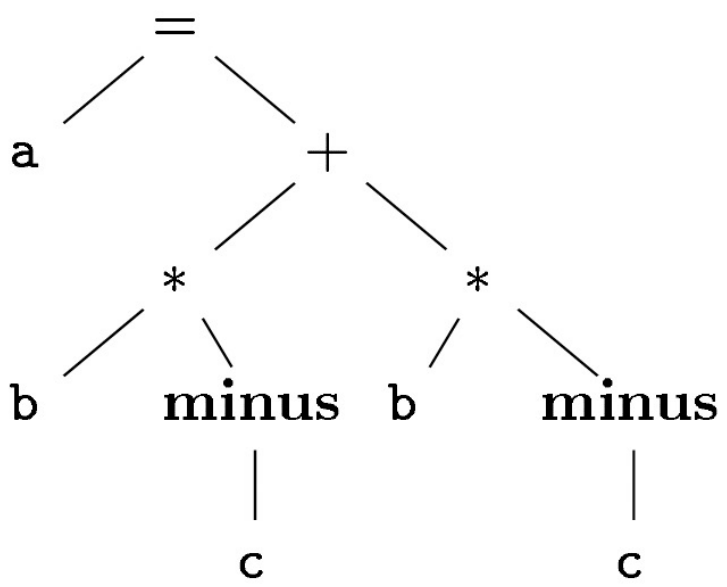
Three-Address Code

	op	arg_1	arg_2	*result*
0	minus	c		t_1
1	*	b	t_1	t_2
2	minus	c		t_3
3	*	b	t_3	t_4
4	+	t_2	t_4	t_5
5	=	t_5		a
	...			

Quadruples

Triples

- a = b * (-c) + b * (-c)



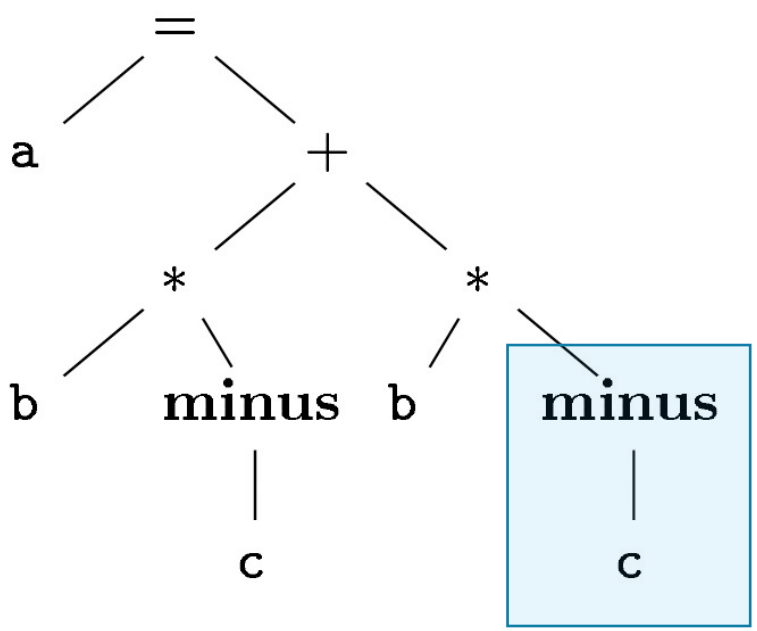
Syntax Tree

	op	arg_1	arg_2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

Triples

Triples

- a = b * (-c) + b * (-c)



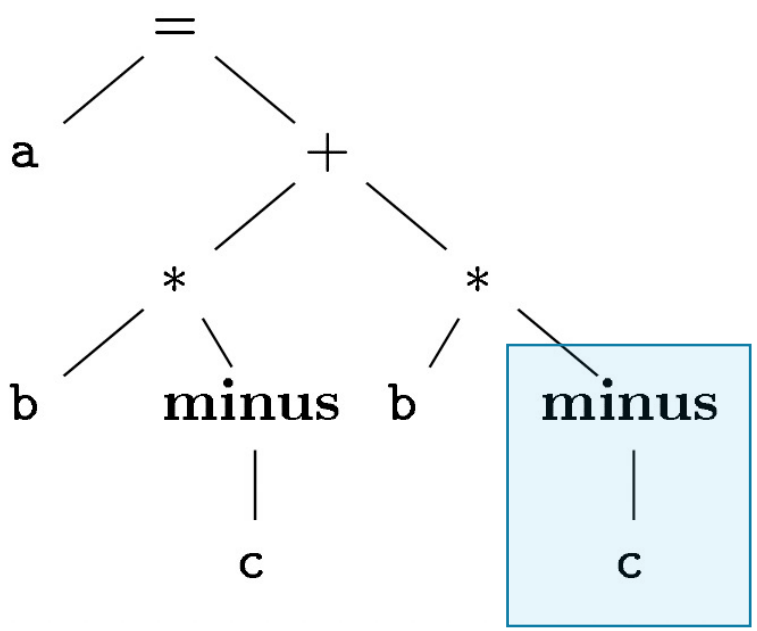
Syntax Tree

	op	arg_1	arg_2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

Triples

Triples

- a = b * (-c) + b * (-c)



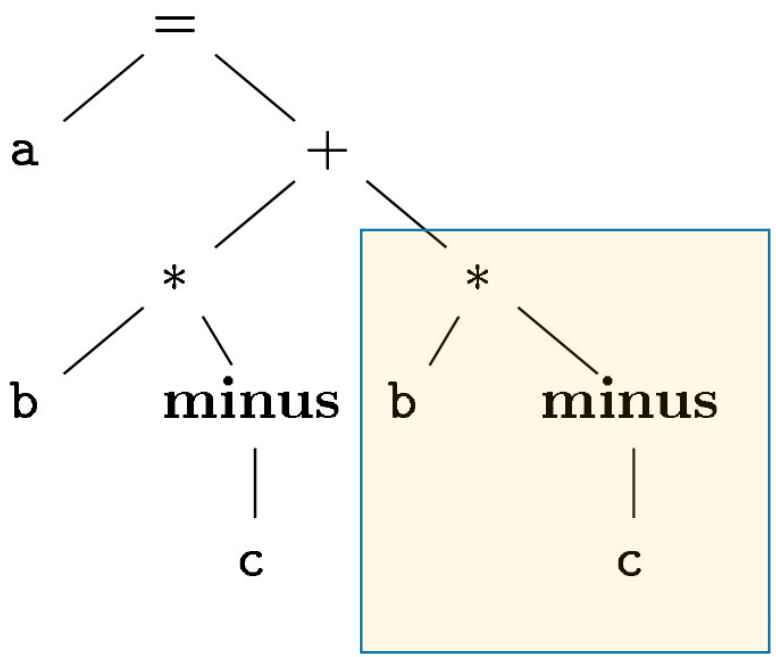
Syntax Tree

	op	arg_1	arg_2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

Triples

Triples

- a = b * (-c) + b * (-c)



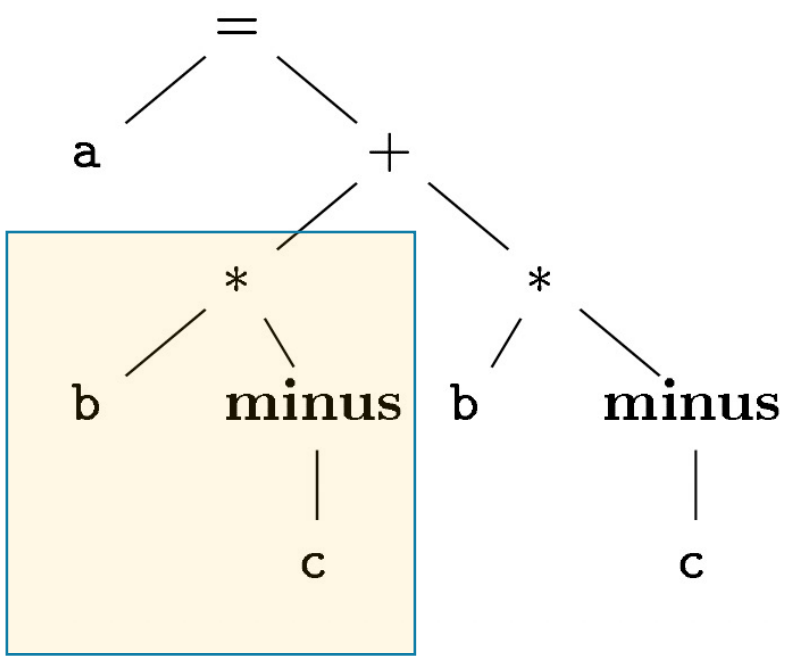
Syntax Tree

	op	arg_1	arg_2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

Triples

Triples

- a = b * (-c) + b * (-c)



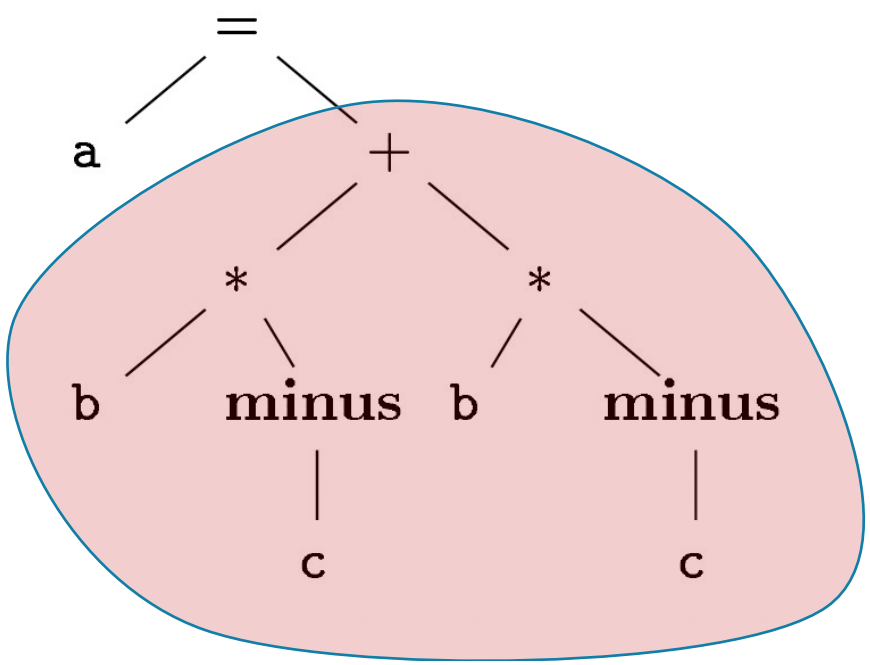
Syntax Tree

	op	arg_1	arg_2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

Triples

Triples

- a = b * (-c) + b * (-c)



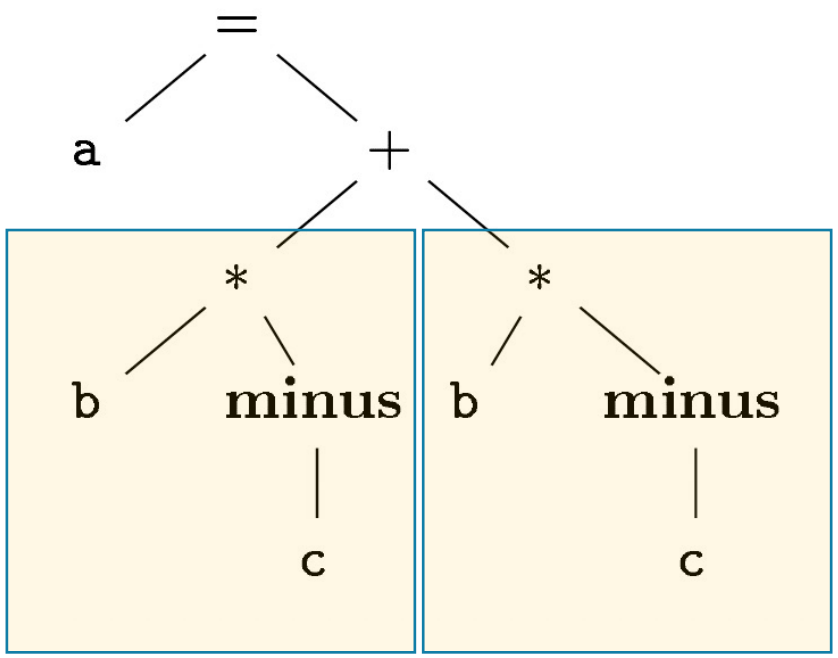
Syntax Tree

	op	arg_1	arg_2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

Triples

Triples

- a = b * (-c) + b * (-c)



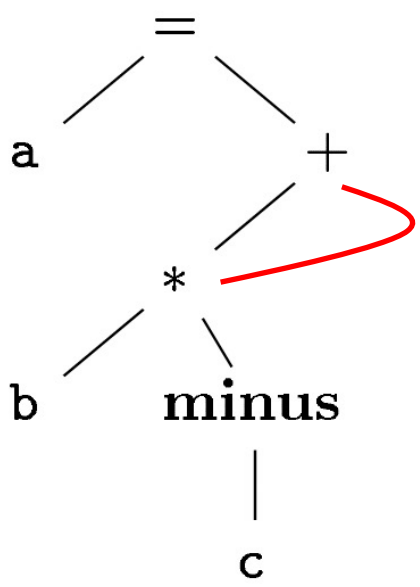
Syntax Tree

	op	arg_1	arg_2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

Triples

Triples with Syntax DAG

- a = b * (-c) + b * (-c)



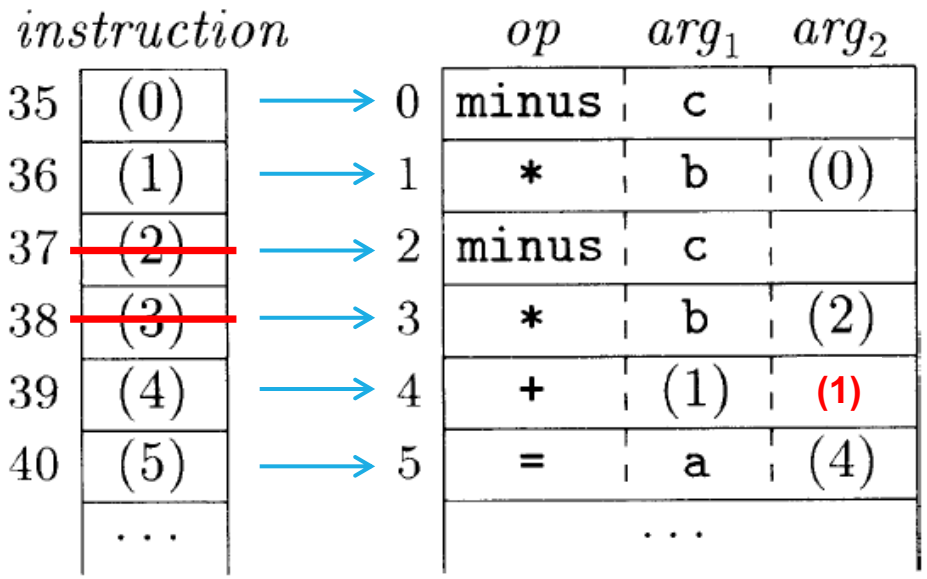
Syntax DAG

	op	arg_1	arg_2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(1)
5	=	a	(4)
	...		

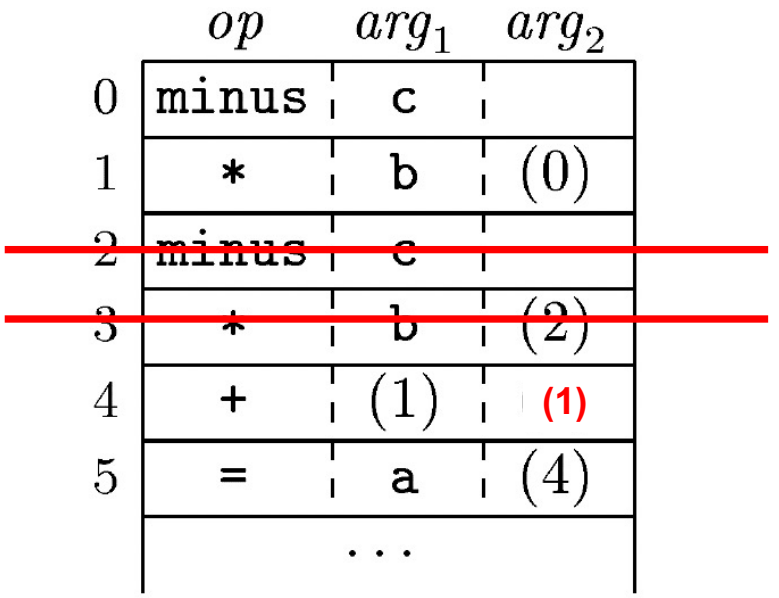
Triples

Indirect Triples with Syntax DAG

- a = b * (-c) + b * (-c)



Indirect Triples



Triples

Generation of Syntax DAG

PART II-1:
Syntax-Directed Translation

Syntax-Directed Definition (SDD)

- SDD = Context-Free Grammar + Attributes + Rules

Syntax-Directed Definition (SDD)

- SDD = Context-Free Grammar + Attributes + Rules
- **Attributes** are associated with **symbols** in the grammar
- **Rules** are associated with **productions** in the grammar

Syntax-Directed Definition (SDD)

- **Example**: Grammar for arithmetic expressions

Syntax-Directed Definition (SDD)

	PRODUCTION	SEMANTIC RULES
1)	$L \to E$ **n**	$L.val = E.val$
2)	$E \to E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \to T$	$E.val = T.val$
4)	$T \to T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \to F$	$T.val = F.val$
6)	$F \to (E)$	$F.val = E.val$
7)	$F \to$ **digit**	$F.val =$ **digit**.lexval

*** Grammar for arithmetic expressions**

Syntax-Directed Definition (SDD)

	PRODUCTION	SEMANTIC RULES
1)	$L \to E$ **n**	$L.val = E.val$
2)	$E \to E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \to T$	$E.val = T.val$
4)	$T \to T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \to F$	$T.val = F.val$
6)	$F \to (E)$	$F.val = E.val$
7)	$F \to$ **digit**	$F.val =$ **digit**.lexval

Rules of computing the result

*** Grammar for arithmetic expressions**

Syntax-Directed Definition (SDD)

	PRODUCTION	SEMANTIC RULES
1)	$L \to E$ **n**	$L.val = E.val$
2)	$E \to E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \to T$	$E.val = T.val$
4)	$T \to T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \to F$	$T.val = F.val$
6)	$F \to (E)$	$F.val = E.val$
7)	$F \to$ **digit**	$F.val =$ **digit**.lexval

Rules of computing the result

Attributes

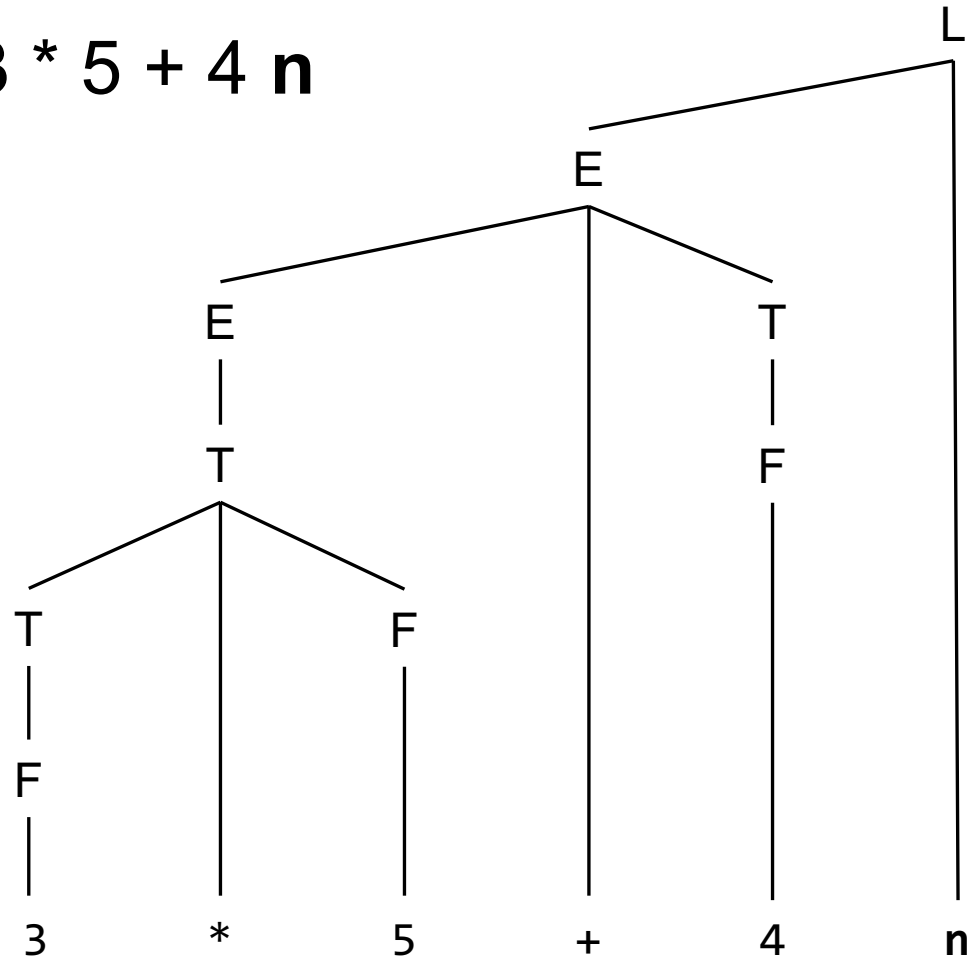
Attributes

*** Grammar for arithmetic expressions**

Understanding SDD via AST

- **Example**: (Annotated) AST for 3 * 5 + 4 **n**

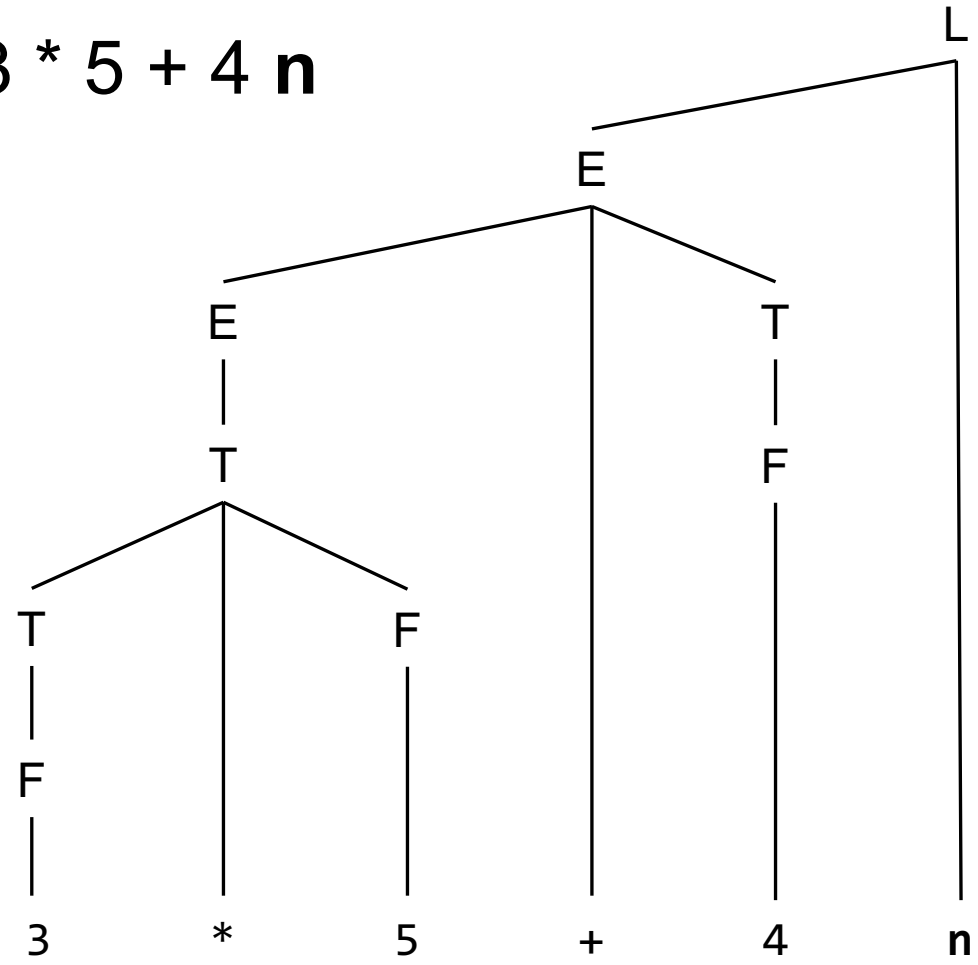
	PRODUCTION	SEMANTIC RULES
1)	$L \to E$ **n**	$L.val = E.val$
2)	$E \to E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \to T$	$E.val = T.val$
4)	$T \to T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \to F$	$T.val = F.val$
6)	$F \to (E)$	$F.val = E.val$
7)	$F \to$ **digit**	$F.val =$ **digit**.lexval



Understanding SDD via AST

- **Example**: (Annotated) AST for 3 * 5 + 4 **n**

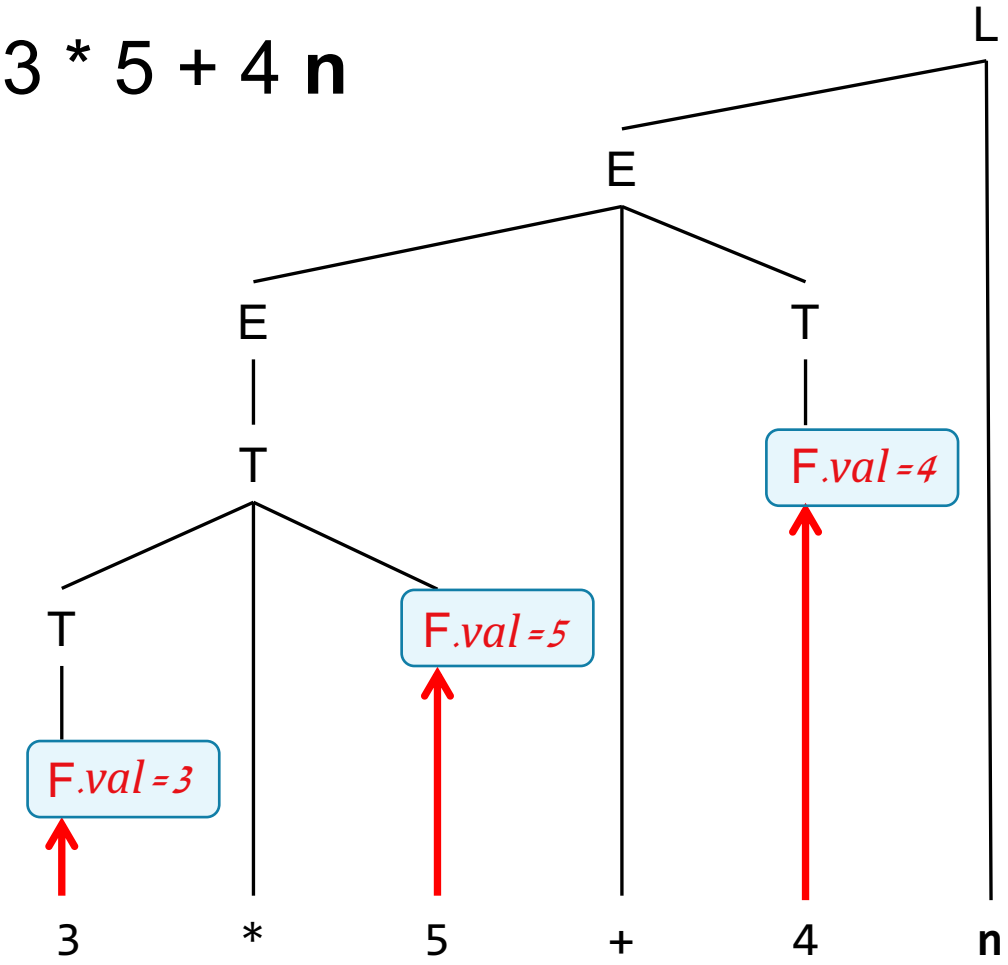
	PRODUCTION	SEMANTIC RULES
1)	$L \to E$ **n**	$L.val = E.val$
2)	$E \to E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \to T$	$E.val = T.val$
4)	$T \to T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \to F$	$T.val = F.val$
6)	$F \to (E)$	$F.val = E.val$
7)	$F \to$ **digit**	$F.val =$ **digit**.lexval



Understanding SDD via AST

- **Example**: (Annotated) AST for 3 * 5 + 4 **n**

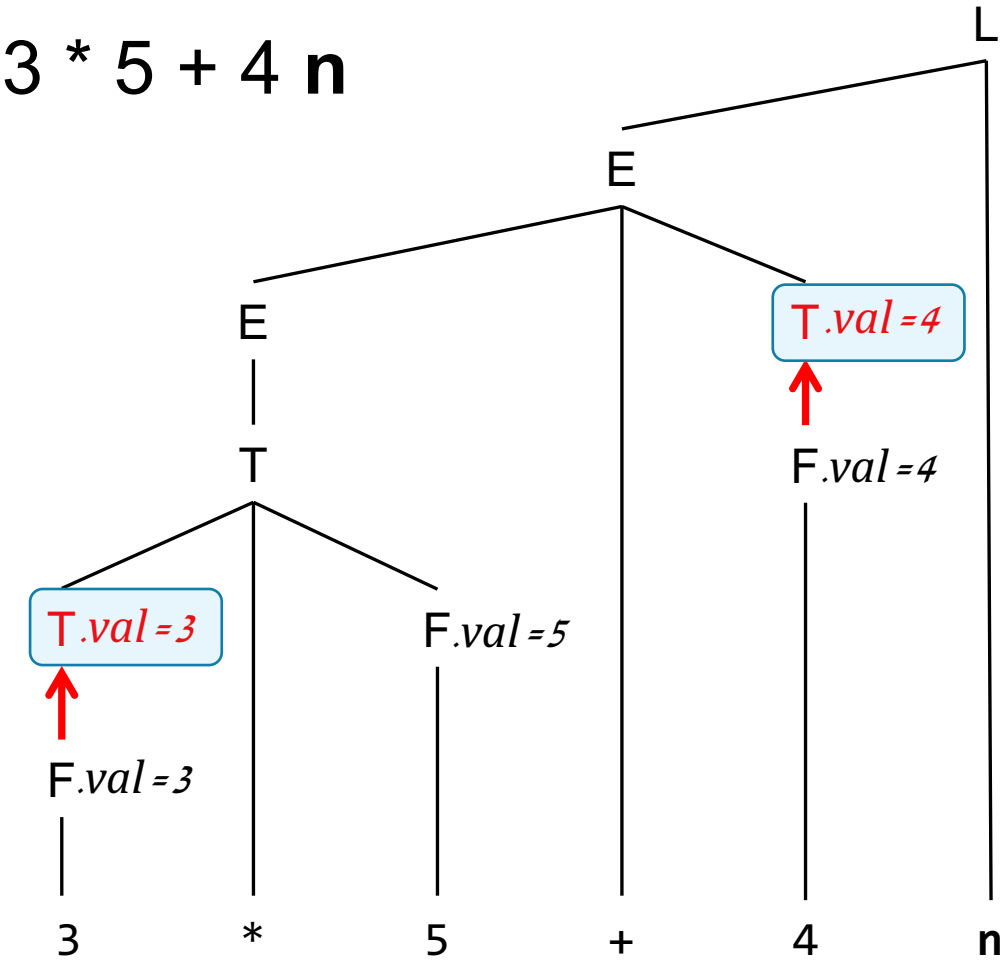
	PRODUCTION	SEMANTIC RULES
1)	$L \to E$ **n**	$L.val = E.val$
2)	$E \to E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \to T$	$E.val = T.val$
4)	$T \to T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \to F$	$T.val = F.val$
6)	$F \to (E)$	$F.val = E.val$
7)	$F \to$ **digit**	$F.val =$ **digit**.lexval



Understanding SDD via AST

- **Example**: (Annotated) AST for 3 * 5 + 4 **n**

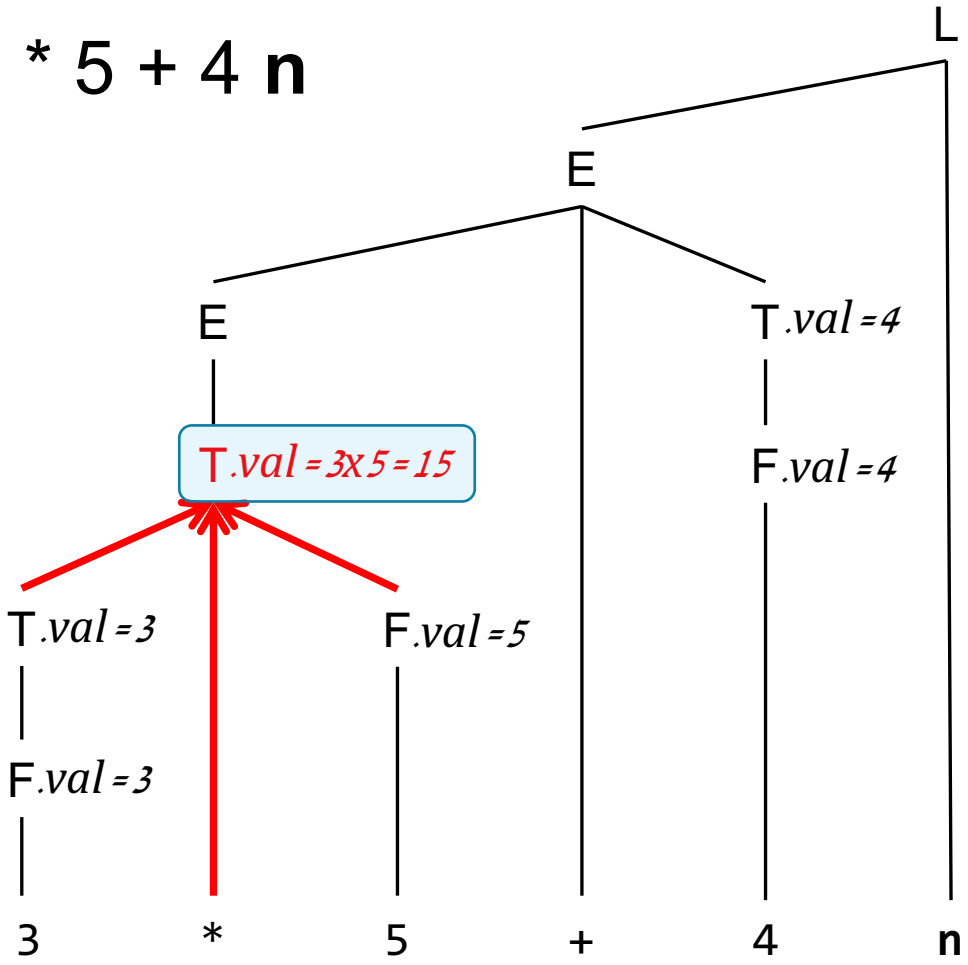
	Production	Semantic Rules
1)	$L \to E$ **n**	$L.val = E.val$
2)	$E \to E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \to T$	$E.val = T.val$
4)	$T \to T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \to F$	$T.val = F.val$
6)	$F \to (E)$	$F.val = E.val$
7)	$F \to$ **digit**	$F.val =$ **digit**.lexval



Understanding SDD via AST

- **Example**: (Annotated) AST for 3 * 5 + 4 **n**

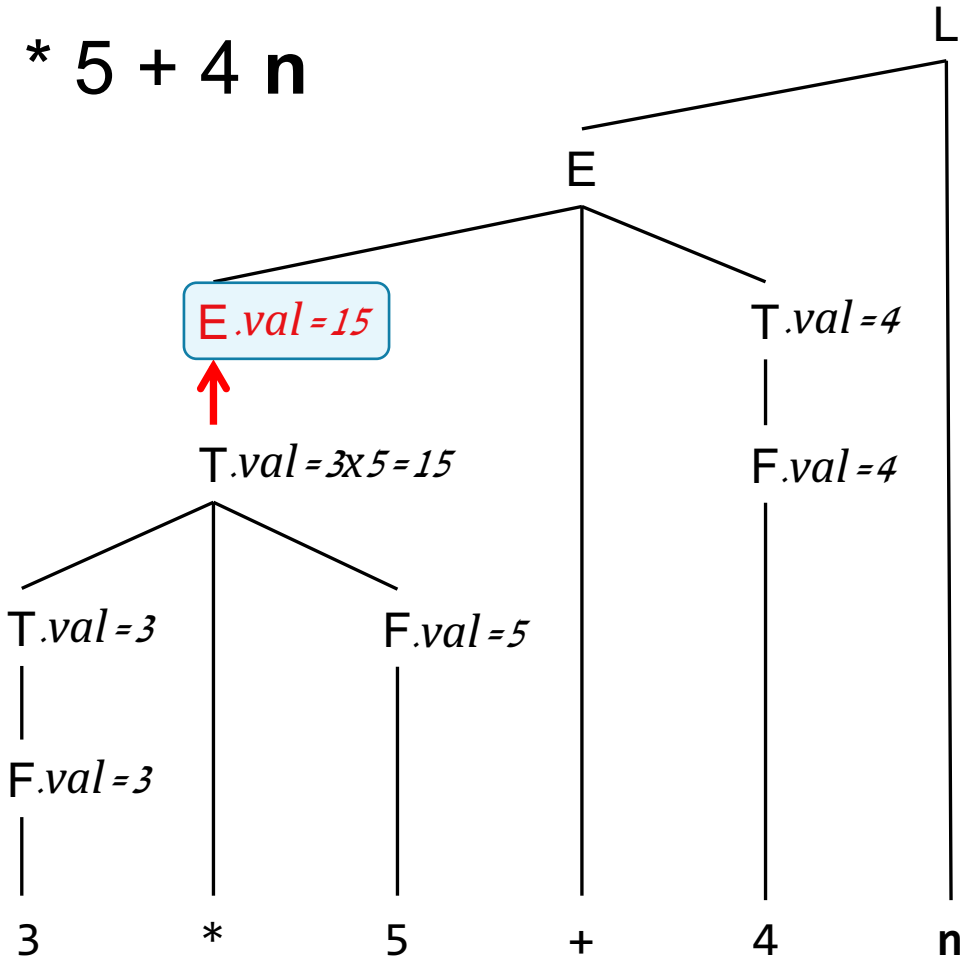
	PRODUCTION	SEMANTIC RULES
1)	$L \to E$ **n**	$L.val = E.val$
2)	$E \to E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \to T$	$E.val = T.val$
4)	$T \to T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \to F$	$T.val = F.val$
6)	$F \to (E)$	$F.val = E.val$
7)	$F \to$ **digit**	$F.val =$ **digit**.lexval



Understanding SDD via AST

- **Example**: (Annotated) AST for 3 * 5 + 4 **n**

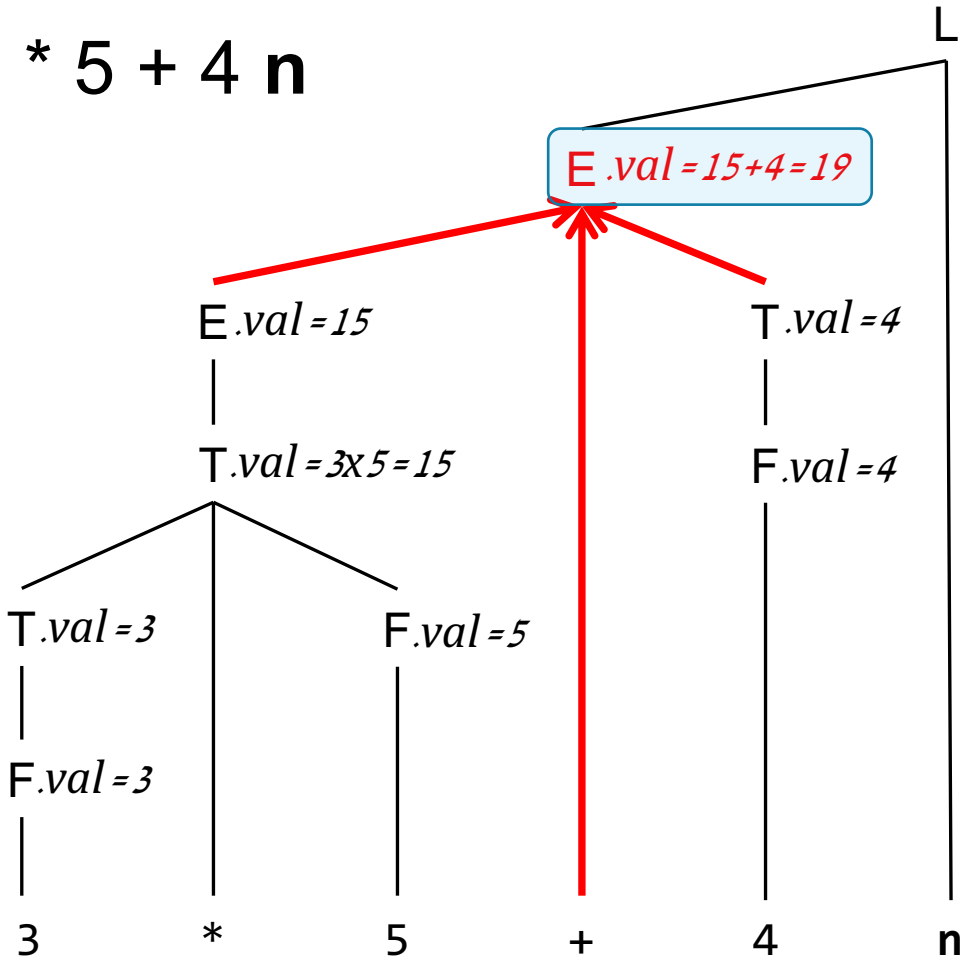
	PRODUCTION	SEMANTIC RULES
1)	$L \to E$ **n**	$L.val = E.val$
2)	$E \to E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \to T$	$E.val = T.val$
4)	$T \to T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \to F$	$T.val = F.val$
6)	$F \to (E)$	$F.val = E.val$
7)	$F \to$ **digit**	$F.val =$ **digit**.lexval



Understanding SDD via AST

- **Example**: (Annotated) AST for 3 * 5 + 4 **n**

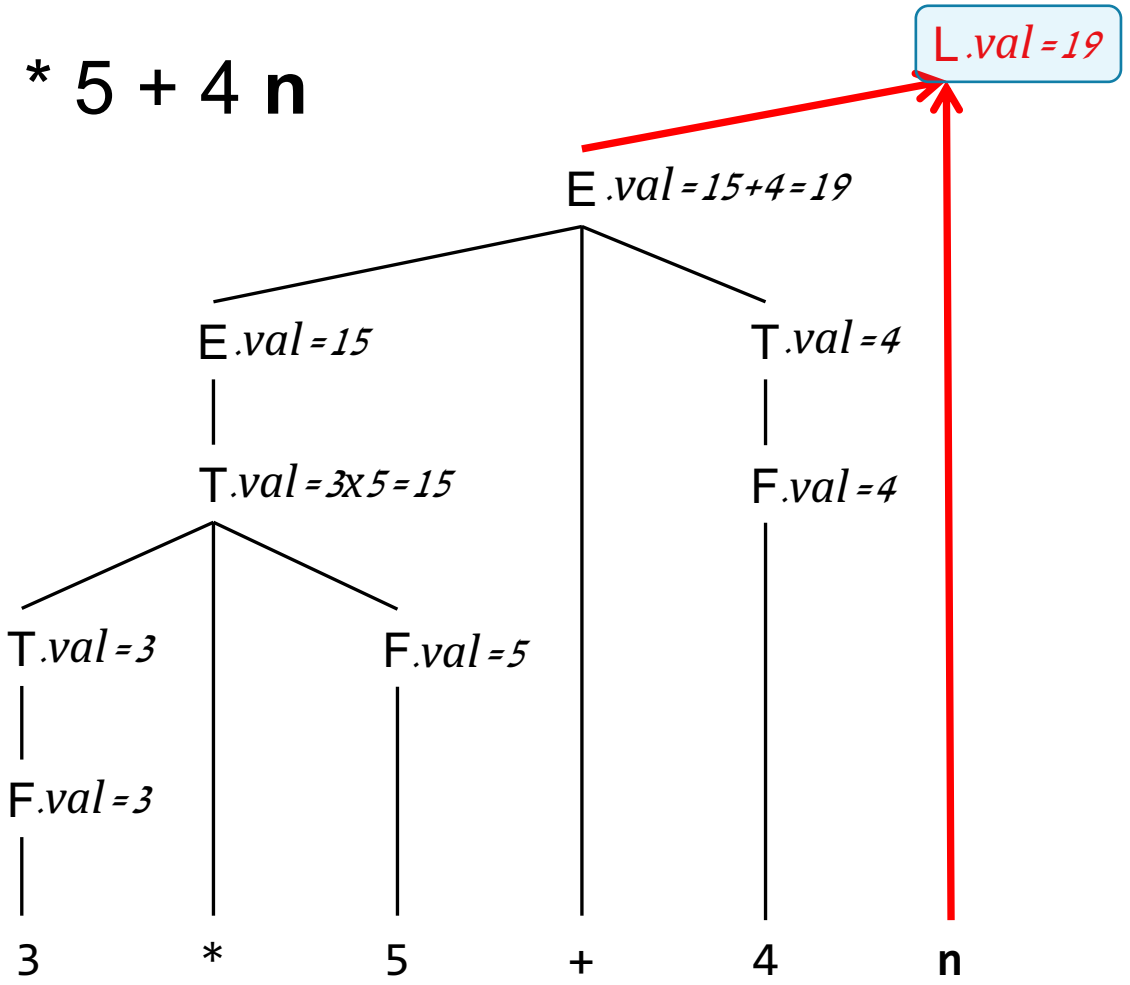
	PRODUCTION	SEMANTIC RULES
1)	$L \to E$ **n**	$L.val = E.val$
2)	$E \to E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \to T$	$E.val = T.val$
4)	$T \to T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \to F$	$T.val = F.val$
6)	$F \to (E)$	$F.val = E.val$
7)	$F \to$ **digit**	$F.val =$ **digit**.lexval



Understanding SDD via AST

- **Example**: (Annotated) AST for 3 * 5 + 4 **n**

	Production	Semantic Rules
1)	$L \to E$ **n**	$L.val = E.val$
2)	$E \to E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \to T$	$E.val = T.val$
4)	$T \to T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \to F$	$T.val = F.val$
6)	$F \to (E)$	$F.val = E.val$
7)	$F \to$ **digit**	$F.val =$ **digit**.lexval



Understanding SDD via AST

- SDD provides semantic rules that allow us to interpret a char string by traversing the syntax tree.

Implementing SDD via ANTLR4

```
parser grammar arith;

l   : e EOF;

e   : e ADD t | t;

t   : t MUL f | f;

f   : '(' e ')' | INT;

ADD : '+';
MUL : '*';
INT : [0-9]+;
```

arith.g4

Implementing SDD via ANTLR4

```
parser grammar arith;

l   : e EOF;

e   : e ADD t | t;

t   : t MUL f | f;

f   : '(' e ')' | INT;

ADD : '+';
MUL : '*';
INT : [0-9]+;
```

arith.g4

```
class ArithVisitor implements arithVisitor<Integer> {









}
```

ArithVisitor.java

Implementing SDD via ANTLR4

```
parser grammar arith;

l   : e EOF;

e   : e ADD t | t;

t   : t MUL f | f;

f   : '(' e ')' | INT;

ADD : '+';
MUL : '*';
INT : [0-9]+;
```

arith.g4

```
class ArithVisitor implements arithVisitor<Integer> {




}
```

3)	$E \to T$	$E.val = T.val$

ArithVisitor.java

Implementing SDD via ANTLR4

```
parser grammar arith;

l   : e EOF;

e   : e ADD t | t;

t   : t MUL f | f;

f   : '(' e ')' | INT;

ADD : '+';
MUL : '*';
INT : [0-9]+;
```

arith.g4

```
class ArithVisitor implements arithVisitor<Integer> {
    Map<Object, Integer> AttrMap = new Map<…>();

}
```

3) $E \rightarrow T$	$E.val = T.val$

ArithVisitor.java

Implementing SDD via ANTLR4

```
parser grammar arith;

l   : e EOF;

e   : e ADD t | t;

t   : t MUL f | f;

f   : '(' e ')' | INT;

ADD : '+';
MUL : '*';
INT : [0-9]+;
```

arith.g4

```
class ArithVisitor implements arithVisitor<Integer> {
    Map<Object, Integer> AttrMap = new Map<…>();

    @override
    public Integer visitE(arithParser.EContext ctx) {



    }
}
```

3) $E \rightarrow T$	$E.val = T.val$

ArithVisitor.java

Implementing SDD via ANTLR4

```
parser grammar arith;

l   : e EOF;

e   : e ADD t | t;

t   : t MUL f | f;

f   : '(' e ')' | INT;

ADD : '+';
MUL : '*';
INT : [0-9]+;
```

arith.g4

```
class ArithVisitor implements arithVisitor<Integer> {
    Map<Object, Integer> AttrMap = new Map<…>();

    @override
    public Integer visitE(arithParser.EContext ctx) {
        if (ctx.getChildCount() == 1) {
            Integer val = visitT(ctx.t());
            AttrMap.put(ctx, val);
            return val;
        } else ……
    }
}
```

3) $E \to T$	$E.val = T.val$

ArithVisitor.java

Syntax-Directed Translation (SDT)

- Using SDD to define a series of rules

Syntax-Directed Translation (SDT)

- Using SDD to define a series of rules, which translate a string in a high-level language into another language.

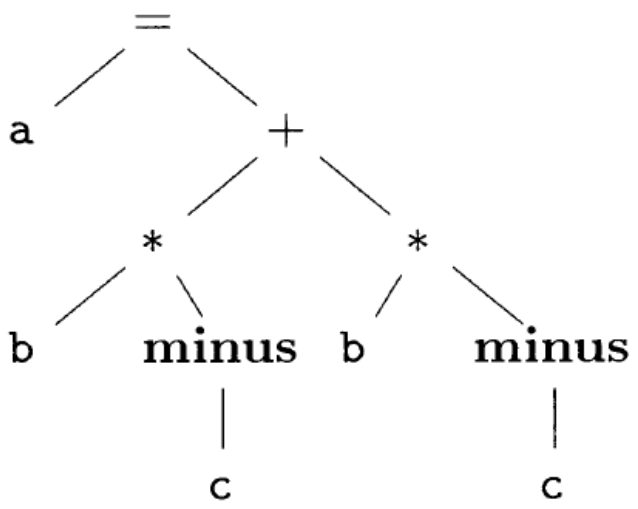
Syntax-Directed Translation (SDT)

- Using SDD to define a series of rules, which translate a string in a high-level language into another language.

- Particularly useful for IR generation.

Generation of Syntax Tree

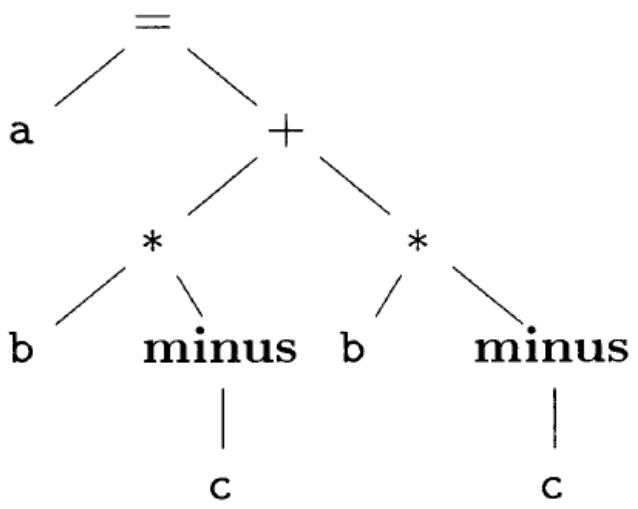
- a = b * (-c) + b * (-c)



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax Tree

- a = b * (-c) + b * (-c)



node(X): build a tree rooted at X, return the root

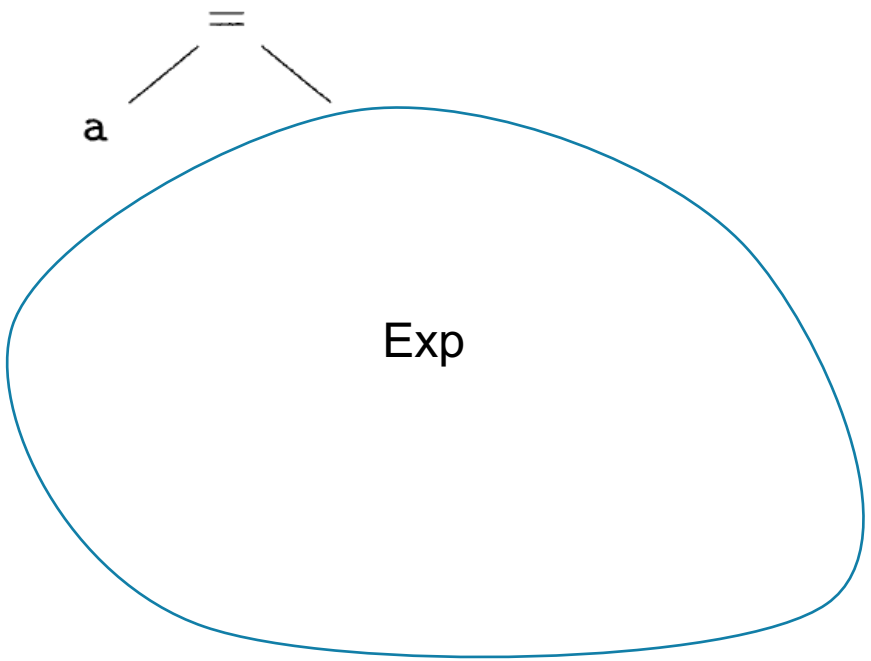
X.addChild(Y, Z, …): add child to a node

Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax Tree

- a = b * (-c) + b * (-c)

variable　　Exp

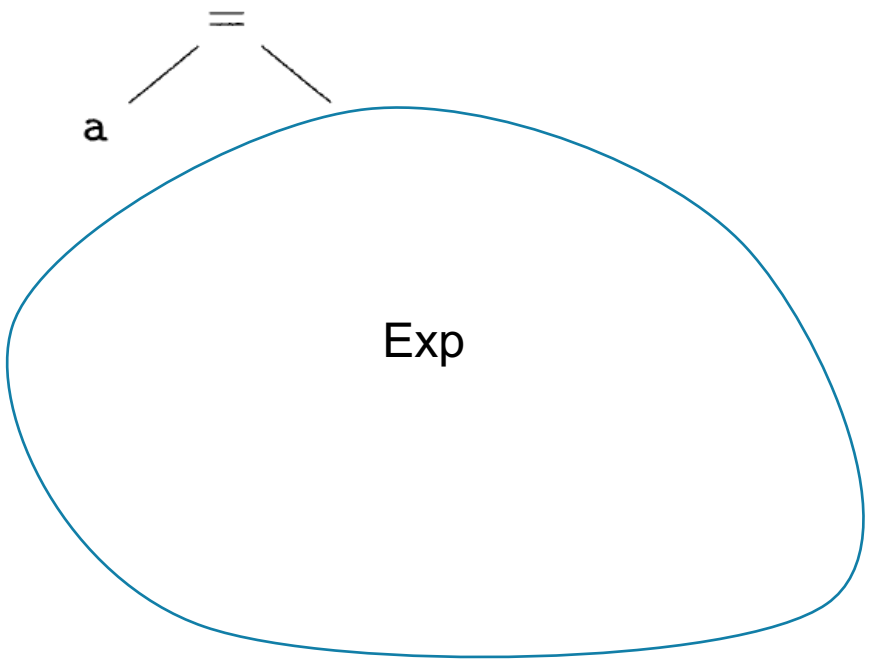


Production	Rules
Assignment → **variable** = Exp	node(=)　node(**variable**)　node(Exp)

Generation of Syntax Tree

- a = b * (-c) + b * (-c)

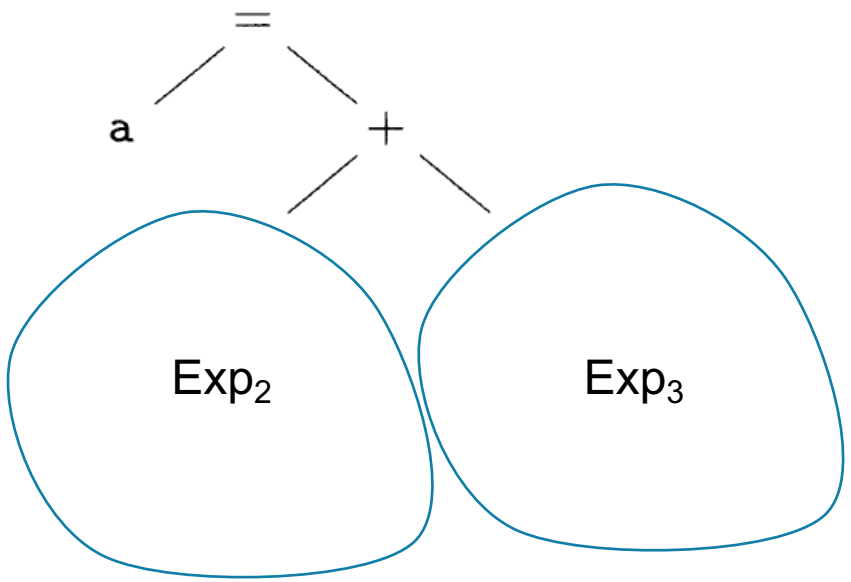
variable　　Exp



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))

Generation of Syntax Tree

- a = b * (-c) + b * (-c)
  - Exp_2: b * (-c)
  - Exp_3: b * (-c)

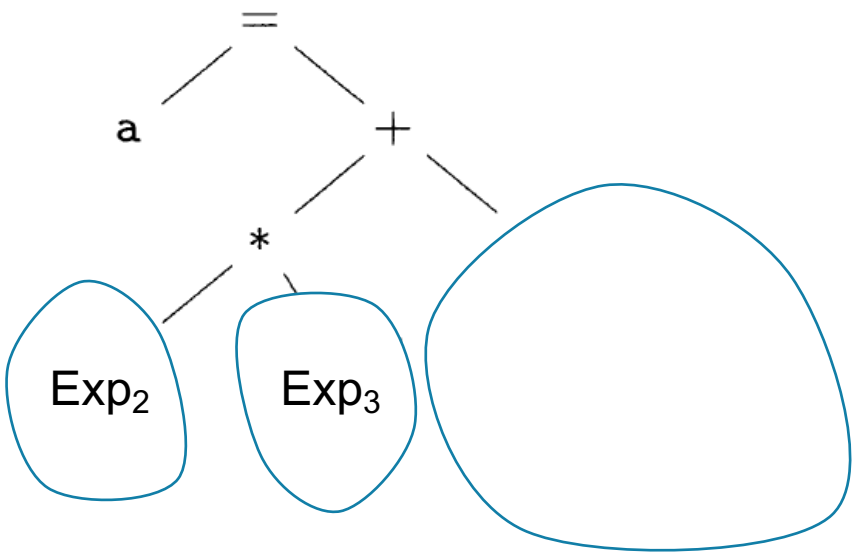


Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))

Generation of Syntax Tree

- a = b * (-c) + b * (-c)

Exp_2 Exp_3

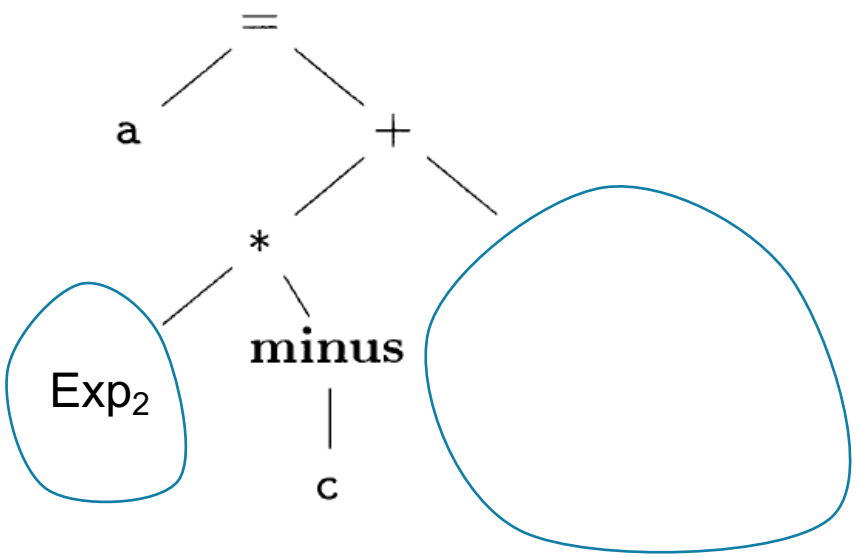


Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))

Generation of Syntax Tree

- a = b * (-c) + b * (-c)

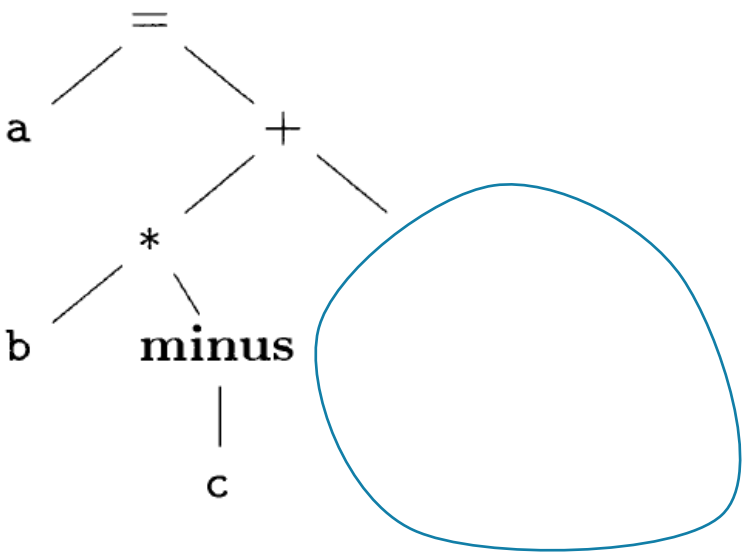
Exp_2 Exp_3



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))

Generation of Syntax Tree

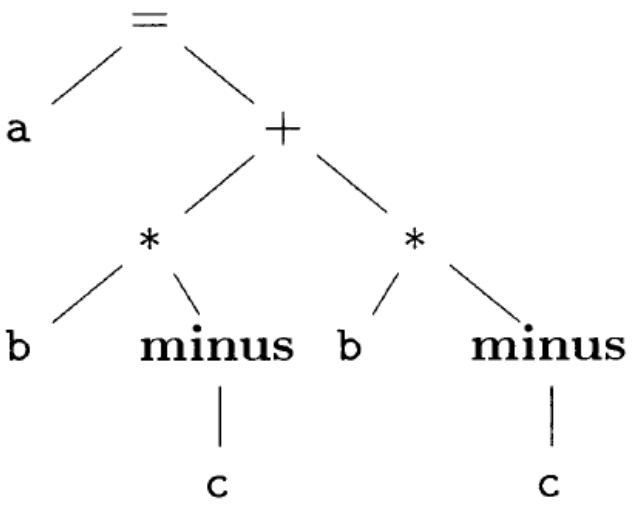
- a = b * (-c) + b * (-c)



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax Tree

- a = b * (-c) + b * (-c)



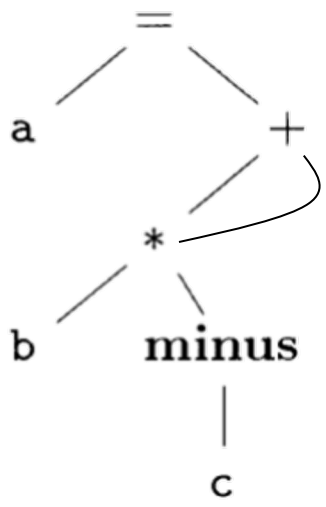
Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax DAG

If we already build a same tree, return it

node(X): build a tree rooted at X, return the root

X.addChild(Y, Z, …): add child to a node



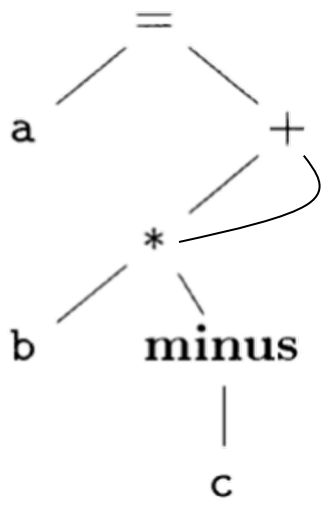
Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax DAG

Hash consing is the technique

node(X): build a tree rooted at X, return the root

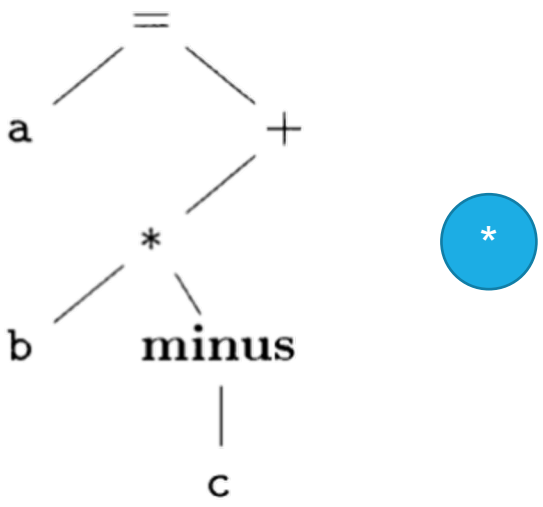
X.addChild(Y, Z, …): add child to a node



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax DAG

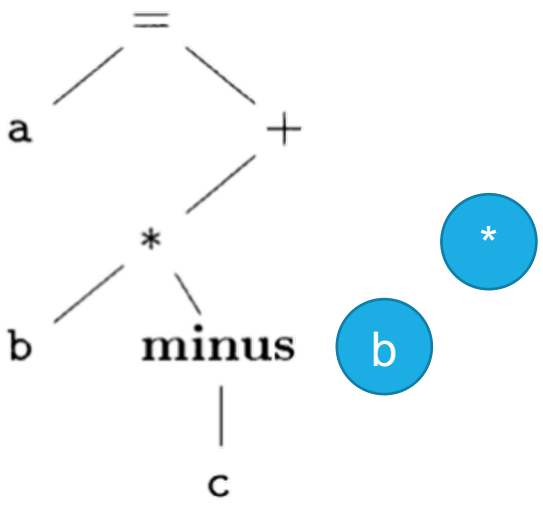
- a = b * (-c) + **b * (-c)**



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax DAG

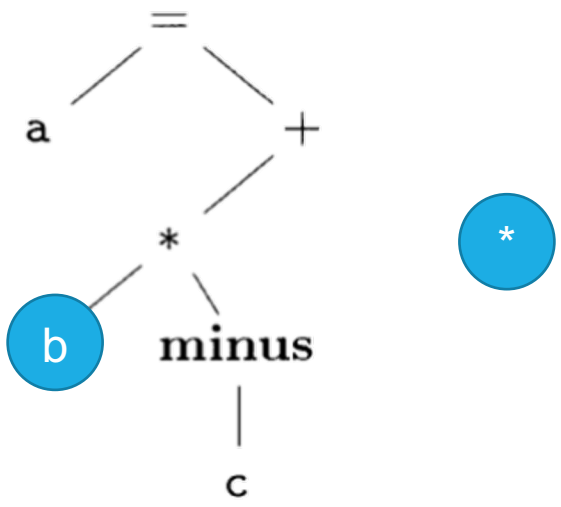
- a = b * (-c) + **b * (-c)**



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax DAG

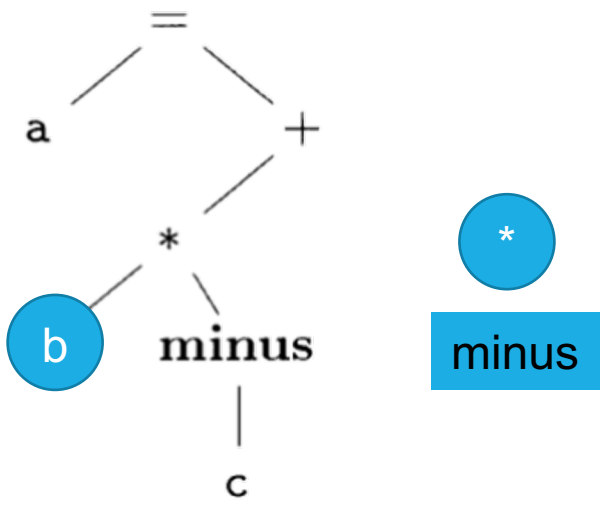
- a = b * (-c) + **b * (-c)**



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax DAG

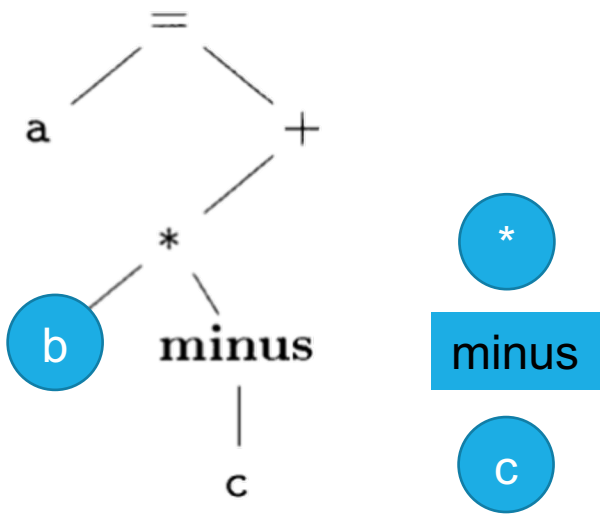
- a = b * (-c) + **b * (-c)**



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax DAG

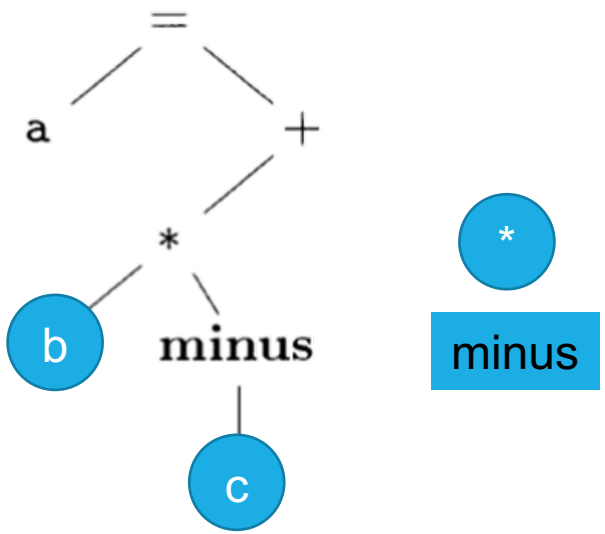
- a = b * (-c) + **b * (-c)**



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax DAG

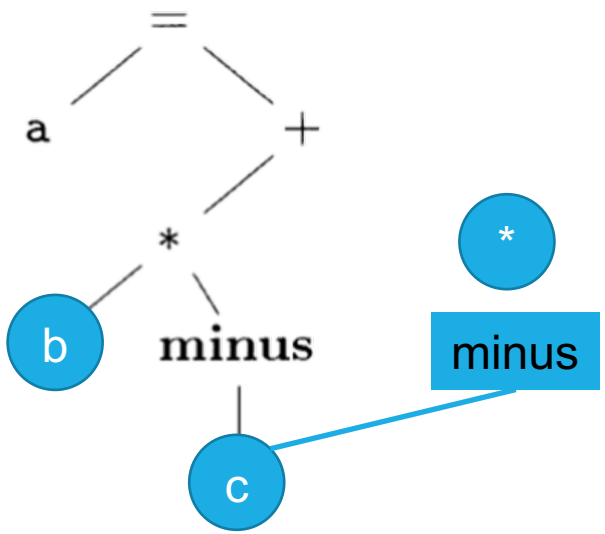
- a = b * (-c) + **b * (-c)**



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax DAG

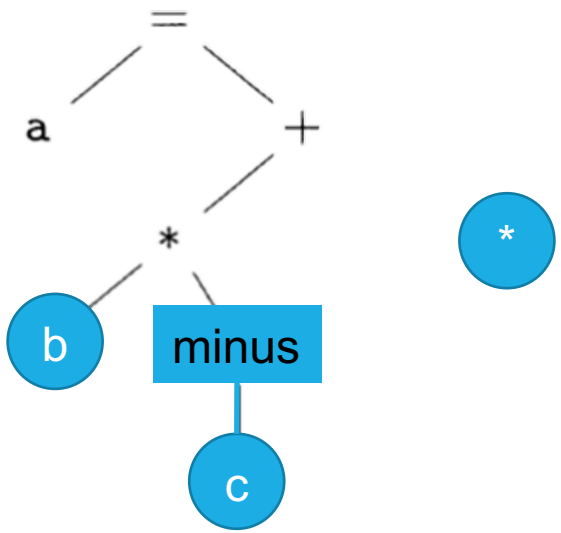
- a = b * (-c) + **b * (-c)**



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax DAG

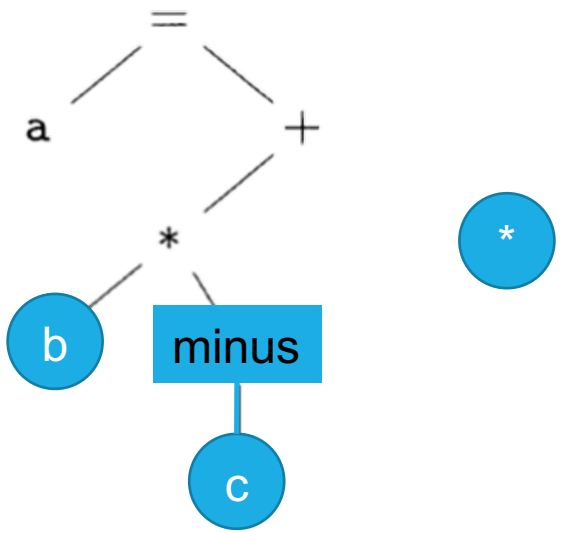
- a = b * (-c) + **b * (-c)**



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax DAG

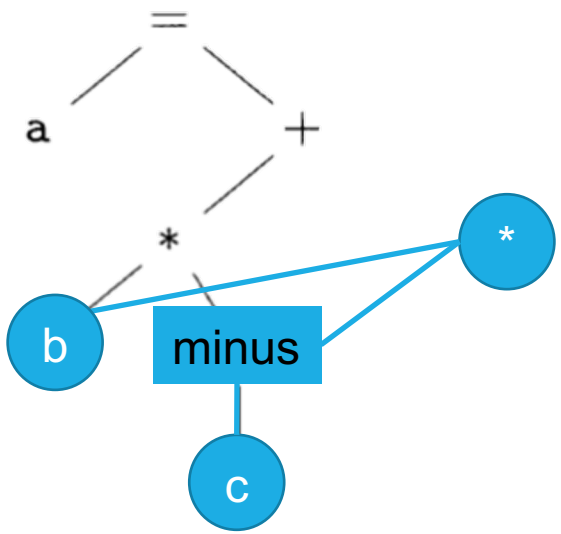
- a = b * (-c) + **b * (-c)**



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax DAG

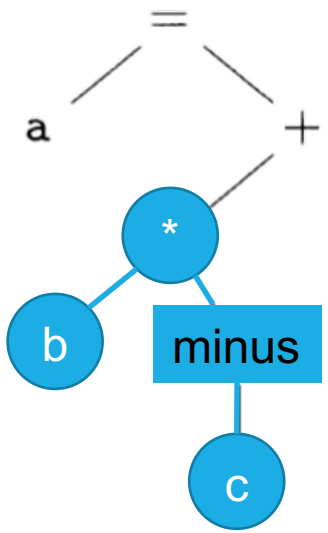
- a = b * (-c) + **b * (-c)**



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax DAG

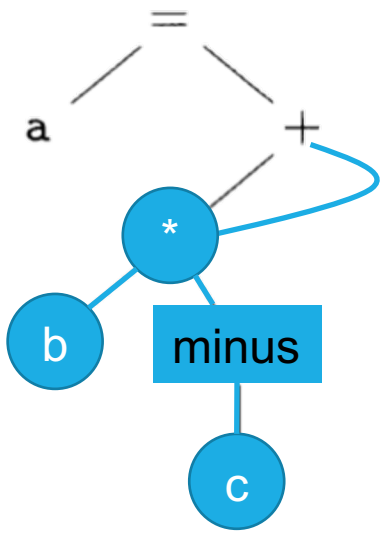
- a = b * (-c) + **b * (-c)**



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

Generation of Syntax DAG

- a = b * (-c) + **b * (-c)**



Production	Rules
Assignment → **variable** = Exp	node(=).addChild(node(**variable**), node(Exp))
Exp_1 → Exp_2 + Exp_3	node(+).addChild(node(Exp_2), node(Exp_3))
Exp_1 → Exp_2 * Exp_3	node(*).addChild(node(Exp_2), node(Exp_3))
Exp_1 → (-Exp_2)	node(minus).addChild(node(Exp_2))
Exp → **variable**	node(**variable**)

PART II-2:
Translating Variable Declarations

Variable Declaration

```
int main(){
    int a; int b; int c;
    scanf("%d%d", &a, &b);
    c = a + b;
    printf("%d", c);
    return 0;
}
```

Declaration → TypeExpression Variable;

Generating IR for variable declaration:

1. **Type of a variable!**
2. **Memory layout!**

Type Expression

- Primary Types
  - int, char, float, …

Type Expression

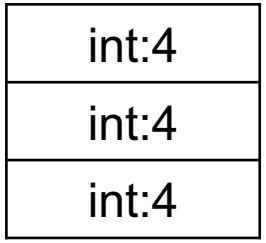
- Primary Types
  - int, char, float, …
- Composite Types
  - type expression [number]
    - int[3]
    - int[3][4]

Type Expression

- Primary Types
  - int, char, float, …
- Composite Types
  - type expression [number]
    - int[3]
    - int[3][4]
  - record {list of declarations}
    - record {float x; int[3] y;}
    - record {float x; record {int a; int b;} y}

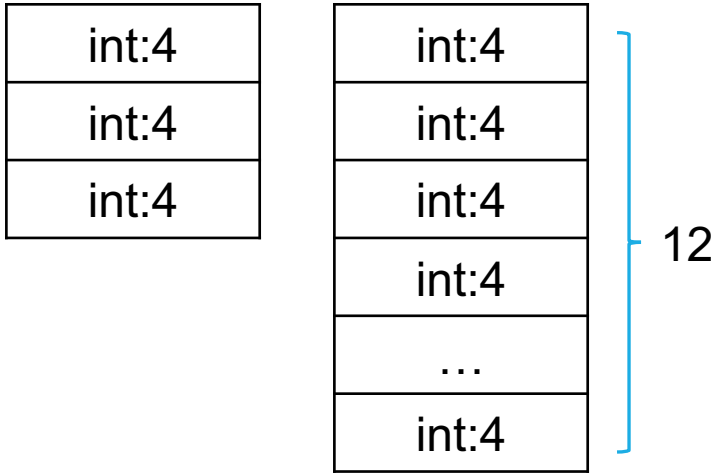
Type Expression

- Primary Types
  - int, char, float, …
- Composite Types
  - type expression [number]
    - int[3]
    - int[3][4]
  - record {list of declarations}
    - record {float x; int[3] y;}
    - record {float x; record {int a; int b;} y}



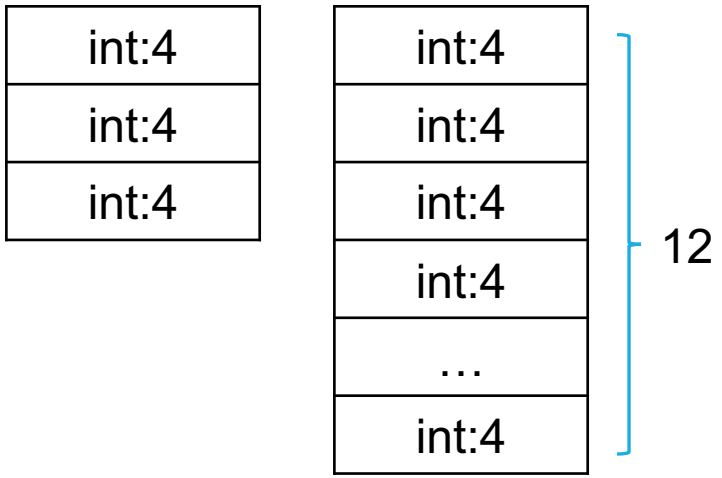
Type Expression

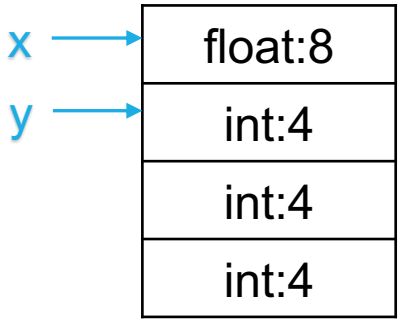
- Primary Types
  - int, char, float, …
- Composite Types
  - type expression [number]
    - int[3]
    - int[3][4]
  - record {list of declarations}
    - record {float x; int[3] y;}
    - record {float x; record {int a; int b;} y}



Type Expression

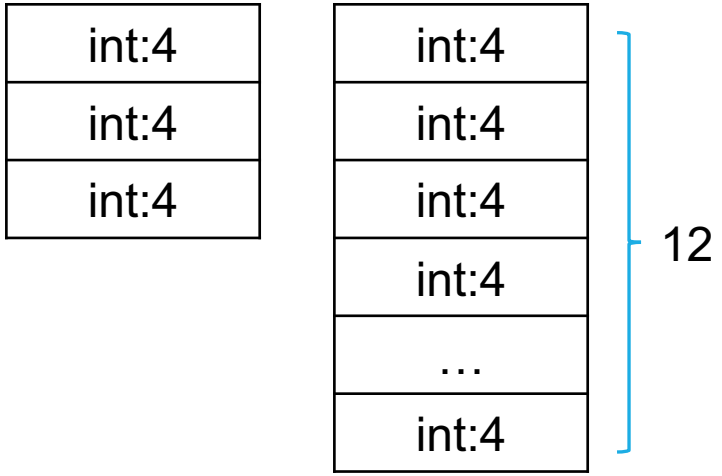
- Primary Types
  - int, char, float, …
- Composite Types
  - type expression [number]
    - int[3]
    - int[3][4]
  - record {list of declarations}
    - record {float x; int[3] y;}
    - record {float x; record {int a; int b;} y}

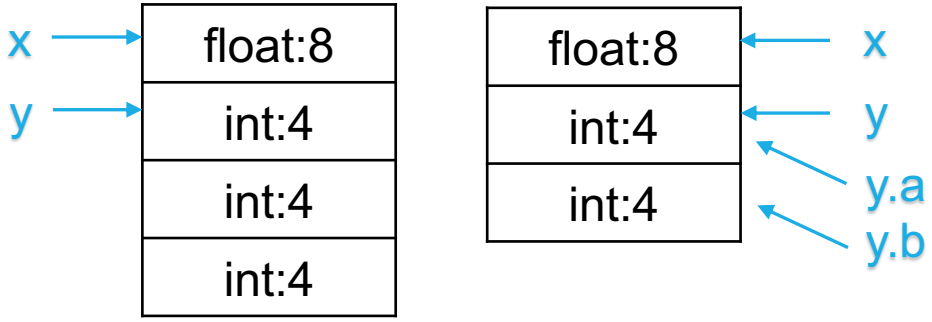




Type Expression

- Primary Types
  - int, char, float, …
- Composite Types
  - type expression [number]
    - int[3]
    - int[3][4]
  - record {list of declarations}
    - record {float x; int[3] y;}
    - record {float x; record {int a; int b;} y}





Variable Declaration

$$
\begin{array}{lcl}
D & \rightarrow & T\ \mathbf{id}\ ;\ D \mid \epsilon \\
T & \rightarrow & B\ C \mid \mathbf{record}\ '\{'\ D\ '\}' \\
B & \rightarrow & \mathbf{int} \mid \mathbf{float} \\
C & \rightarrow & \epsilon \mid [\ \mathbf{num}\ ]\ C
\end{array}
$$

Variable Declaration

Declaration List

$$
\begin{array}{lcl}
D & \rightarrow & T\ \mathbf{id}\ ;\ D \mid \epsilon \\
T & \rightarrow & B\ C \mid \mathbf{record}\ '\{'\ D\ '\}' \\
B & \rightarrow & \mathbf{int} \mid \mathbf{float} \\
C & \rightarrow & \epsilon \mid [\ \mathbf{num}\ ]\ C
\end{array}
$$

Variable Declaration

$$
\begin{aligned}
D &\rightarrow T\ \mathbf{id}\ ;\ D \mid \epsilon \\
T &\rightarrow B\ C \mid \mathbf{record}\ '\{'\ D\ '\}' \\
B &\rightarrow \mathbf{int} \mid \mathbf{float} \\
C &\rightarrow \epsilon \mid [\ \mathbf{num}\ ]\ C
\end{aligned}
$$

Type Expressions

Variable Declaration

$$
\begin{array}{lcl}
D & \rightarrow & T\ \mathbf{id}\ ;\ D \mid \epsilon \\
T & \rightarrow & B\ C \mid \mathbf{record}\ '\{'\ D\ '\}' \\
B & \rightarrow & \mathbf{int} \mid \mathbf{float} \\
C & \rightarrow & \epsilon \mid [\ \mathbf{num}\ ]\ C
\end{array}
$$

Type Expressions

SDT for Type Expressions

$$
\begin{array}{lcl}
D & \rightarrow & T\ \mathbf{id}\ ;\ D \mid \epsilon \\
T & \rightarrow & B\ C \mid \mathbf{record}\ '\{'\ D\ '\}' \\
B & \rightarrow & \mathbf{int} \mid \mathbf{float} \\
C & \rightarrow & \epsilon \mid [\ \mathbf{num}\ ]\ C
\end{array}
$$

SDT for Type Expressions

- Compute the **type** as well as the **type width**

$$
\begin{array}{lcll}
T & \rightarrow & B & \{\ t = B.type;\ w = B.width;\ \} \\
 & & C & \{\ T.type = C.type;\ T.width = C.width;\ \} \\
B & \rightarrow & \textbf{int} & \{\ B.type = integer;\ B.width = 4;\ \} \\
B & \rightarrow & \textbf{float} & \{\ B.type = float;\ B.width = 8;\ \} \\
C & \rightarrow & \epsilon & \{\ C.type = t;\ C.width = w;\ \} \\
C & \rightarrow & [\ \textbf{num}\ ]\ C_1 & \{\ C.type = array(\textbf{num}.value,\ C_1.type); \\
 & & & \quad C.width = \textbf{num}.value \times C_1.width;\ \}
\end{array}
$$

SDT for Type Expressions

- Compute the **type** as well as the **type width**

$$
\begin{array}{lcll}
T & \rightarrow & B & \{\ t = B.type;\ w = B.width;\ \} \\
 & & C & \{\ T.type = C.type;\ T.width = C.width;\ \} \\
B & \rightarrow & \textbf{int} & \{\ B.type = integer;\ B.width = 4;\ \} \\
B & \rightarrow & \textbf{float} & \{\ B.type = float;\ B.width = 8;\ \} \\
C & \rightarrow & \epsilon & \{\ C.type = t;\ C.width = w;\ \} \\
C & \rightarrow & \textbf{[num]}\ C_1 & \{\ C.type = array(\textbf{num}.value,\ C_1.type); \\
 & & & \quad C.width = \textbf{num}.value \times C_1.width;\ \}
\end{array}
$$

SDT for Type Expressions

- Compute the **type** as well as the **type width**

$$
\begin{array}{lcll}
T & \rightarrow & B & \{\ t = B.type;\ w = B.width;\ \} \\
 & & C & \{\ T.type = C.type;\ T.width = C.width;\ \} \\
B & \rightarrow & \textbf{int} & \{\ B.type = integer;\ B.width = 4;\ \} \\
B & \rightarrow & \textbf{float} & \{\ B.type = float;\ B.width = 8;\ \} \\
C & \rightarrow & \epsilon & \{\ C.type = t;\ C.width = w;\ \} \\
C & \rightarrow & [\ \textbf{num}\ ]\ C_1 & \{\ C.type = array(\textbf{num}.value,\ C_1.type); \\
 & & & \quad C.width = \textbf{num}.value \times C_1.width;\ \}
\end{array}
$$

SDT for Type Expressions

- Compute the **type** as well as the **type width**

$$
\begin{array}{lcll}
T & \rightarrow & B & \{\ t = B.type;\ w = B.width;\ \} \\
 & & C & \{\ T.type = C.type;\ T.width = C.width;\ \} \\
B & \rightarrow & \textbf{int} & \{\ B.type = integer;\ B.width = 4;\ \} \\
B & \rightarrow & \textbf{float} & \{\ B.type = float;\ B.width = 8;\ \} \\
C & \rightarrow & \epsilon & \{\ C.type = t;\ C.width = w;\ \} \\
C & \rightarrow & [\ \textbf{num}\ ]\ C_1 & \{\ C.type = array(\textbf{num}.value,\ C_1.type); \\
 & & & \quad C.width = \textbf{num}.value \times C_1.width;\ \}
\end{array}
$$

SDT for Type Expressions

- Compute the **type** as well as the **type width**

$T \rightarrow B$		$\{\ t = B.type;\ w = B.width;\ \}$
C		$\{\ T.type = C.type;\ T.width = C.width;\ \}$
$B \rightarrow$	**int**	$\{\ B.type = integer;\ B.width = 4;\ \}$
$B \rightarrow$	**float**	$\{\ B.type = float;\ B.width = 8;\ \}$
$C \rightarrow$	ϵ	$\{\ C.type = t;\ C.width = w;\ \}$
$C \rightarrow$	[**num**] C_1	$\{\ C.type = array(\mathbf{num}.value,\ C_1.type);$ $C.width = \mathbf{num}.value \times C_1.width;\ \}$

SDT for Type Expressions

- Compute the **type** as well as the **type width**

$T \rightarrow B$		$\{\ t = B.type;\ w = B.width;\ \}$
C		$\{\ T.type = C.type;\ T.width = C.width;\ \}$
$B \rightarrow$	**int**	$\{\ B.type = integer;\ B.width = 4;\ \}$
$B \rightarrow$	**float**	$\{\ B.type = float;\ B.width = 8;\ \}$
$C \rightarrow$	ϵ	$\{\ C.type = t;\ C.width = w;\ \}$
$C \rightarrow$	[**num**] C_1	$\{\ C.type = array(\mathbf{num}.value,\ C_1.type);$ $C.width = \mathbf{num}.value \times C_1.width;\ \}$

SDT for Type Expressions

- Compute the **type** as well as the **type width**

T	$\rightarrow$	B	$\{\ t = B.type;\ w = B.width;\ \}$
		C	$\{\ T.type = C.type;\ T.width = C.width;\ \}$
B	$\rightarrow$	**int**	$\{\ B.type = integer;\ B.width = 4;\ \}$
B	$\rightarrow$	**float**	$\{\ B.type = float;\ B.width = 8;\ \}$
C	$\rightarrow$	ϵ	$\{\ C.type = t;\ C.width = w;\ \}$
C	$\rightarrow$	[**num**] C_1	$\{\ C.type = array(\mathbf{num}.value,\ C_1.type);$ $C.width = \mathbf{num}.value \times C_1.width;\ \}$

SDT for Type Expressions

- Compute the **type** as well as the **type width**

T	$\rightarrow$ B	$\{\ t = B.type;\ w = B.width;\ \}$ ①
	C	$\{\ T.type = C.type;\ T.width = C.width;\ \}$ ③
B	$\rightarrow$ **int**	$\{\ B.type = integer;\ B.width = 4;\ \}$
B	$\rightarrow$ **float**	$\{\ B.type = float;\ B.width = 8;\ \}$
C	$\rightarrow$ ϵ	$\{\ C.type = t;\ C.width = w;\ \}$ ②
C	$\rightarrow$ [**num**] C_1	$\{\ C.type = array(\mathbf{num}.value,\ C_1.type);$ $C.width = \mathbf{num}.value \times C_1.width;\ \}$

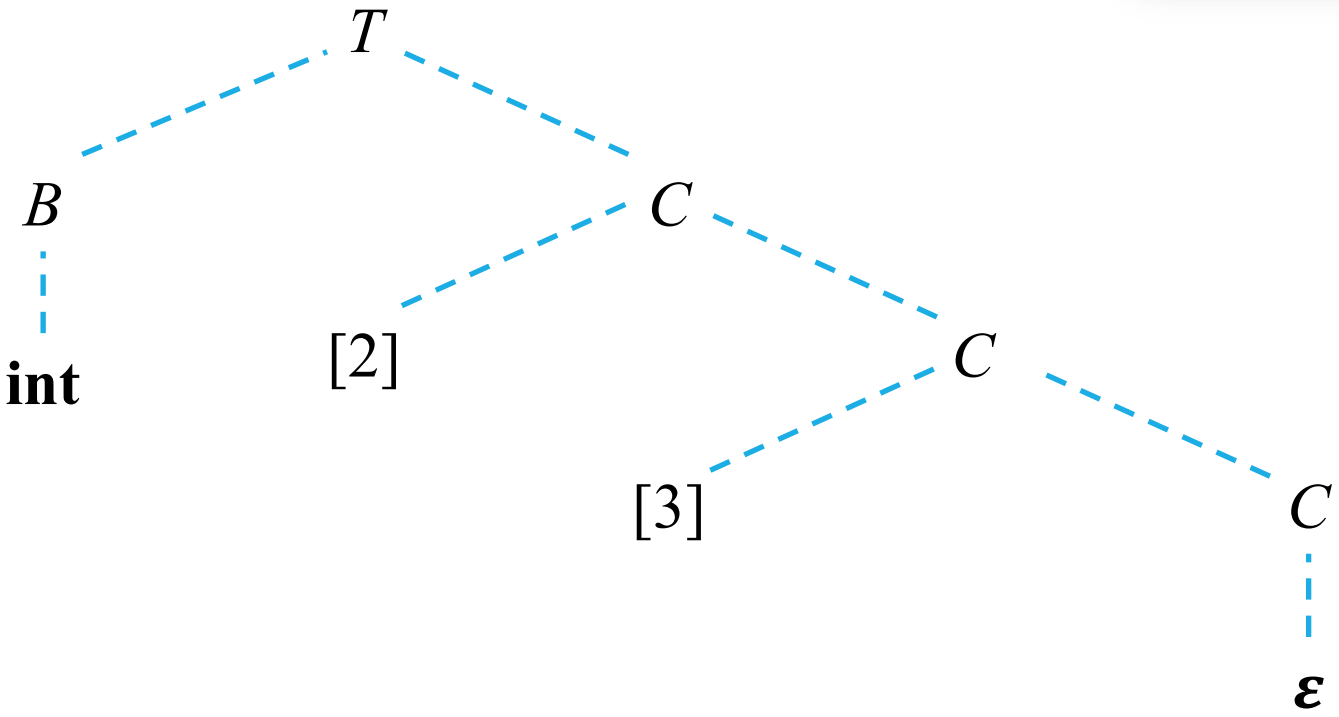
SDT for Type Expressions

- Compute the **type** as well as the **type width**

$$
\begin{array}{lcll}
T & \rightarrow & B & \{\ t = B.type;\ w = B.width;\ \} \\
 & & C & \{\ T.type = C.type;\ T.width = C.width;\ \} \\
B & \rightarrow & \textbf{int} & \{\ B.type = integer;\ B.width = 4;\ \} \\
B & \rightarrow & \textbf{float} & \{\ B.type = float;\ B.width = 8;\ \} \\
C & \rightarrow & \epsilon & \{\ C.type = t;\ C.width = w;\ \} \\
C & \rightarrow & [\ \textbf{num}\ ]\ C_1 & \{\ C.type = array(\textbf{num}.value,\ C_1.type); \\
 & & & \quad C.width = \textbf{num}.value \times C_1.width;\ \}
\end{array}
$$

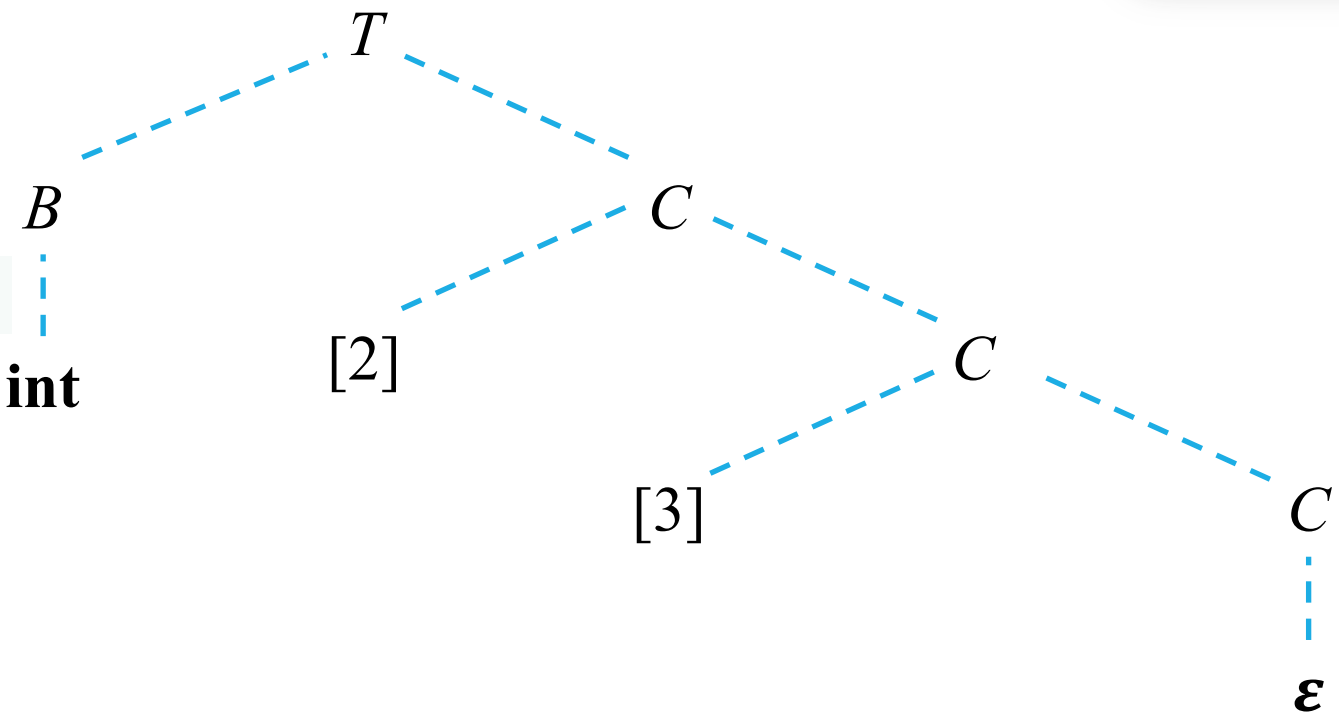
Example: int[2][3]

T	$\rightarrow$	B	$\{\ t = B.type;\ w = B.width;\ \}$
		C	$\{\ T.type = C.type;\ T.width = C.width;\ \}$
B	$\rightarrow$	**int**	$\{\ B.type = integer;\ B.width = 4;\ \}$
B	$\rightarrow$	**float**	$\{\ B.type = float;\ B.width = 8;\ \}$
C	$\rightarrow$	ϵ	$\{\ C.type = t;\ C.width = w;\ \}$
C	$\rightarrow$	[**num**] C_1	$\{\ C.type = array(\mathbf{num}.value,\ C_1.type);$ $C.width = \mathbf{num}.value \times C_1.width;\ \}$



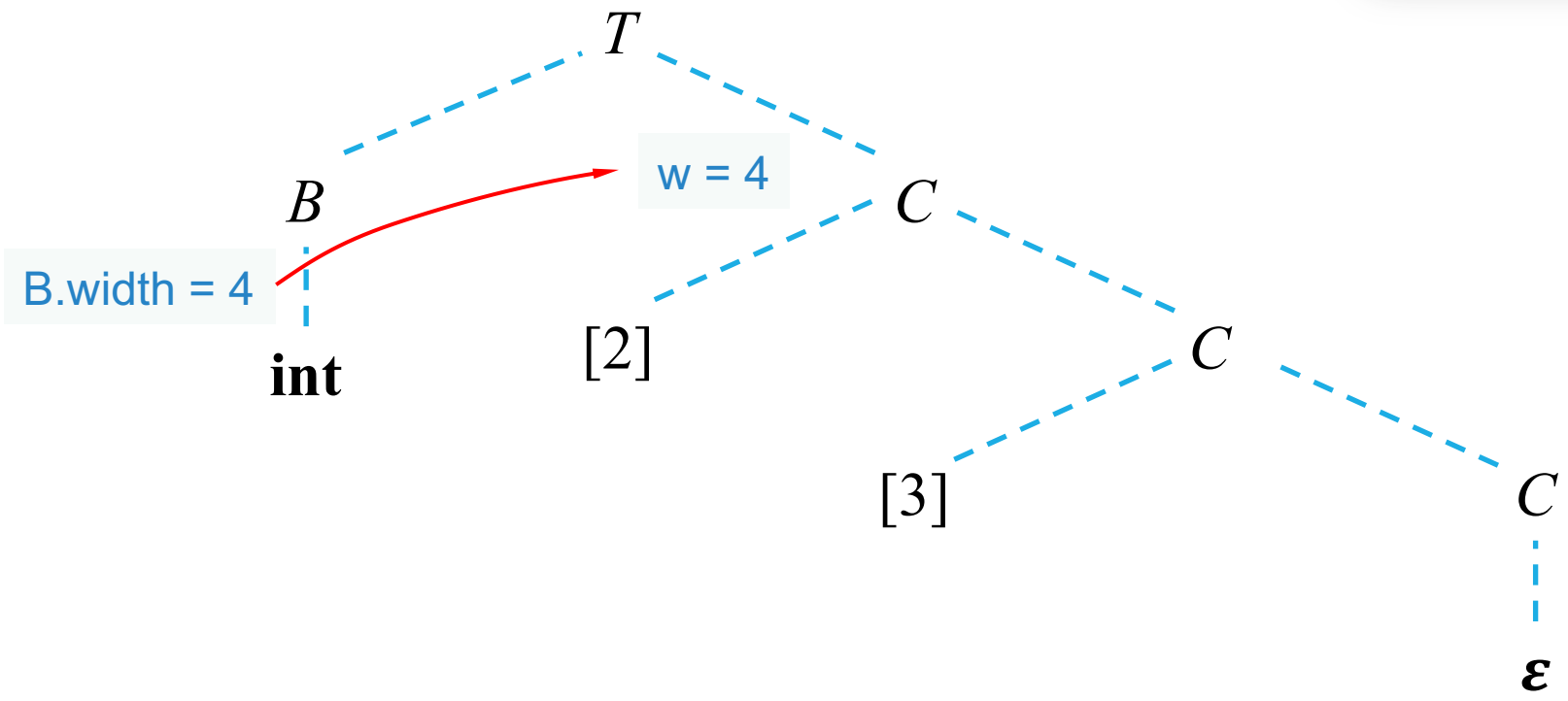
Example: int[2][3]

T	$\rightarrow$	B	$\{\ t = B.type;\ w = B.width;\ \}$
		C	$\{\ T.type = C.type;\ T.width = C.width;\ \}$
B	$\rightarrow$	**int**	$\{\ B.type = integer;\ B.width = 4;\ \}$
B	$\rightarrow$	**float**	$\{\ B.type = float;\ B.width = 8;\ \}$
C	$\rightarrow$	ϵ	$\{\ C.type = t;\ C.width = w;\ \}$
C	$\rightarrow$	[**num**] C_1	$\{\ C.type = array(\mathbf{num}.value,\ C_1.type);$ $C.width = \mathbf{num}.value \times C_1.width;\ \}$



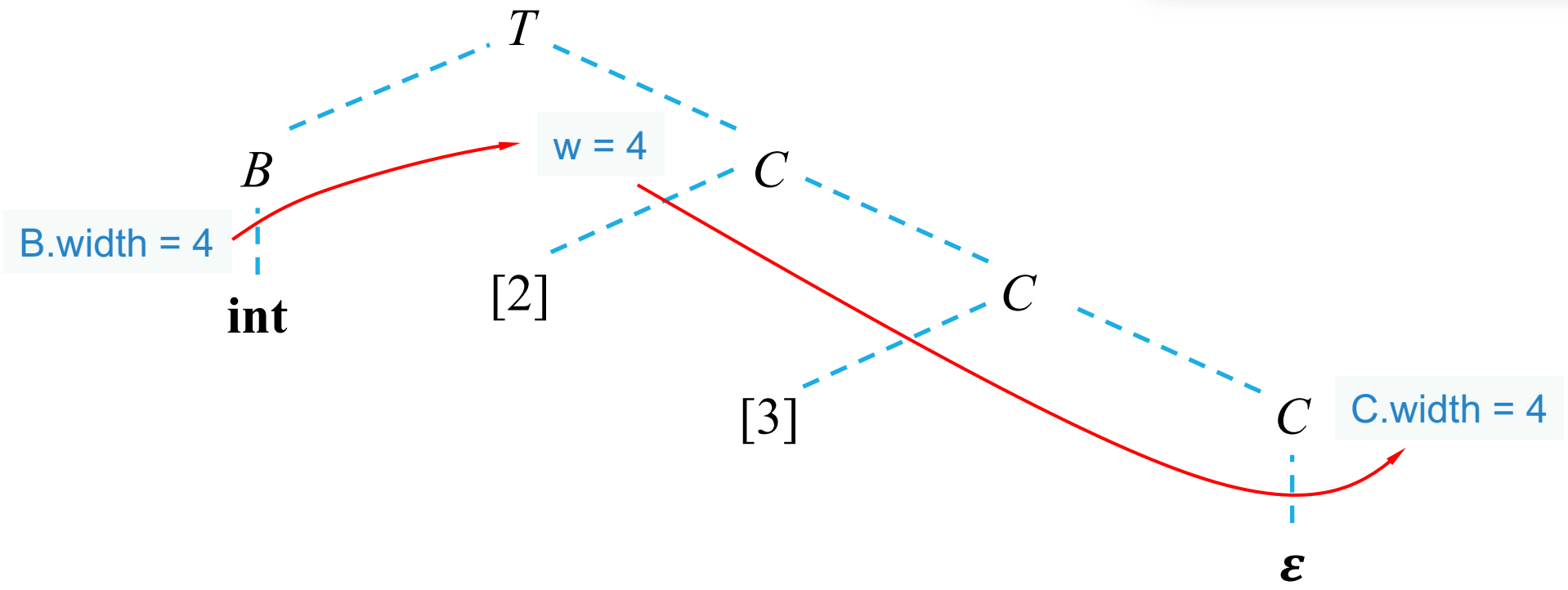
Example: int[2][3]

T	$\rightarrow$	B	$\{\ t = B.type;\ w = B.width;\ \}$
		C	$\{\ T.type = C.type;\ T.width = C.width;\ \}$
B	$\rightarrow$	**int**	$\{\ B.type = integer;\ B.width = 4;\ \}$
B	$\rightarrow$	**float**	$\{\ B.type = float;\ B.width = 8;\ \}$
C	$\rightarrow$	ϵ	$\{\ C.type = t;\ C.width = w;\ \}$
C	$\rightarrow$	[**num**] C_1	$\{\ C.type = array(\mathbf{num}.value,\ C_1.type);$ $C.width = \mathbf{num}.value \times C_1.width;\ \}$



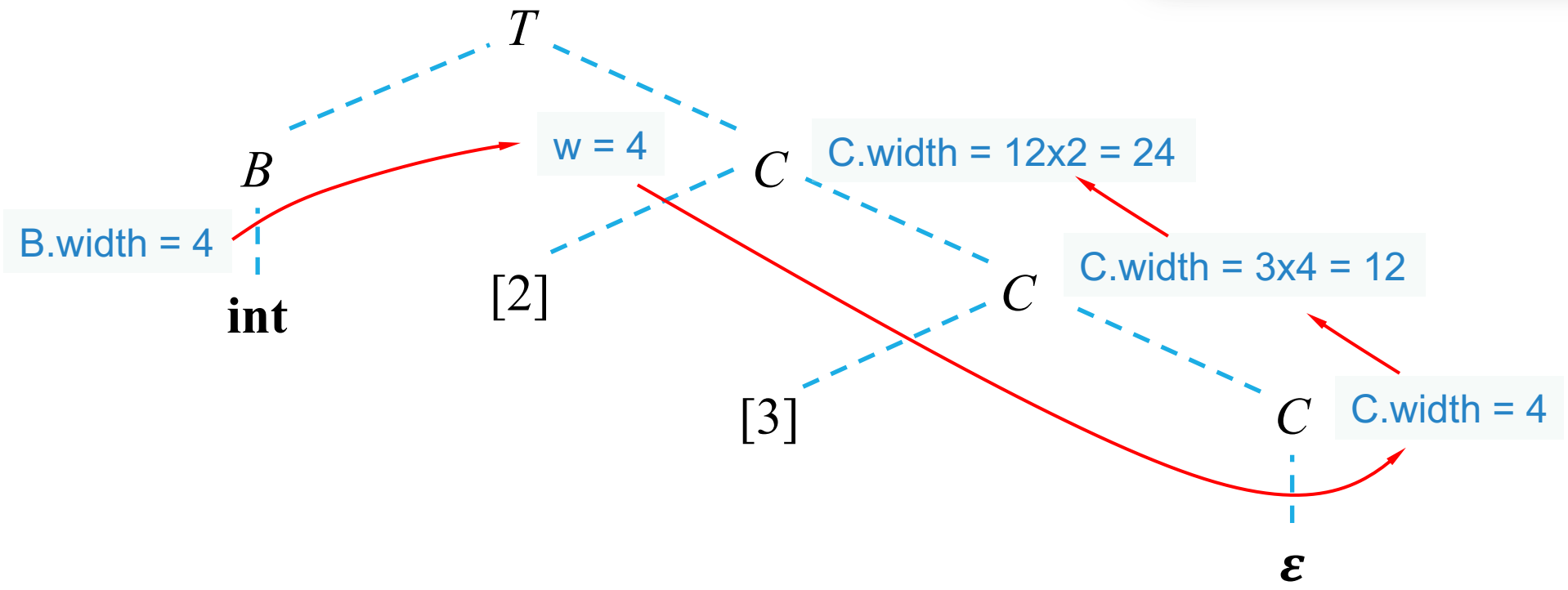
Example: int[2][3]

T	$\rightarrow$	B	$\{\ t = B.type;\ w = B.width;\ \}$
		C	$\{\ T.type = C.type;\ T.width = C.width;\ \}$
B	$\rightarrow$	**int**	$\{\ B.type = integer;\ B.width = 4;\ \}$
B	$\rightarrow$	**float**	$\{\ B.type = float;\ B.width = 8;\ \}$
C	$\rightarrow$	ϵ	$\{\ C.type = t;\ C.width = w;\ \}$
C	$\rightarrow$	[**num**] C_1	$\{\ C.type = array(\mathbf{num}.value,\ C_1.type);$ $C.width = \mathbf{num}.value \times C_1.width;\ \}$



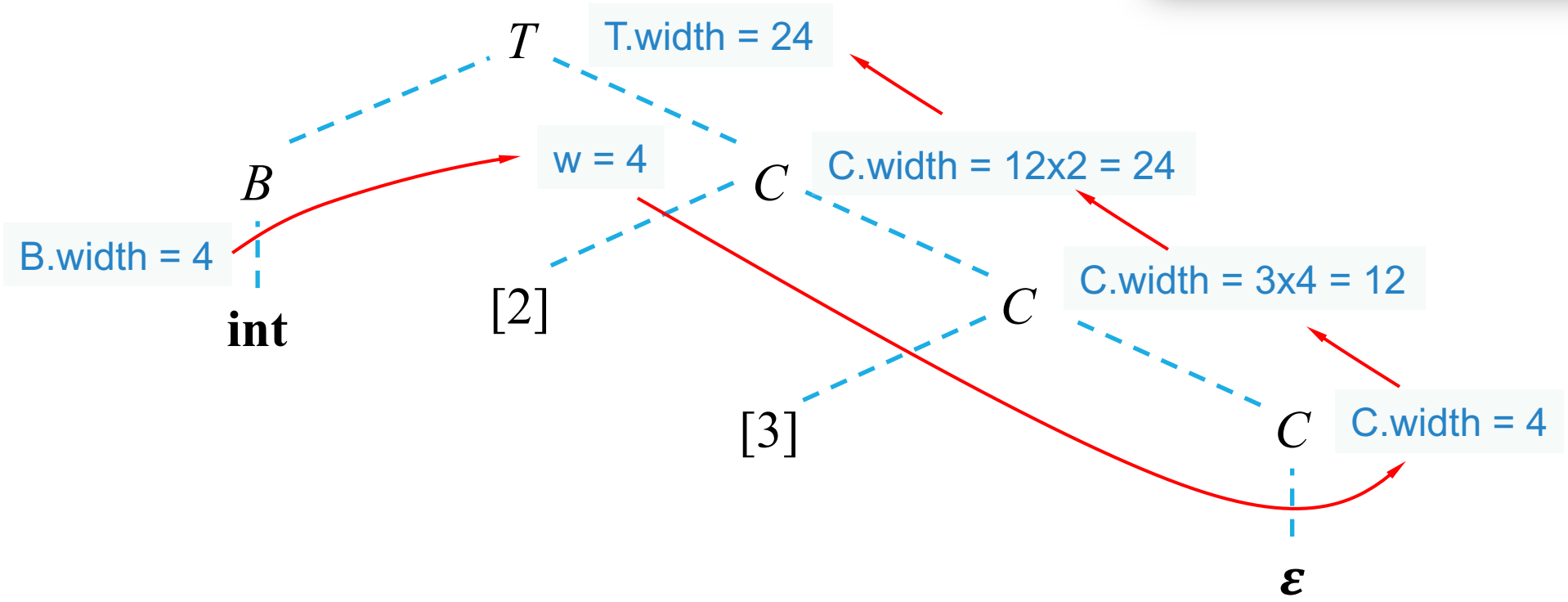
Example: int[2][3]

T	$\rightarrow$	B	$\{\ t = B.type;\ w = B.width;\ \}$
		C	$\{\ T.type = C.type;\ T.width = C.width;\ \}$
B	$\rightarrow$	**int**	$\{\ B.type = integer;\ B.width = 4;\ \}$
B	$\rightarrow$	**float**	$\{\ B.type = float;\ B.width = 8;\ \}$
C	$\rightarrow$	ϵ	$\{\ C.type = t;\ C.width = w;\ \}$
C	$\rightarrow$	[**num**] C_1	$\{\ C.type = array(\mathbf{num}.value,\ C_1.type);$ $C.width = \mathbf{num}.value \times C_1.width;\ \}$



Example: int[2][3]

T	$\rightarrow$	B	$\{\ t = B.type;\ w = B.width;\ \}$
		C	$\{\ T.type = C.type;\ T.width = C.width;\ \}$
B	$\rightarrow$	**int**	$\{\ B.type = integer;\ B.width = 4;\ \}$
B	$\rightarrow$	**float**	$\{\ B.type = float;\ B.width = 8;\ \}$
C	$\rightarrow$	ϵ	$\{\ C.type = t;\ C.width = w;\ \}$
C	$\rightarrow$	[**num**] C_1	$\{\ C.type = array(\mathbf{num}.value,\ C_1.type);$ $C.width = \mathbf{num}.value \times C_1.width;\ \}$



SDT for Variable Declarations

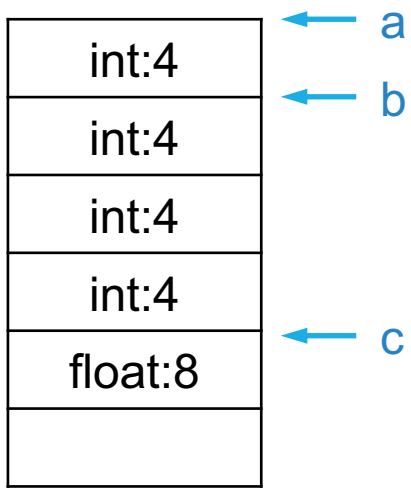
$$
\begin{array}{lcl}
D & \rightarrow & T\ \mathbf{id}\ ;\ D \mid \epsilon \\
T & \rightarrow & B\ C \mid \mathbf{record}\ '\{'\ D\ '\}' \\
B & \rightarrow & \mathbf{int} \mid \mathbf{float} \\
C & \rightarrow & \epsilon \mid [\ \mathbf{num}\ ]\ C
\end{array}
$$

SDT for Variable Declarations

$$
\begin{array}{lcl}
D & \rightarrow & T\ \mathbf{id}\ ;\ D \mid \epsilon \\
T & \rightarrow & B\ C \mid \mathbf{record}\ '\{'\ D\ '\}' \\
B & \rightarrow & \mathbf{int} \mid \mathbf{float} \\
C & \rightarrow & \epsilon \mid [\ \mathbf{num}\ ]\ C
\end{array}
$$

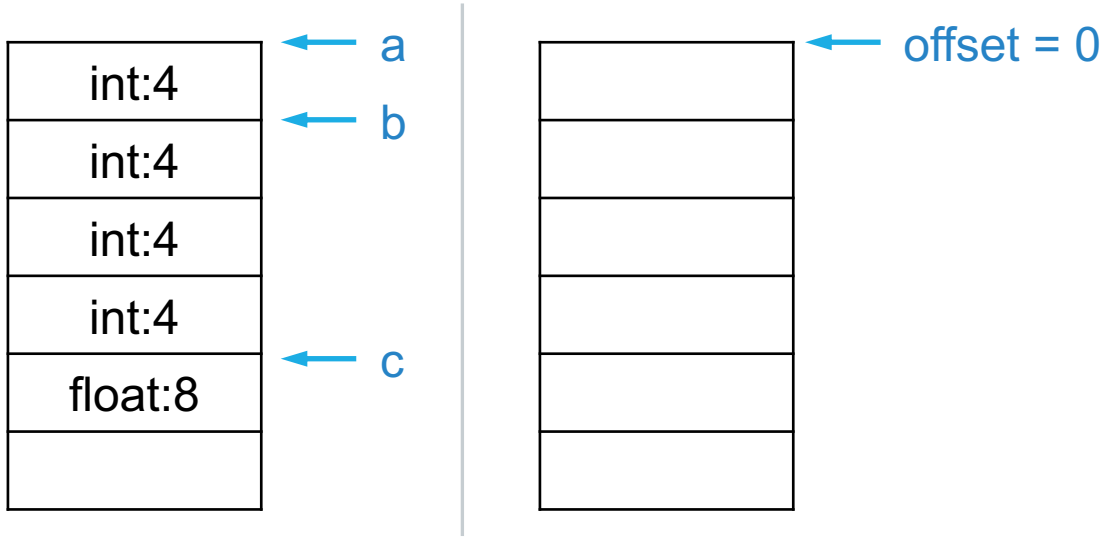
SDT for Variable Declarations

- **int** a; **int**[3] b; **float** c;…



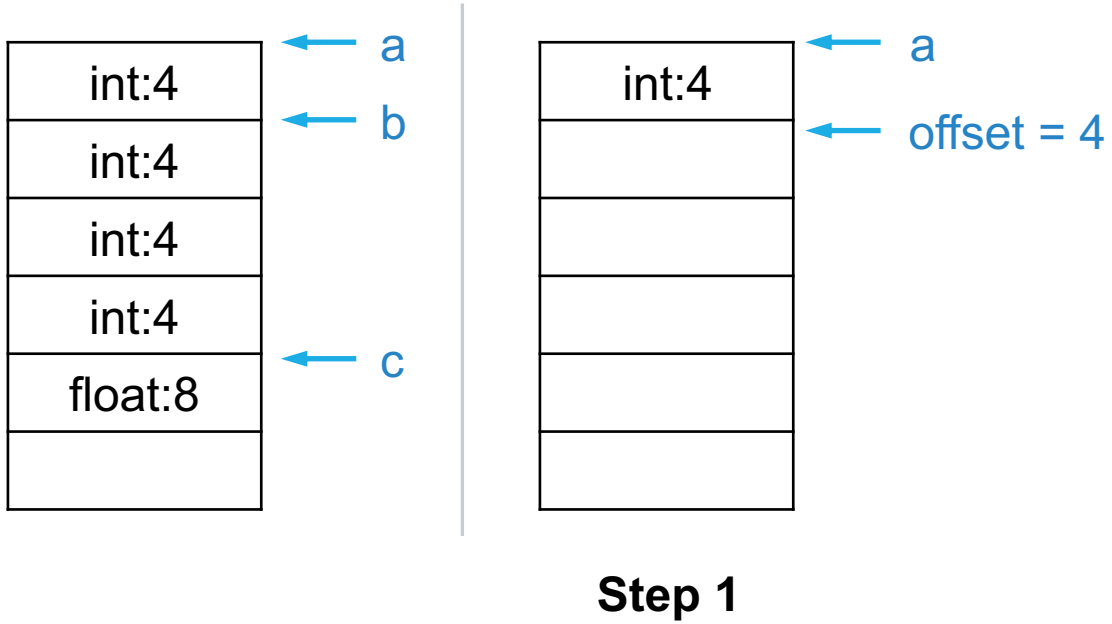
SDT for Variable Declarations

- **int** a; **int**[3] b; **float** c;…



SDT for Variable Declarations

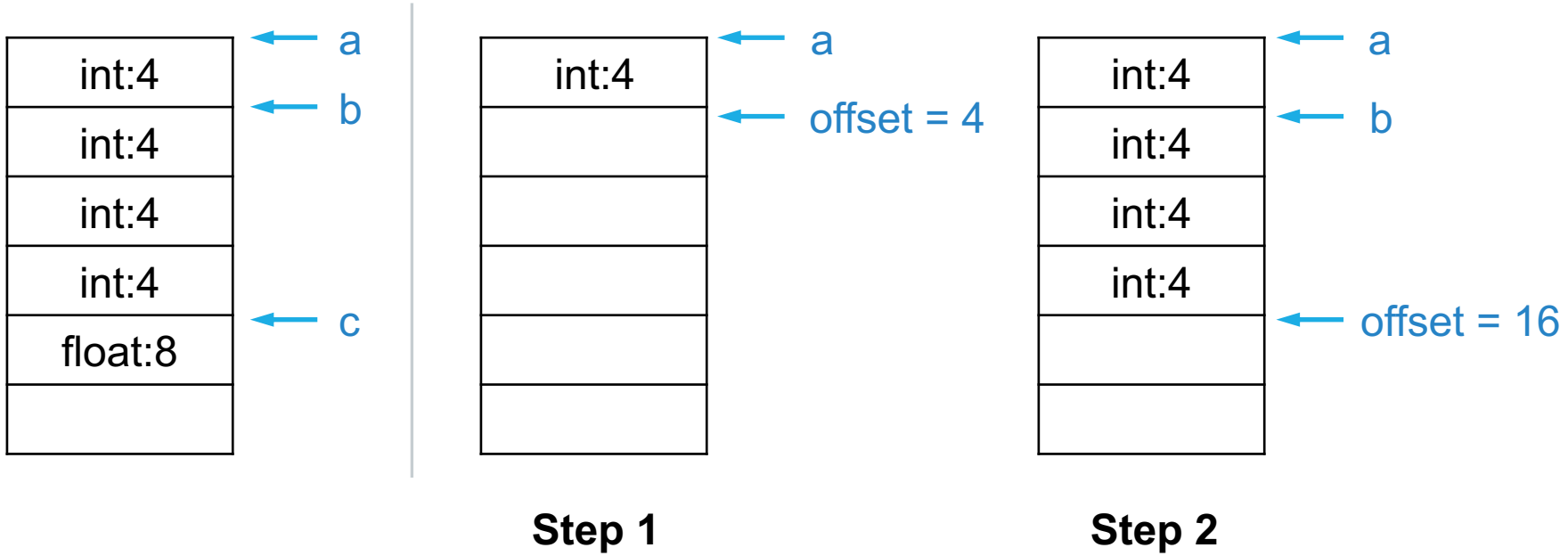
- **int** a; **int**[3] b; **float** c;…



Step 1

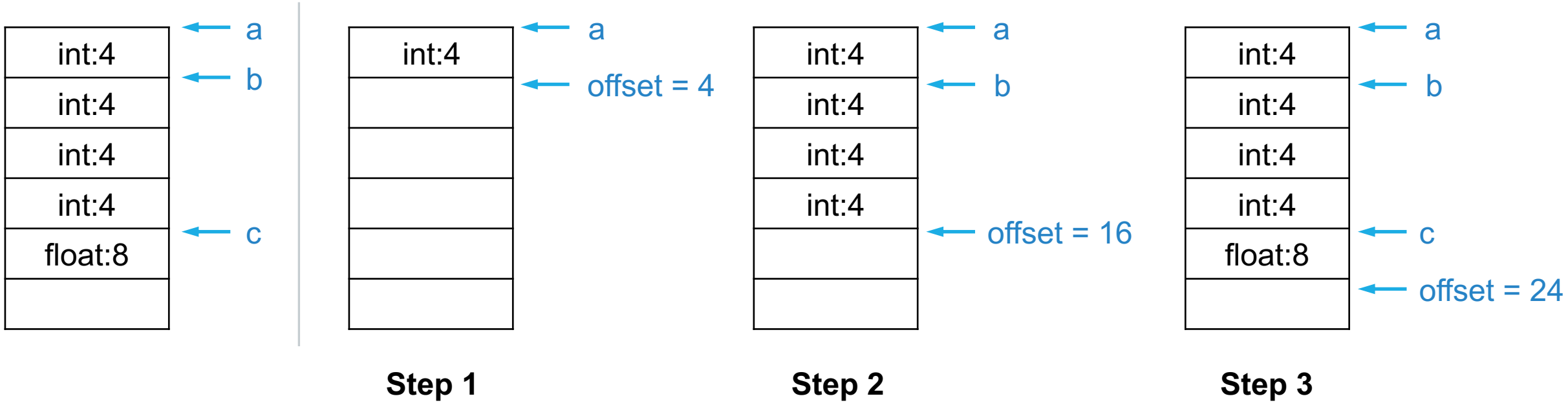
SDT for Variable Declarations

- **int** a; **int**[3] b; **float** c;…



SDT for Variable Declarations

- **int** a; **int**[3] b; **float** c;…



SDT for Variable Declarations

$$\{\ offset = 0;\ \}$$

$$D \rightarrow T\ \mathbf{id}\ ; \quad \{\ top.put(\mathbf{id}.lexeme,\ T.type,\ offset);\ offset = offset + T.width;\ \}$$

$$D \rightarrow D_1$$

$$D \rightarrow \epsilon$$

SDT for Variable Declarations

$$\{\ offset = 0;\ \}$$

Simulate the procedure of allocating space on stack!

$$D \rightarrow T\ \mathbf{id}\ ; \quad \{\ top.put(\mathbf{id}.lexeme,\ T.type,\ offset);\ offset = offset + T.width;\ \}$$

$$D \rightarrow D_1$$

$$D \rightarrow \epsilon$$

SDT for Variable Declarations

$$\{\ offset = 0;\ \}$$

Simulate the procedure of allocating space on stack!

$$D \rightarrow T\ \mathbf{id}\ ; \quad \{\ top.put(\mathbf{id}.lexeme,\ T.type,\ offset);\ offset = offset + T.width;\ \}$$

Advance the offset

$$D \rightarrow D_1$$

$$D \rightarrow \epsilon$$

SDT for Record Type

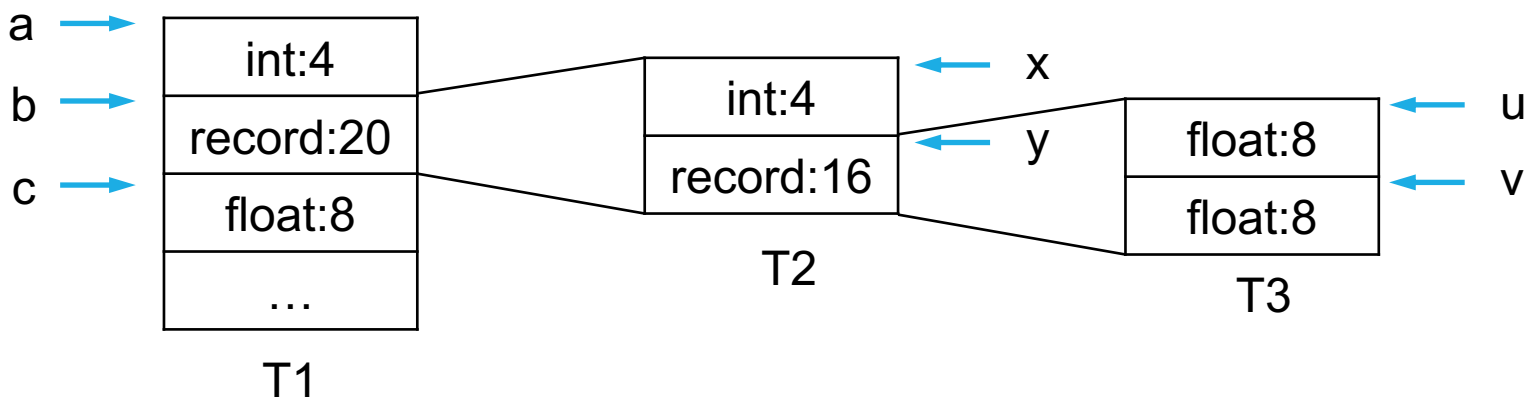
$$
\begin{array}{lcl}
D & \rightarrow & T\ \mathbf{id}\ ;\ D \mid \epsilon \\
T & \rightarrow & B\ C \mid \mathbf{record}\ \text{'\{'}\ D\ \text{'\}'} \\
B & \rightarrow & \mathbf{int} \mid \mathbf{float} \\
C & \rightarrow & \epsilon \mid [\ \mathbf{num}\ ]\ C
\end{array}
$$

SDT for Record Type

$$
\begin{array}{lcl}
D & \rightarrow & T\ \mathbf{id}\ ;\ D \mid \epsilon \\
T & \rightarrow & B\ C \mid \mathbf{record}\ '\{'\ D\ '\}' \\
B & \rightarrow & \mathbf{int} \mid \mathbf{float} \\
C & \rightarrow & \epsilon \mid [\ \mathbf{num}\ ]\ C
\end{array}
$$

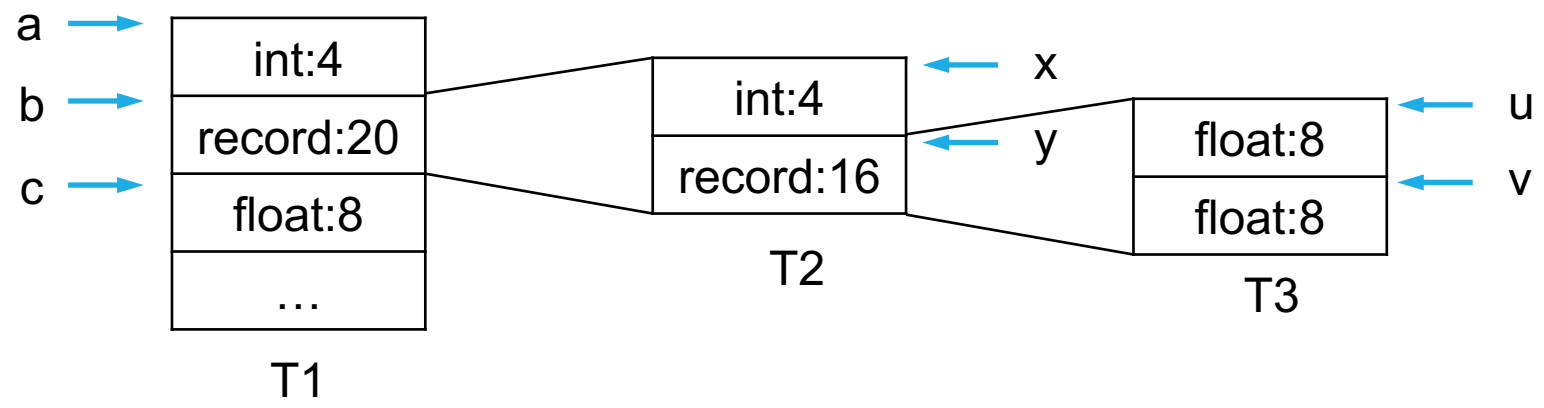
SDT for Record Type

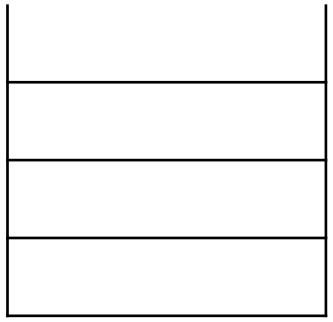
- **int** a; **record { int** x; **record { float** u; **float** v;**}** y; **}** b; **float** c;



SDT for Record Type

- **int** a; **record { int** x; **record { float** u; **float** v;**}** y; **}** b; **float** c;





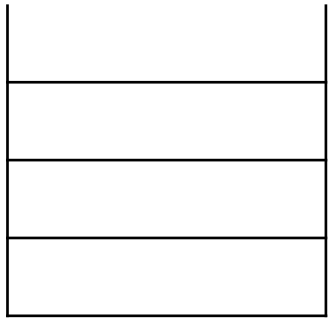
Stack

Build a stack for nested types;
Once the parser visits '{', a new type is created and pushed to the stack;
Once the parser visits '}', the top-of-stack type is popped.

SDT for Record Type

- **int** a; **record { int** x; **record { float** u; **float** v;**}** y; **}** b; **float** c;

parser

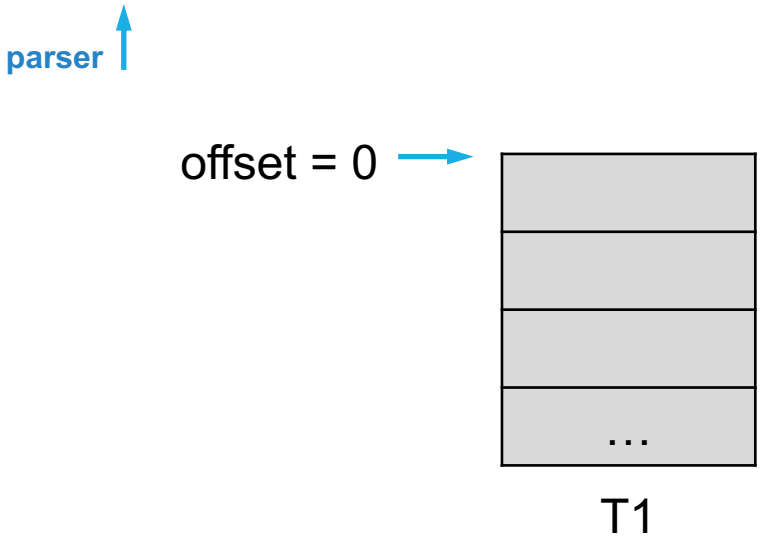


Stack

Build a stack for nested types;
Once the parser visits '{', a new type is created and pushed to the stack;
Once the parser visits '}', the top-of-stack type is popped.

SDT for Record Type

- **int** a; **record { int** x; **record { float** u; **float** v;**}** y; **}** b; **float** c;



Build a stack for nested types;
Once the parser visits '{', a new type is created and pushed to the stack;
Once the parser visits '}', the top-of-stack type is popped.

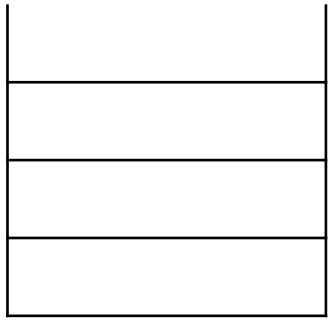
SDT for Record Type

- **int** a; **record { int** x; **record { float** u; **float** v;**}** y; **}** b; **float** c;

parser

offset = 0 →

T1



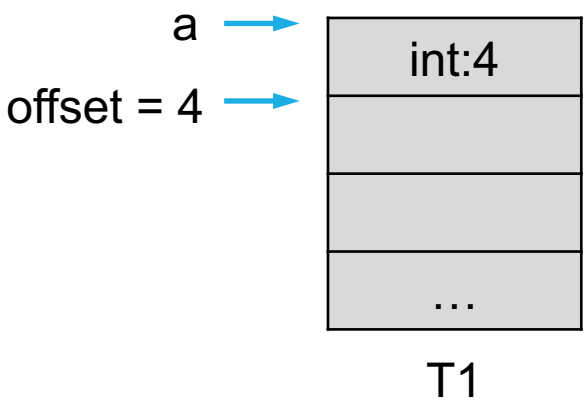
Stack

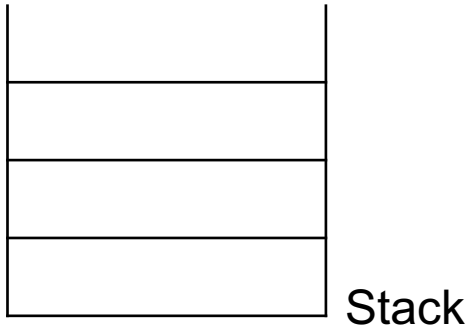
Build a stack for nested types;
Once the parser visits '{', a new type is created and pushed to the stack;
Once the parser visits '}', the top-of-stack type is popped.

SDT for Record Type

- **int** a; **record { int** x; **record { float** u; **float** v;**}** y; **}** b; **float** c;

parser



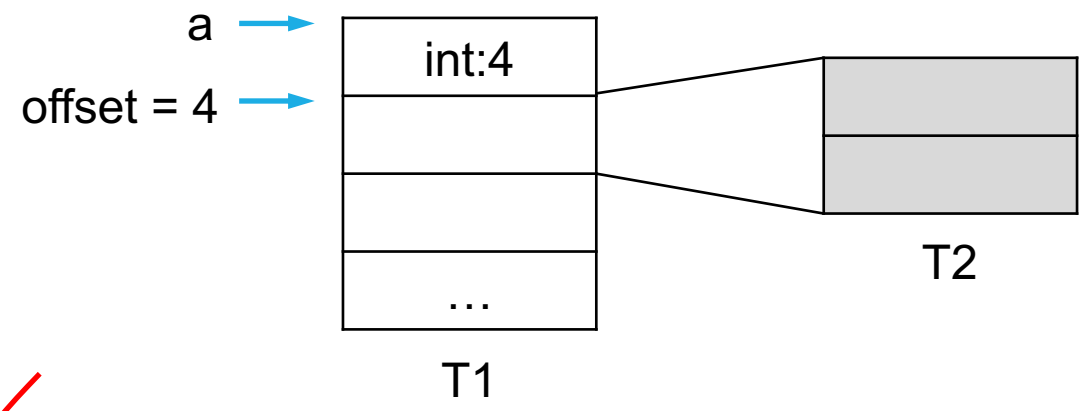


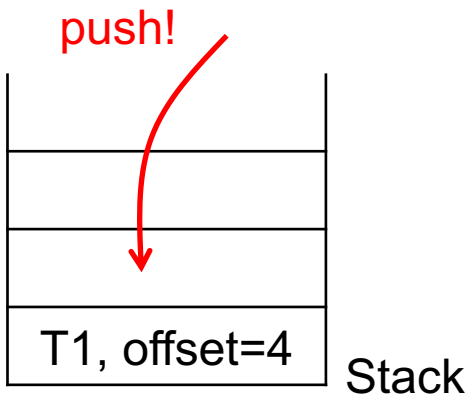
Build a stack for nested types;
Once the parser visits '{', a new type is created and pushed to the stack;
Once the parser visits '}', the top-of-stack type is popped.

SDT for Record Type

- **int** a; **record { int** x; **record { float** u; **float** v;**}** y; **}** b; **float** c;

parser





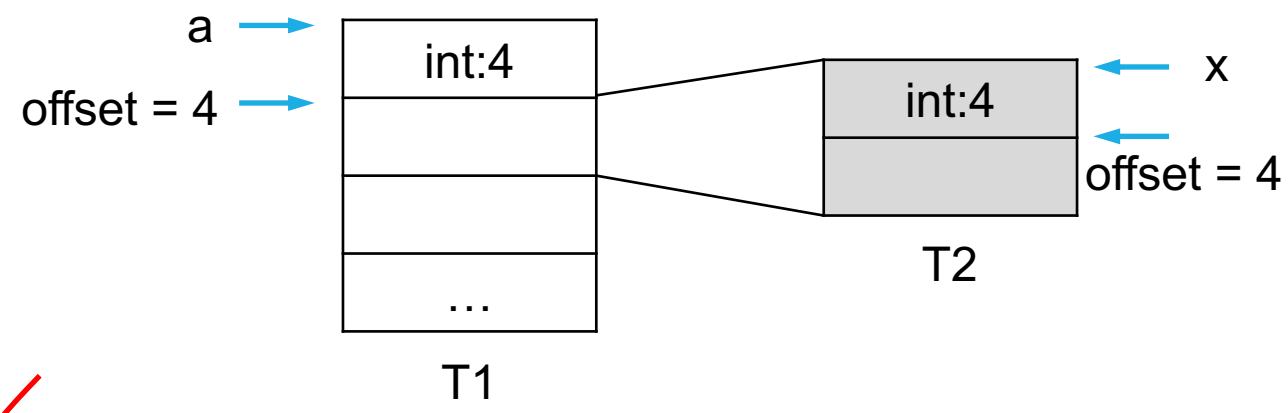
Build a stack for nested types;

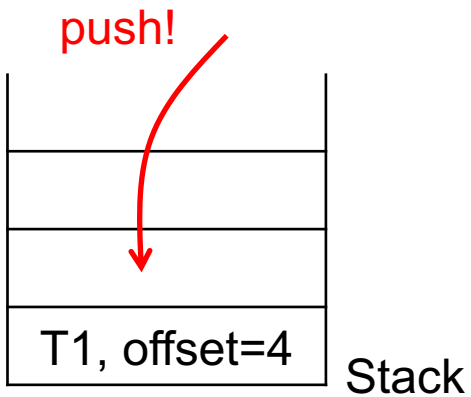
Once the parser visits '{', a new type is created and pushed to the stack;

Once the parser visits '}', the top-of-stack type is popped.

SDT for Record Type

- **int** a; **record { int** x; **record { float** u; **float** v;**}** y; **}** b; **float** c;





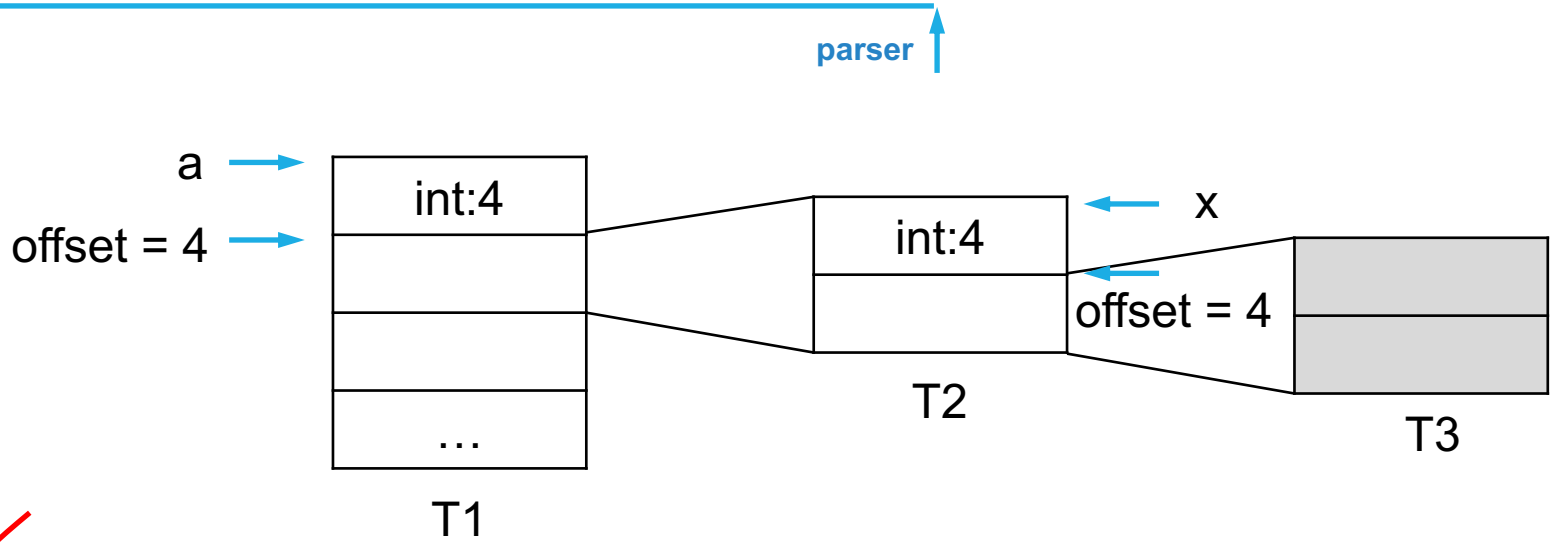
Build a stack for nested types;

Once the parser visits '{', a new type is created and pushed to the stack;

Once the parser visits '}', the top-of-stack type is popped.

SDT for Record Type

- **int** a; **record { int** x; **record { float** u; **float** v;**}** y; **}** b; **float** c;



Build a stack for nested types;

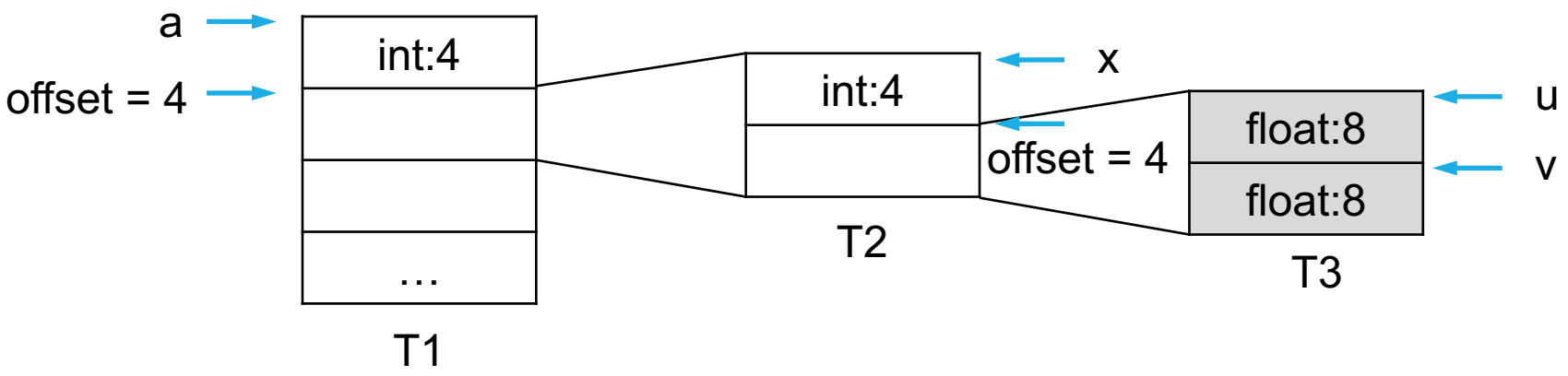
Once the parser visits '{', a new type is created and pushed to the stack;

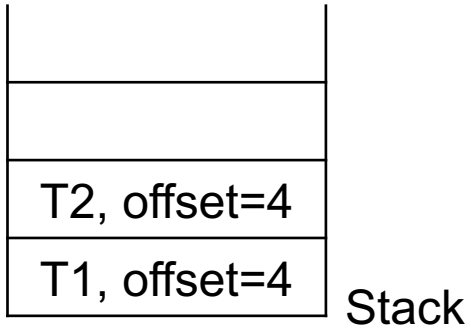
Once the parser visits '}', the top-of-stack type is popped.

SDT for Record Type

- **int** a; **record { int** x; **record { float** u; **float** v;**}** y; **}** b; **float** c;

parser





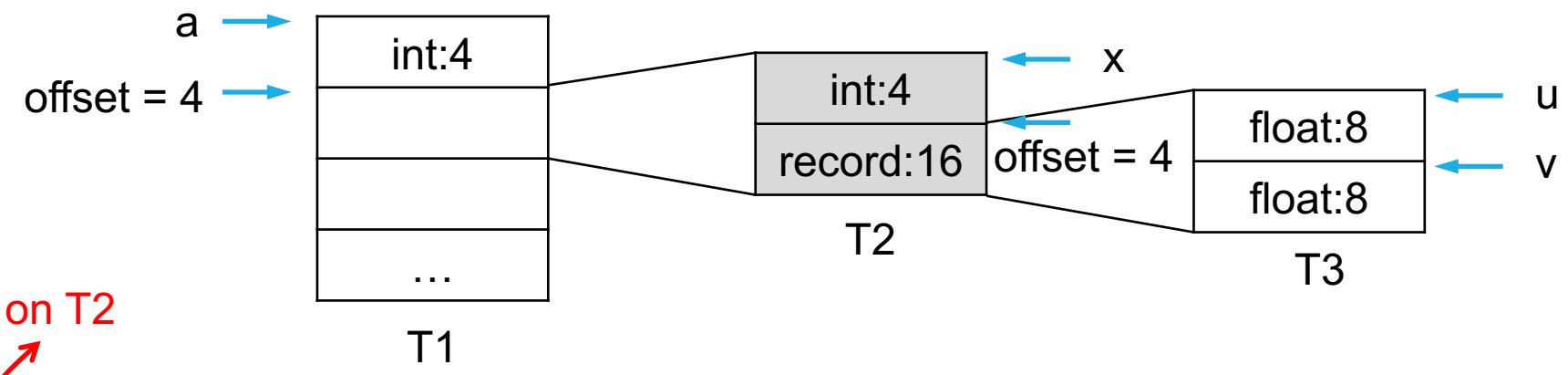
Build a stack for nested types;

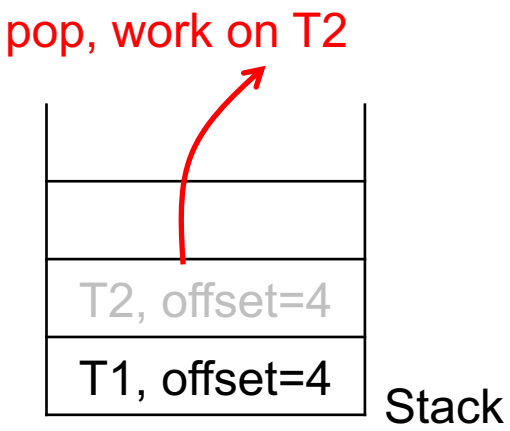
Once the parser visits '{', a new type is created and pushed to the stack;

Once the parser visits '}', the top-of-stack type is popped.

SDT for Record Type

- **int** a; **record { int** x; **record { float** u; **float** v;**}** y; **}** b; **float** c;



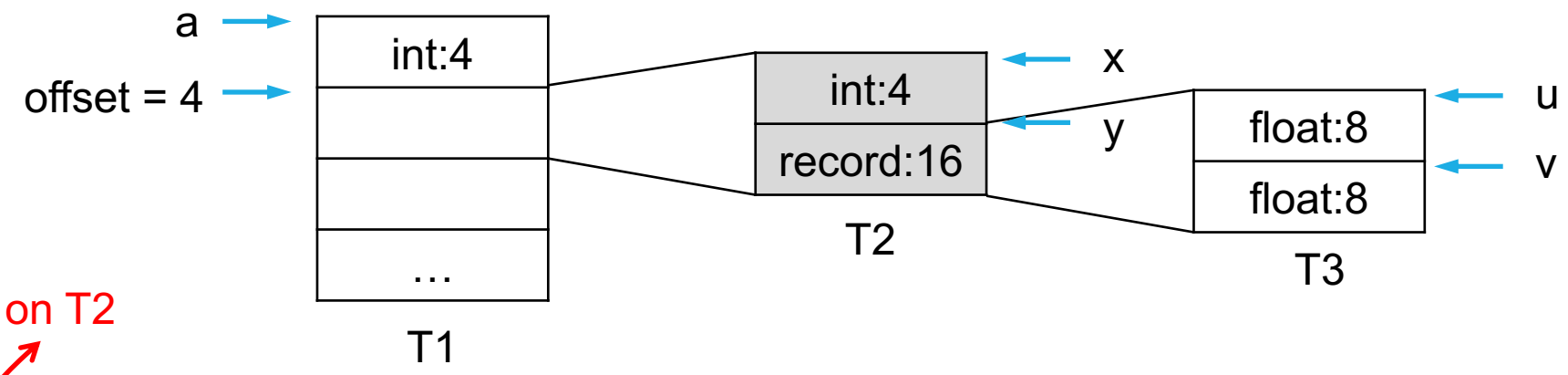


Build a stack for nested types;
Once the parser visits '{', a new type is created and pushed to the stack;
Once the parser visits '}', the top-of-stack type is popped.

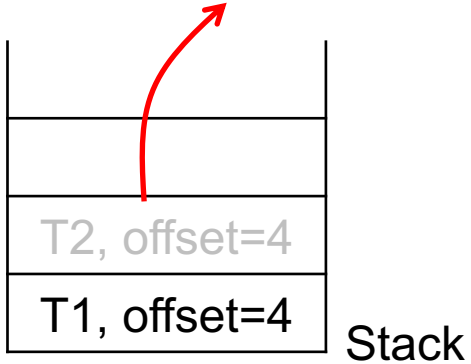
SDT for Record Type

- **int** a; **record { int** x; **record { float** u; **float** v;**}** y; **}** b; **float** c;

parser



pop, work on T2



Build a stack for nested types;

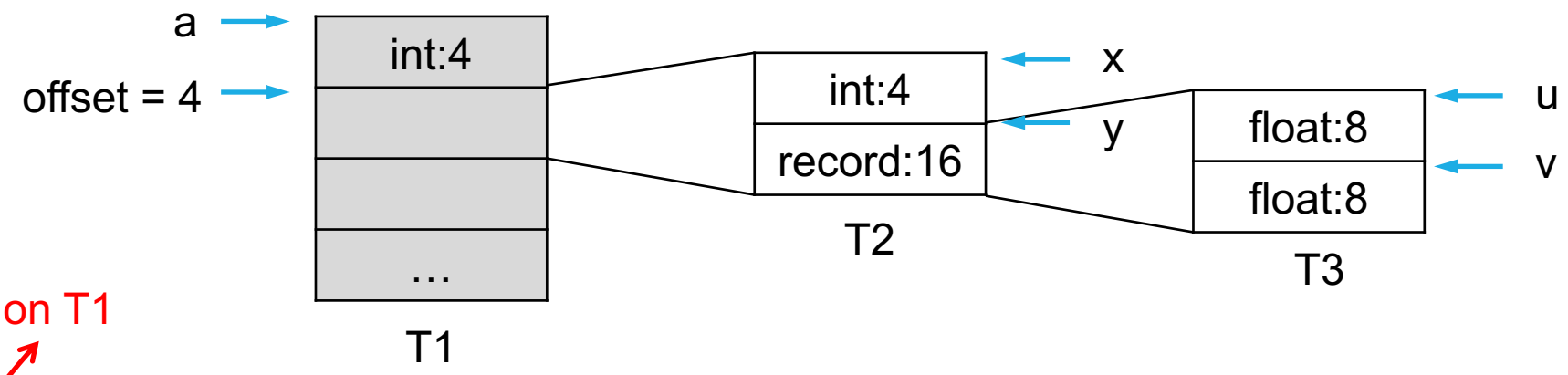
Once the parser visits '{', a new type is created and pushed to the stack;

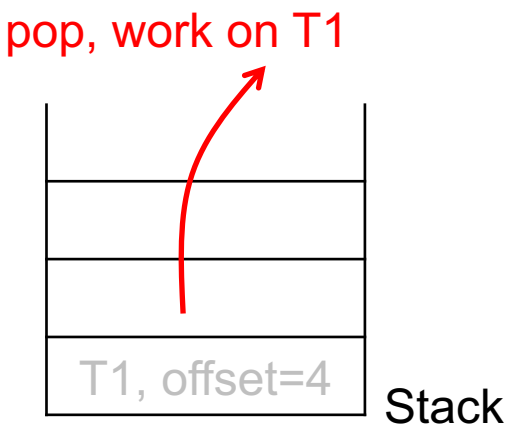
Once the parser visits '}', the top-of-stack type is popped.

SDT for Record Type

- **int** a; **record { int** x; **record { float** u; **float** v;**}** y; **}** b; **float** c;

parser





Build a stack for nested types;

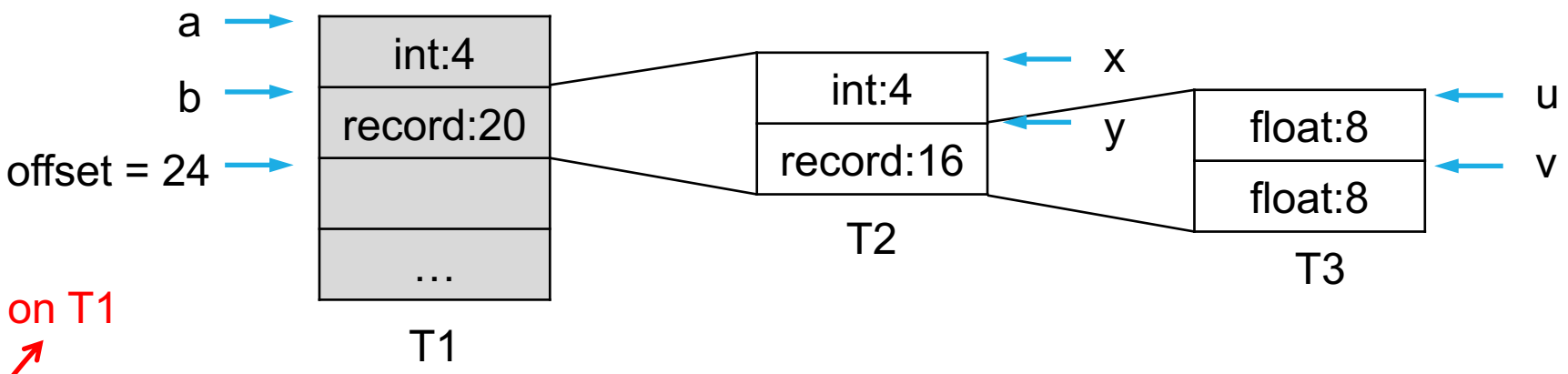
Once the parser visits '{', a new type is created and pushed to the stack;

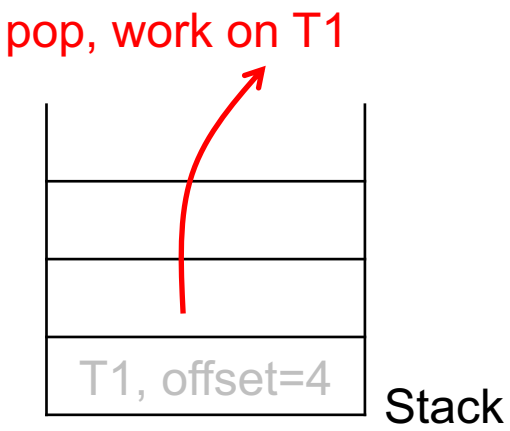
Once the parser visits '}', the top-of-stack type is popped.

SDT for Record Type

- **int** a; **record { int** x; **record { float** u; **float** v;**}** y; **}** b; **float** c;

parser



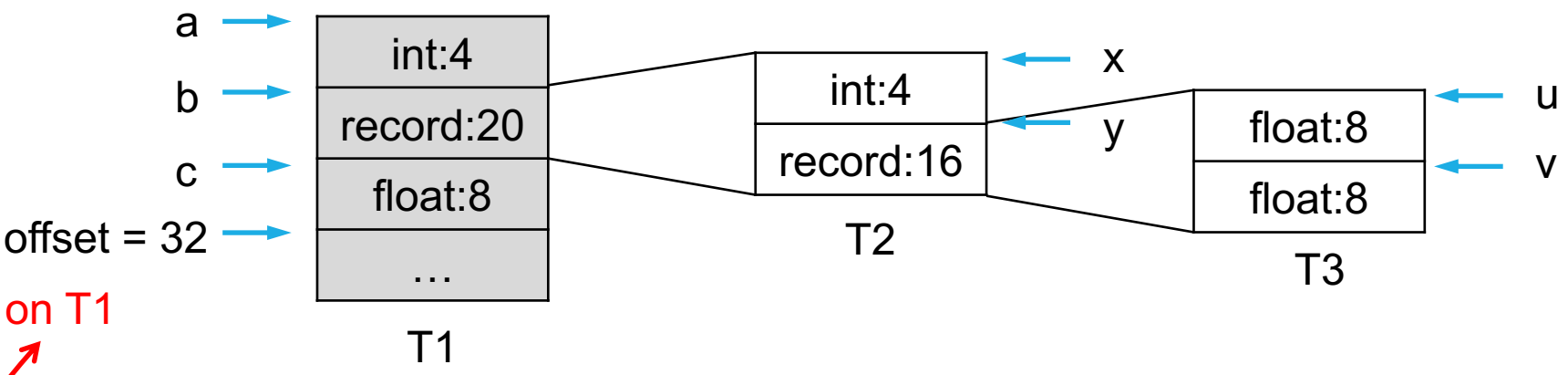


Build a stack for nested types;
Once the parser visits '{', a new type is created and pushed to the stack;
Once the parser visits '}', the top-of-stack type is popped.

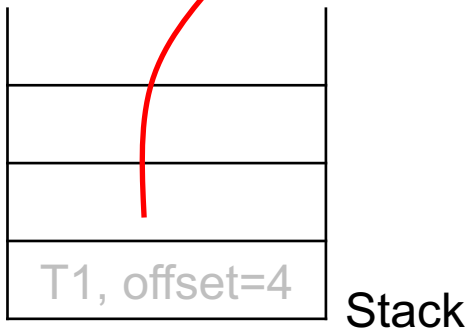
SDT for Record Type

- **int** a; **record { int** x; **record { float** u; **float** v;**}** y; **}** b; **float** c;

parser



pop, work on T1



Stack

Build a stack for nested types;

Once the parser visits '{', a new type is created and pushed to the stack;

Once the parser visits '}', the top-of-stack type is popped.

SDT for Record Type

$$
\begin{array}{lll}
T \rightarrow \textbf{record}\ '\{' & \{\ Env.push(top);\ top = \textbf{new}\ Env();\\
 & \quad Stack.push(offset);\ offset = 0;\ \} \\
D\ '\}' & \{\ T.type = record(top);\ T.width = offset;\\
 & \quad top = Env.pop();\ offset = Stack.pop();\ \}
\end{array}
$$

Read the Dragon book for details…

PART II-3:
Translating Statements

Expressions

```
int main(){
    int a; int b; int c;
    scanf("%d%d", &a, &b);
    c = a + b;
    printf("%d", c);
    return 0;
}
```

Common Expressions & Statements

Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (E_1)$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

Expressions

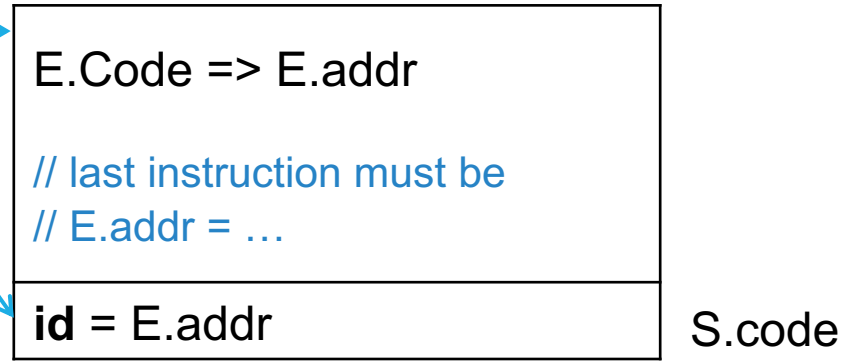
PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

S.code & E.code: code generated by S & E

E.addr: a variable receiving result of E

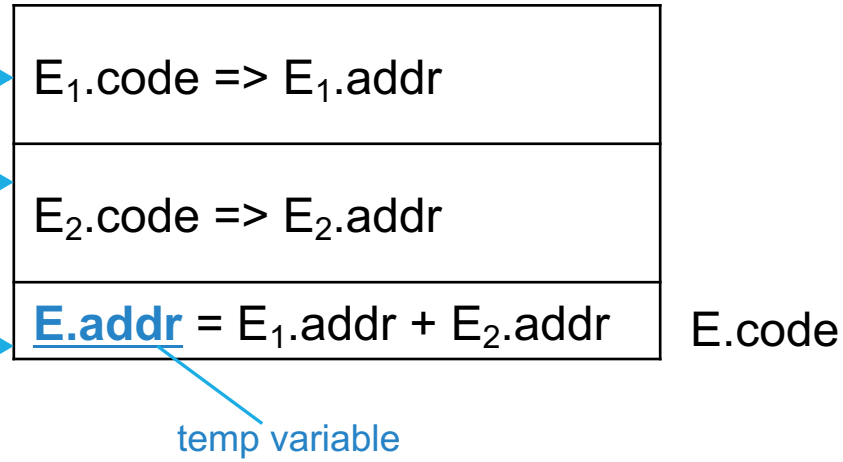
Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$



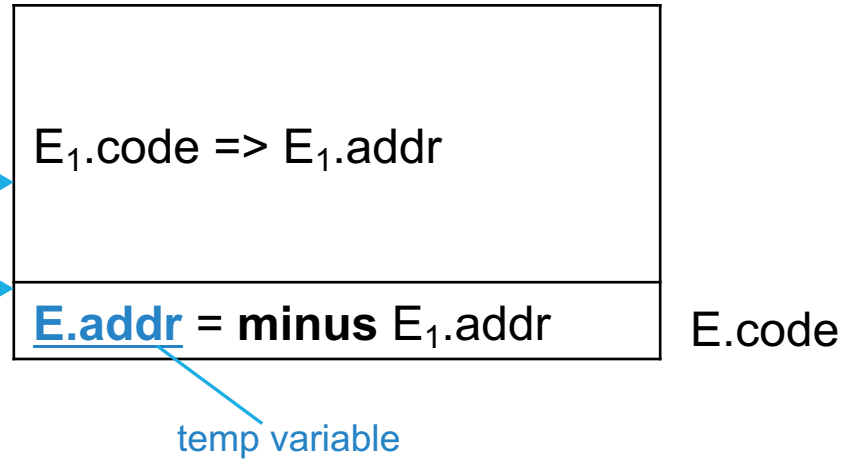
Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ -\ E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$



Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$



Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \textbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\textbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \textbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \textbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\textbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \textbf{id}$	$E.addr = top.get(\textbf{id}.lexeme)$ $E.code = ''$

Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

Example: `x = y + (-c)`

$temp_2$ = **minus** c

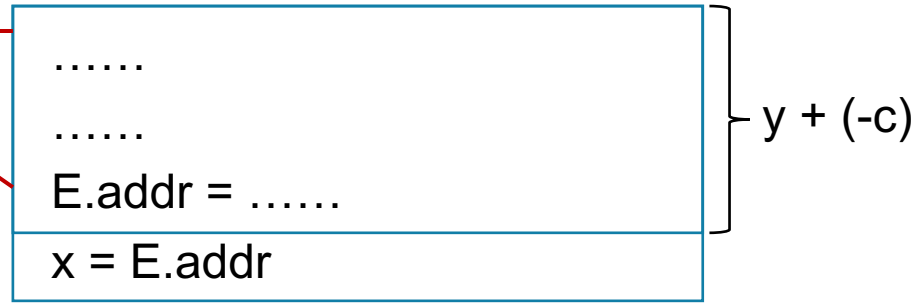
$temp_1$ = y + $temp_2$

x = $temp_1$

Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (E_1)$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

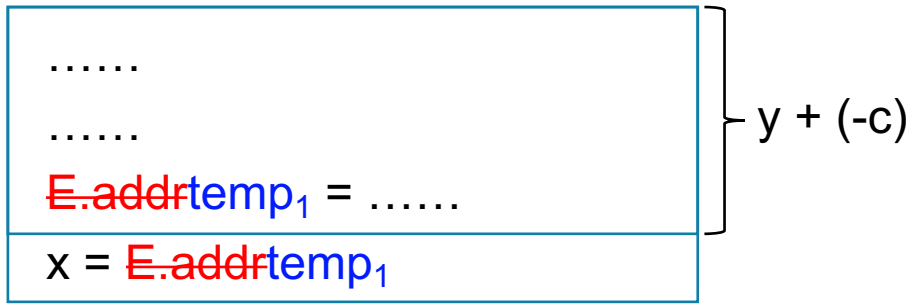
Example: `x = y + (-c)`



Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \textbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\textbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \textbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \textbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\textbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \textbf{id}$	$E.addr = top.get(\textbf{id}.lexeme)$ $E.code = ''$

Example: x = y + (-c)



Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \textbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\textbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \textbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \textbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\textbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \textbf{id}$	$E.addr = top.get(\textbf{id}.lexeme)$ $E.code = ''$

Example: x = y + (-c)

E1.addr = ……
E2.addr = ……
$temp_1$ = E1.addr + E2.addr

y

(-c)

x = $temp_1$

Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

Example: x = y + (-c)

~~E1.addr = ……~~
E2.addr = ……
$temp_1$ = ~~E1.addr~~ y + E2.addr

y

(-c)

x = $temp_1$

Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

Example: x = y + (-c)

E.addr = ……
$temp_1$ = y + E.addr

y

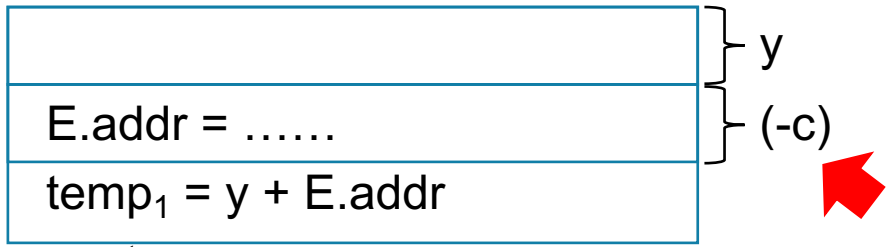
(-c)

x = $temp_1$

Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

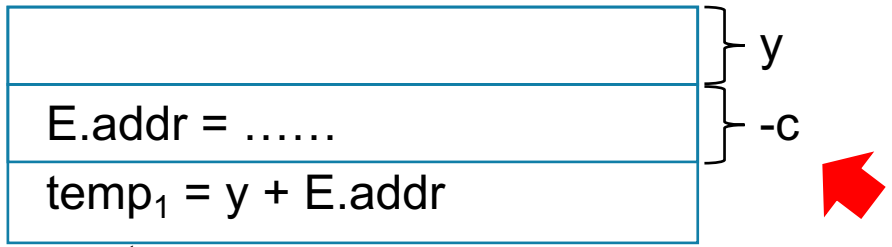
Example: x = y + (-c)



Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

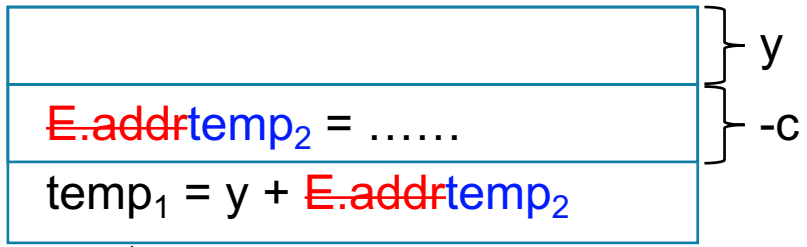
Example: x = y + (-c)



Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

Example: x = y + (-c)



Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

Example: x = y + (-c)

	y
~~E.addr~~temp$_2$ = **minus** c	-c
temp$_1$ = y + ~~E.addr~~temp$_2$	
x = temp$_1$	

Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ - E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

Example: x = y + (-c)

$temp_2$ = **minus** c

$temp_1$ = y + $temp_2$

x = $temp_1$

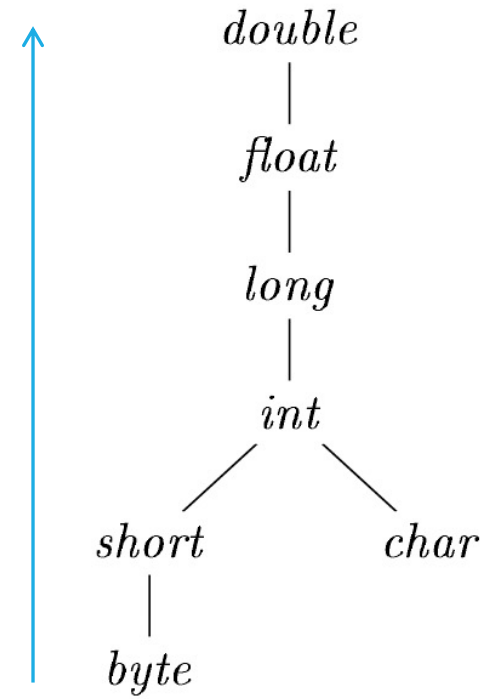
Type Analysis

Type Analysis

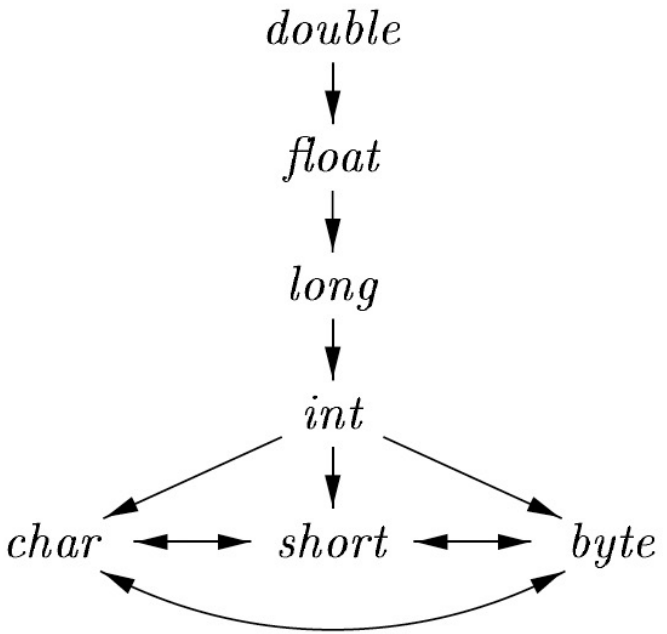
```
int main(){
    int a; char b; char c;
    scanf("%d%d", &a, &b);
    c = a + b;
    printf("%d", c);
    return 0;
}
```

1. What is the type of a + b?
2. Is it safe to assign the result to c?
3. Why type analysis in compilers?

Type Widening/Narrowing (Java)



(a) Widening conversions



(b) Narrowing conversions

Narrowing loses precision!

Type Widening

Compute Target Type

$$
\begin{aligned}
E \rightarrow E_1 + E_2 \quad \{ & E.type = max(E_1.type, E_2.type); \\
& a_1 = widen(E_1.addr, E_1.type, E.type); \\
& a_2 = widen(E_2.addr, E_2.type, E.type); \\
& E.addr = \textbf{new}\ Temp\,(); \\
& gen(E.addr\ '='\ a_1\ '+'\ a_2);\ \}
\end{aligned}
$$

Type Widening

$$
\begin{aligned}
E \rightarrow E_1 + E_2 \quad \{\ & E.type = max(E_1.type, E_2.type); \\
& a_1 = widen(E_1.addr, E_1.type, E.type); \\
& a_2 = widen(E_2.addr, E_2.type, E.type); \\
& E.addr = \textbf{new}\ Temp\,(); \\
& gen(E.addr\ '='\ a_1\ '+'\ a_2);\ \}
\end{aligned}
$$

Cast to Target Type

Type Widening

$$
\begin{aligned}
E \rightarrow E_1 + E_2 \quad \{\ & E.type = max(E_1.type, E_2.type); \\
& a_1 = widen(E_1.addr, E_1.type, E.type); \\
& a_2 = widen(E_2.addr, E_2.type, E.type); \\
& E.addr = \textbf{new}\ Temp\,(); \\
& gen(E.addr\ '='\ a_1\ '+'\ a_2);\ \}
\end{aligned}
$$

Type Widening

$$
\begin{aligned}
E \rightarrow E_1 + E_2 \quad \{\ & E.type = max(E_1.type, E_2.type); \\
& a_1 = widen(E_1.addr, E_1.type, E.type); \\
& a_2 = widen(E_2.addr, E_2.type, E.type); \\
& E.addr = \textbf{new}\ Temp\,(); \\
& gen(E.addr\ '='\ a_1\ '+'\ a_2);\ \}
\end{aligned}
$$

E_1.code => E_1.addr
E_2.code => E_2.addr
E.addr = E_1.addr + E_2.addr

E.code

Type Widening

$$
\begin{aligned}
E \rightarrow E_1 + E_2 \quad \{\ & E.type = max(E_1.type, E_2.type); \\
& a_1 = widen(E_1.addr, E_1.type, E.type); \\
& a_2 = widen(E_2.addr, E_2.type, E.type); \\
& E.addr = \textbf{new}\ Temp\,(); \\
& gen(E.addr\ '='\ a_1\ '+'\ a_2);\ \}
\end{aligned}
$$

E_1.code => E_1.addr
E_2.code => E_2.addr
E.addr = E_1.addr + E_2.addr

E.code

➡

E_1.code => E_1.addr **a_1 = (target type) E_1.addr**
E_2.code => E_2.addr **a_2 = (target type) E_2.addr**
E.addr = **a_1** + **a_2**

E.code

Type Checking

Type Checking

- Checking types for assignment
  - S → **id** = E　{ **id**.type vs. E.type }
- Similar cases for function overloading
  - E → **id**(ParamList)　{ compare **id**.type vs. ParamList.types }

PART II-4:
Translating Control Flows

Outline

- if-else statement; while statement
- break and continue
- Boolean expressions and short-circuit

Control Flow

***.code**: code generated by a nonterminal

S.next: next statement

B.true: true branch of a boolean expression

B.false: false branch of a boolean expression

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow \textbf{assign}$	$S.code = \textbf{assign}.code$
$S \rightarrow \textbf{if} \; (\; B \;) \; S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow \textbf{if} \; (\; B \;) \; S_1 \; \textbf{else} \; S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}' \; S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow \textbf{while} \; (\; B \;) \; S_1$	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}' \; begin)$
$S \rightarrow S_1 \; S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$

Control Flow

Production	Semantic Rules
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow$ **assign**	$S.code =$ **assign**$.code$
$S \rightarrow$ **if** (B) S_1	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow$ **if** (B) S_1 **else** S_2	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow$ **while** (B) S_1	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$

Control Flow

Production	Semantic Rules
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow$ **assign**	$S.code =$ **assign**$.code$
$S \rightarrow$ **if** (B) S_1	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow$ **if** (B) S_1 **else** S_2	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow$ **while** (B) S_1	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$

Control Flow

Branching

Production	Semantic Rules
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow$ **assign**	$S.code =$ **assign**$.code$
$S \rightarrow$ **if** (B) S_1	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow$ **if** (B) S_1 **else** S_2	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow$ **while** (B) S_1	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$

Control Flow

Branching

Sequencing

Production	Semantic Rules
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow$ **assign**	$S.code =$ **assign**$.code$
$S \rightarrow$ **if** (B) S_1	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow$ **if** (B) S_1 **else** S_2	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow$ **while** (B) S_1	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow$ **assign**	$S.code =$ **assign**$.code$
$S \rightarrow$ **if** (B) S_1	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow$ **if** (B) S_1 **else** S_2	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow$ **while** (B) S_1	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$

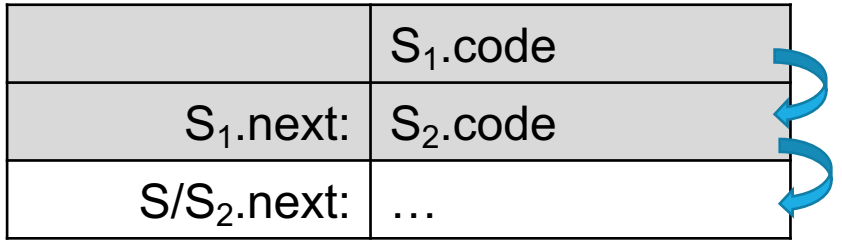
	S_1.code
S_1.next:	S_2.code
S/S_2.next:	…

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow$ **assign**	$S.code =$ **assign**$.code$
$S \rightarrow$ **if** (B) S_1	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow$ **if** (B) S_1 **else** S_2	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow$ **while** (B) S_1	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$

	S_1.code
S_1.next:	S_2.code
S/S_2.next:	…

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow$ **assign**	$S.code =$ **assign**$.code$
$S \rightarrow$ **if** (B) S_1	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow$ **if** (B) S_1 **else** S_2	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow$ **while** (B) S_1	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$

maintain a
linked list of stmts



PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow$ **assign**	$S.code =$ **assign**$.code$
$S \rightarrow$ **if** (B) S_1	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow$ **if** (B) S_1 **else** S_2	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow$ **while** (B) S_1	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$

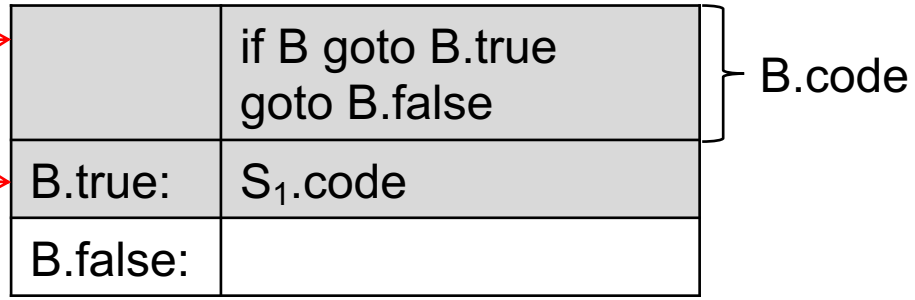
Example:

```
u = v * 2
w = u / 3
x = w + 1
y = x + 2
```

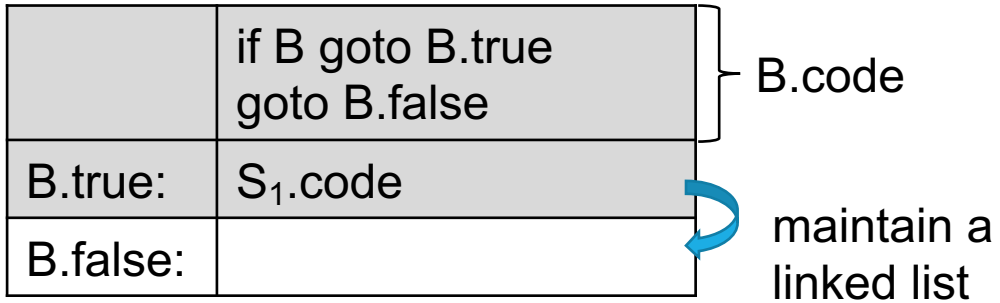
maintain a
linked list of stmts

	S_1.code
S_1.next:	S_2.code
S/S_2.next:	…

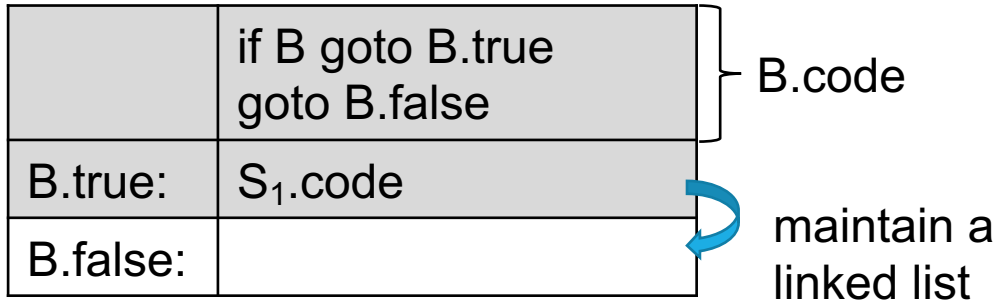
PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow$ **assign**	$S.code =$ **assign**$.code$
$S \rightarrow$ **if** (B) S_1	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow$ **if** (B) S_1 **else** S_2	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow$ **while** (B) S_1	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$



PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow$ **assign**	$S.code =$ **assign**$.code$
$S \rightarrow$ **if** (B) S_1	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow$ **if** (B) S_1 **else** S_2	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow$ **while** (B) S_1	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$



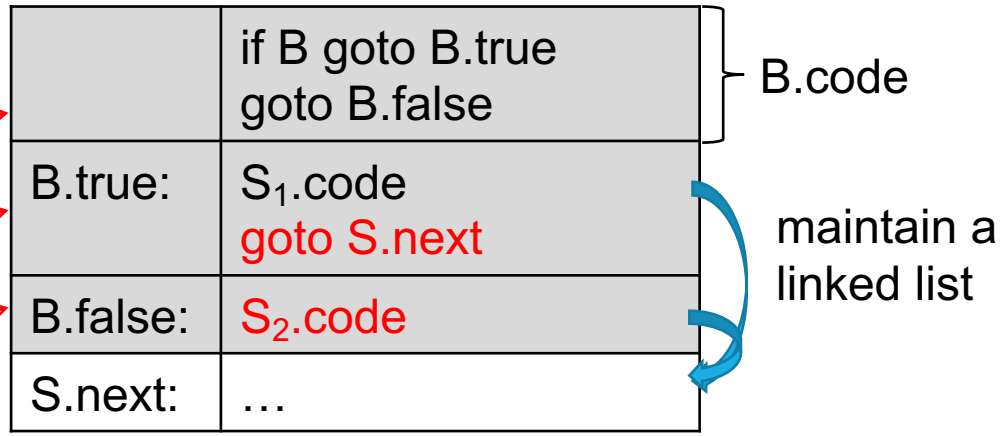
PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow$ **assign**	$S.code =$ **assign**$.code$
$S \rightarrow$ **if** (B) S_1	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow$ **if** (B) S_1 **else** S_2	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow$ **while** (B) S_1	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$



Example:

```
if (v)
    w = u / 3
    x = w + 1
y = x + 2
```

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow$ **assign**	$S.code =$ **assign**$.code$
$S \rightarrow$ **if** (B) S_1	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow$ **if** (B) S_1 **else** S_2	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow$ **while** (B) S_1	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$



PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow$ **assign**	$S.code =$ **assign**$.code$
$S \rightarrow$ **if** (B) S_1	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow$ **if** (B) S_1 **else** S_2	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow$ **while** (B) S_1	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$

Begin:	if B goto B.true goto B.false
B.true:	S_1.code
	goto Begin
B.false:	…

maintain a linked list

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \mid\mid label(S.next)$
$S \rightarrow$ **assign**	$S.code =$ **assign**$.code$
$S \rightarrow$ **if** (B) S_1	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow$ **if** (B) S_1 **else** S_2	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow$ **while** (B) S_1	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$

Begin:	if B goto B.true goto B.false
B.true:	S_1.code
	goto Begin
B.false:	…

maintain a linked list

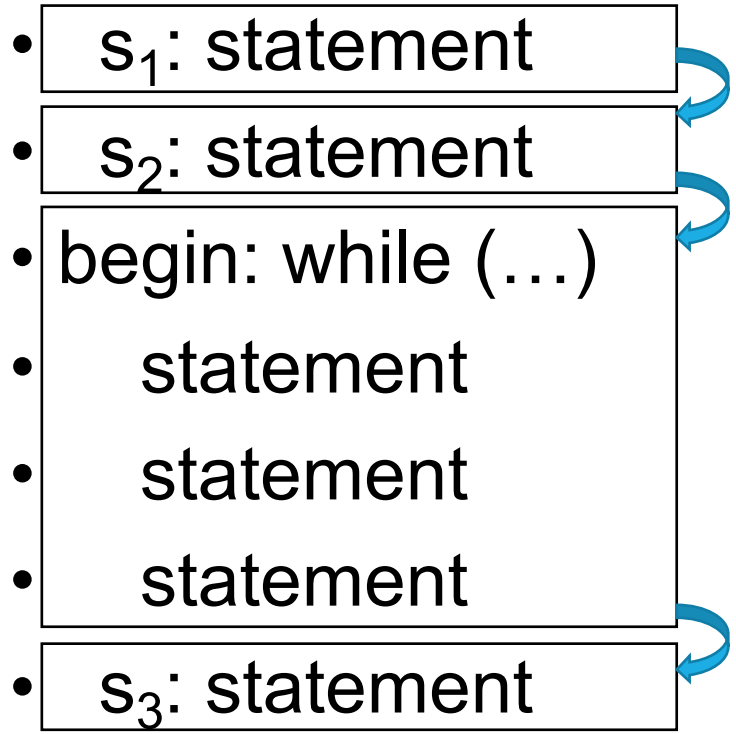
Example:

```
while (v)
    w = u / 3
    x = w + 1
y = x + 2
```

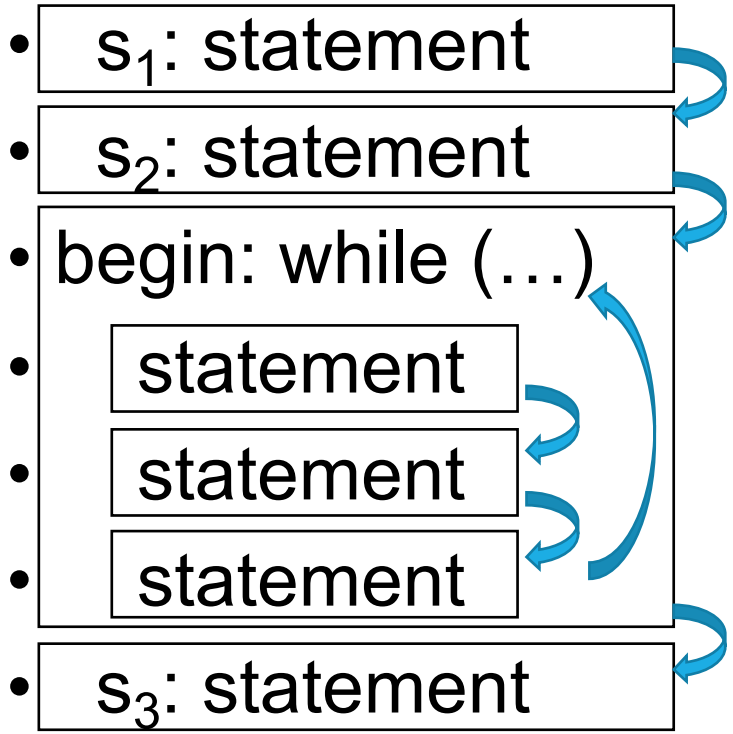
LinkedList of Statements

- s_1: statement
- s_2: statement
- begin: while (…)
- statement
- statement
- statement
- s_3: statement
- …

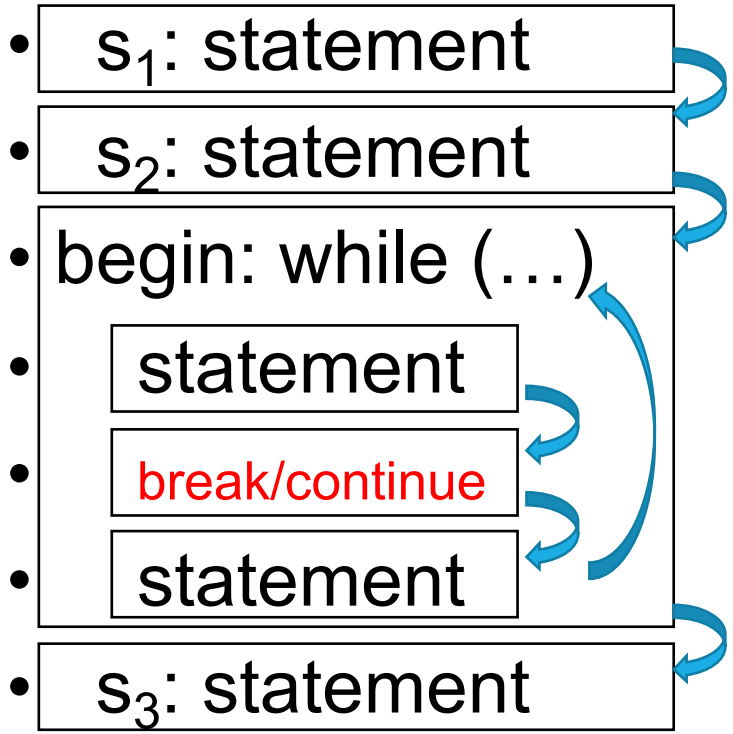
LinkedList of Statements



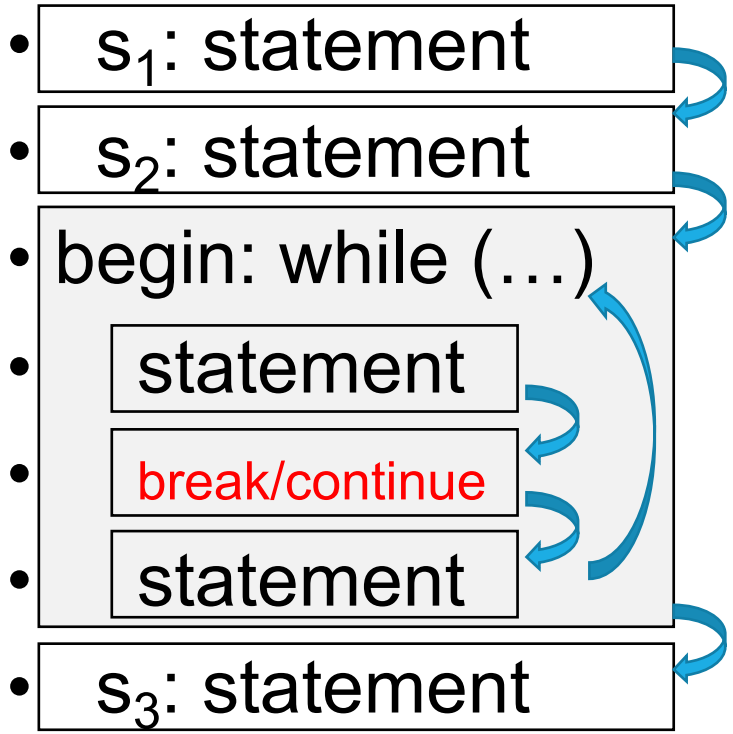
LinkedList of Statements



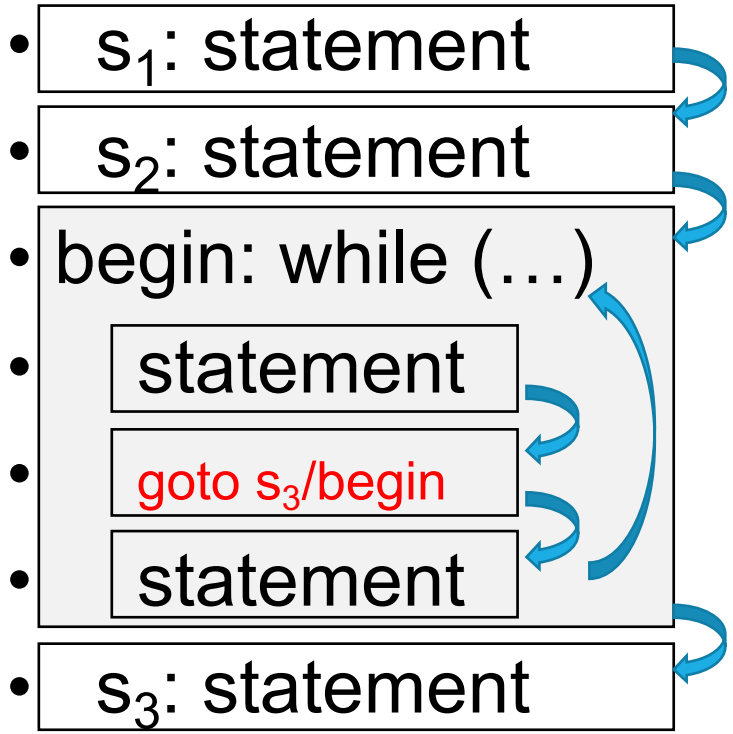
Break and Continue



Break and Continue



Break and Continue



Boolean Expressions

Boolean Expressions

$S \rightarrow \textbf{if}\ (\ B\ )\ S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid label(B.true) \mid\mid S_1.code$
$S \rightarrow \textbf{if}\ (\ B\ )\ S_1\ \textbf{else}\ S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ S.next)$ $\mid\mid label(B.false) \mid\mid S_2.code$
$S \rightarrow \textbf{while}\ (\ B\ )\ S_1$	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \mid\mid B.code$ $\mid\mid label(B.true) \mid\mid S_1.code$ $\mid\mid gen('\texttt{goto}'\ begin)$
$S \rightarrow S_1\ S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \mid\mid label(S_1.next) \mid\mid S_2.code$

	if B goto B.true goto B.false
B.true:	S_1.code
B.false:	

B.code

Boolean Expressions

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \ \texttt{||} \ B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \ \|\| \ label(B_1.false) \ \|\| \ B_2.code$
$B \rightarrow B_1 \ \&\& \ B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \ \|\| \ label(B_1.true) \ \|\| \ B_2.code$
$B \rightarrow \texttt{!} \ B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \ \textbf{rel} \ E_2$	$B.code = E_1.code \ \|\| \ E_2.code$ $\|\| \ gen('\texttt{if}' \ E_1.addr \ \textbf{rel}.op \ E_2.addr \ '\texttt{goto}' \ B.true)$ $\|\| \ gen('\texttt{goto}' \ B.false)$
$B \rightarrow \textbf{true}$	$B.code = gen('\texttt{goto}' \ B.true)$
$B \rightarrow \textbf{false}$	$B.code = gen('\texttt{goto}' \ B.false)$

Boolean Expressions

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \mid\mid B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.false) \mid\mid B_2.code$
$B \rightarrow B_1 \;\&\&\; B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.true) \mid\mid B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1$ **rel** E_2	$B.code = E_1.code \mid\mid E_2.code$ $\mid\mid gen('\texttt{if}'\ E_1.addr\ \textbf{rel}.op\ E_2.addr\ '\texttt{goto}'\ B.true)$ $\mid\mid gen('\texttt{goto}'\ B.false)$
$B \rightarrow$ **true**	$B.code = gen('\texttt{goto}'\ B.true)$
$B \rightarrow$ **false**	$B.code = gen('\texttt{goto}'\ B.false)$

E_1.code => E_1.addr
E_2.code => E_2.addr
If E_1.addr ***rel*** E_2.addr goto B.true
goto B.false

Boolean Expressions

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \mid\mid B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.false) \mid\mid B_2.code$
$B \rightarrow B_1 \;\&\&\; B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.true) \mid\mid B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1$ **rel** E_2	$B.code = E_1.code \mid\mid E_2.code$ $\mid\mid gen(\texttt{'if'}\ E_1.addr\ \textbf{rel}.op\ E_2.addr\ \texttt{'goto'}\ B.true)$ $\mid\mid gen(\texttt{'goto'}\ B.false)$
$B \rightarrow$ **true**	$B.code = gen(\texttt{'goto'}\ B.true)$
$B \rightarrow$ **false**	$B.code = gen(\texttt{'goto'}\ B.false)$

E_1.code => E_1.addr
E_2.code => E_2.addr
If E_1.addr ***rel*** E_2.addr goto B.true
goto B.false

Boolean Expressions

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \mid\mid B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.false) \mid\mid B_2.code$
$B \rightarrow B_1 \;\&\&\; B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.true) \mid\mid B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1$ **rel** E_2	$B.code = E_1.code \mid\mid E_2.code$ $\mid\mid gen('\texttt{if}'\ E_1.addr\ \textbf{rel}.op\ E_2.addr\ '\texttt{goto}'\ B.true)$ $\mid\mid gen('\texttt{goto}'\ B.false)$
$B \rightarrow$ **true**	$B.code = gen('\texttt{goto}'\ B.true)$
$B \rightarrow$ **false**	$B.code = gen('\texttt{goto}'\ B.false)$

Example:

```
if (v + 1 > z + 2)
    w = u / 3
```

E_1.code => E_1.addr
E_2.code => E_2.addr
If E_1.addr ***rel*** E_2.addr goto B.true
goto B.false

Boolean Expressions

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \mid\mid B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.false) \mid\mid B_2.code$
$B \rightarrow B_1 \;\&\&\; B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.true) \mid\mid B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \;\mathbf{rel}\; E_2$	$B.code = E_1.code \mid\mid E_2.code$ $\mid\mid gen('\texttt{if}'\ E_1.addr\ \mathbf{rel}.op\ E_2.addr\ '\texttt{goto}'\ B.true)$ $\mid\mid gen('\texttt{goto}'\ B.false)$
$B \rightarrow \mathbf{true}$	$B.code = gen('\texttt{goto}'\ B.true)$
$B \rightarrow \mathbf{false}$	$B.code = gen('\texttt{goto}'\ B.false)$

	B_1.code
B_1.true:	B_2.code
B_1.false	

Boolean Expressions

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \mid\mid B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.false) \mid\mid B_2.code$
$B \rightarrow B_1 \;\&\&\; B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.true) \mid\mid B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \;\mathbf{rel}\; E_2$	$B.code = E_1.code \mid\mid E_2.code$ $\mid\mid gen('\texttt{if}'\ E_1.addr\ \mathbf{rel}.op\ E_2.addr\ '\texttt{goto}'\ B.true)$ $\mid\mid gen('\texttt{goto}'\ B.false)$
$B \rightarrow \mathbf{true}$	$B.code = gen('\texttt{goto}'\ B.true)$
$B \rightarrow \mathbf{false}$	$B.code = gen('\texttt{goto}'\ B.false)$

	If B_1.addr goto B_1.true goto B_1.false
B_1.true:	B_2.code
B_1.false	

Boolean Expressions

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \mid\mid B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.false) \mid\mid B_2.code$
$B \rightarrow B_1 \;\&\&\; B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.true) \mid\mid B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \;\mathbf{rel}\; E_2$	$B.code = E_1.code \mid\mid E_2.code$ $\mid\mid gen('\texttt{if}'\ E_1.addr\ \mathbf{rel}.op\ E_2.addr\ '\texttt{goto}'\ B.true)$ $\mid\mid gen('\texttt{goto}'\ B.false)$
$B \rightarrow \mathbf{true}$	$B.code = gen('\texttt{goto}'\ B.true)$
$B \rightarrow \mathbf{false}$	$B.code = gen('\texttt{goto}'\ B.false)$

	If B_1.addr goto B_1.true goto B_1.false
B_1.true:	B_2.code
B_1.false	

Example:

```
if (a > b && c > d)
    w = u / 3
```

Boolean Expressions

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \mid\mid B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.false) \mid\mid B_2.code$
$B \rightarrow B_1 \;\&\&\; B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.true) \mid\mid B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \textbf{ rel } E_2$	$B.code = E_1.code \mid\mid E_2.code$ $\mid\mid gen('\texttt{if}'\ E_1.addr\ \textbf{rel}.op\ E_2.addr\ '\texttt{goto}'\ B.true)$ $\mid\mid gen('\texttt{goto}'\ B.false)$
$B \rightarrow \textbf{true}$	$B.code = gen('\texttt{goto}'\ B.true)$
$B \rightarrow \textbf{false}$	$B.code = gen('\texttt{goto}'\ B.false)$

	If B_1.addr goto B_1.true goto B_1.false
B_1.true:	B_2.code
B_1.false	

Example:

```
if (a > b && c > d)
    w = u / 3
```

Short-circuit!

Boolean Expressions

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \mid\mid B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.false) \mid\mid B_2.code$
$B \rightarrow B_1 \;\&\&\; B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \mid\mid label(B_1.true) \mid\mid B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \;\mathbf{rel}\; E_2$	$B.code = E_1.code \mid\mid E_2.code$ $\mid\mid gen('\texttt{if}'\ E_1.addr\ \mathbf{rel}.op\ E_2.addr\ '\texttt{goto}'\ B.true)$ $\mid\mid gen('\texttt{goto}'\ B.false)$
$B \rightarrow \mathbf{true}$	$B.code = gen('\texttt{goto}'\ B.true)$
$B \rightarrow \mathbf{false}$	$B.code = gen('\texttt{goto}'\ B.false)$

	If B_1.addr goto B_1.true goto B_1.false
B_1.true:	B_2.code
B_1.false	

Example:

```
if (a > b && c > d)
    w = u / 3
```

Short-circuit!

Others

- Optimizations:
  - Avoid unnecessary gotos
  - Backpatching
- Translation for:
  - Switch statement
  - Function / Function call

- ***Read the dragon book, Chapter 6.***

Summary

- **Part I: Three-Address Code**
  - Definition and Implementations of Three-Address Code
- **Part II: Generation of Intermediate Representation**
  - Syntax-Directed Translation
  - Generating IR for Variable Declaration/Expressions/Statements
  - Generating IR for Control Flows

THANKS!